Mergeable Replicated Data Types in Sal:
Metatheory, the Embedded-Chain Family, and Sequential Specifications

Sal / FP Launchpad (KC + Claude)

July 20, 2026

one document consolidating three earlier notes (of July 7, July 14, and July 16–17, 2026), extended July 18–19 with the runtime arc (virtual LCAs, commit GC, the stability VC) and the storage layer (re-coding, compaction, the run table), and July 20 with the verified peer-to-peer runtime, a cross-system performance study, and a push-button SMT layer

Abstract

Mergeable replicated data types (MRDTs) resolve concurrent divergence by a *three-way* merge $\mathsf{mergeL}(l, a, b)$: the two divergent states a, b are reconciled relative to their lowest common ancestor l in a git-like version DAG. This document is the consolidated account of the Sal verification effort. Part I develops the mechanized metatheory: what an MRDT is and how to describe one; why the published soundness induction fails (a fully LCA-legal defeater) and why no lattice-style contract can replace it; the corrected architecture of canonical states and a ternary Join Lemma; the eight verification conditions that make it true, with their adequacy theorem and the discharged production catalogue; the conditioned generalization for state-dependent data types; and the factored discharge route (a generic honest-reachability metatheorem consuming per-configuration join lemmas) that the newest instances use, with the mergeable queue as its flagship. Part II presents the embedded-chain RGA, the repository's canonical sequence datatype: a tombstone-free replicated list whose sort keys are immutable birth chains carried as entropy-coded coordinates, with the encoding contract and entropy law, kernel-checked RA-linearizability parametric in the code, the compaction theorem identifying its reads with the published tombstoned RGA's, and the sided generalization that makes two-sidedness a parameter and hosts the Fugue/FugueMax arc; its extension closes the Weidner–Kleppmann maximal-non-interleaving theorem (the first mechanized such result, we believe), proves a representation impossibility naming the exact information a delete must leave behind, and adds a measured storage layer (re-coding at settled cuts, a verified concrete compaction, the run table). Part I is correspondingly extended with the runtime metatheory: virtual LCAs for criss-cross merges, commit GC with a proved keep set, and a stability VC under which state compaction is observationally invisible. Part III records the sequential-specification campaign: for every production RDT in Sal, a kernel-checked theorem that the fold of any sequentially honest single-replica history equals the output of an independent naive sequential program, together with three machine-checked refutations giving a two-axis separation between do-faithfulness and merge-faithfulness. A final part turns the design into a running system: a verified peer-to-peer MRDT runtime, datatype-parametric and content-addressed, whose garbage collectors are the executable form of the GC-safety and stability theorems and whose git-backed store persists the commit DAG; a performance section measures the shipped artifact against Yjs, Automerge, Loro, and list-positions on the standard editing-trace corpus, honest about a three-to-four-orders apply-latency loss to an interface-inherited state copy and about the design's categorical storage win, the only save that shrinks on delete; and an SMT translation-validation layer that makes the flat verification conditions push-button. Everything stated as a theorem is mechanized in Lean 4 with

zero `sorry`s in the cited chains; the negative results are machine-checked countermodels, and every performance number is a labeled run of checked-in scripts.

How this document is organized. The four parts are followed by a single consolidated open-questions section and a single mechanization map. Part I is the metatheory and can be read alone; Part II depends on Part I's signature and its factored discharge route (§13); Part III depends on both, since its refutations calibrate what Part I's convergence theorems do and do not certify; Part IV is the systems pillar, a verified peer-to-peer runtime (§37) with a cross-system performance study (§38) and a push-button SMT layer (§39), and it depends on the embed kernel of Part II and the GC and stability metatheory of Part I. The document consolidates three earlier notes (the metatheory note of July 7, the embedded-chain design note of July 14, and the sequential-specification campaign note of July 16–17); reused material keeps its original wording, duplicated material appears once, and the previously separate open-question lists and mechanization maps are merged. A July 18–19 extension adds, to Part I, the runtime-facing metatheory (virtual LCAs for criss-cross merges, §14; commit GC and its keep set, §15; the stability VC for verified compaction callbacks, §16) and, to Part II, the closed maximal-non-interleaving theorem (§22), the chain-price impossibility (§23), and the storage layer with its measurements (§24). A July 20 extension adds Part IV (the verified runtime, its performance study, and the SMT layer) and closes the storage layer's mechanization (spine fusion, the merge-remap merge clause, multi-epoch composition, and the one-sided run table), leaving one shared honesty-rebasing lemma as the single remaining single-replica compaction obligation. One editorial rule governs the whole document: the *rehoming* tombstone-free RGA, an earlier design demoted after its sequential refutation, receives no narrative treatment here; it appears only as the campaign's named countermodel (Part III), in one-line contrasts where the separation needs the losing side, and once as a generality stress test (§14), where its role is again that of the hardest available instance rather than a design of interest.

Contents

Part I
The metatheory

1 Introduction

An MRDT execution looks like a git history: replicas fork, apply operations locally, and merge. Unlike a state-based CRDT, whose binary merge must reconcile two states with no memory of their common past, an MRDT's merge receives a third argument — the state at the *lowest*

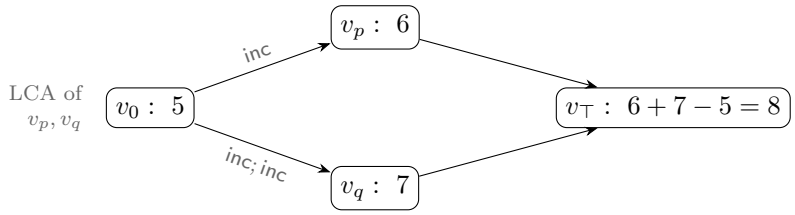


Figure 1: The counter MRDT. Both branches inherit the 5 from the fork point; the LCA argument lets the merge count each branch's *delta* exactly once. No binary merge of 6 and 7 alone can recover 8: the information "how much is shared" lives in the DAG, and the LCA argument is how the data type reads it.

common ancestor (LCA) of the two versions being merged — and may use it to compute *deltas*: what each side did since the fork. The paradigmatic example is the counter,

$$\mathsf{mergeL}(l, a, b) \;=\; a + b - l,$$

which adds the two sides' increments-since-l instead of double-counting their common past (Figure 1).

The Neem methodology (OOPSLA 2025) promises a clean division of labour: the data-type designer discharges a fixed catalogue of equations over do, mergeL and the conflict-resolution relation rc — mostly by SMT — and a soundness meta-theorem, proved once, converts those VCs into *RA-linearizability* of every reachable configuration. This part documents what a Lean 4 mechanization of that meta-theorem, in the full **ternary, version-DAG** setting, found: the meta-theorem's central induction is unsound as written, and for LCA-sensitive data types several of the catalogue's merge VCs are false in principle when quantified over all states. It also develops the corrected metatheory: a set-relative linearization order, canonical states $\sigma(E)$ (each event set determines a unique state), and a ternary *Join Lemma* whose induction consumes a small, contextual VC surface. Its contributions:

1. **The paper's LCA lemma is true but its proof has a gap** (§4.1): $L(v_\ell) = L(v_1) \cap L(v_2)$ holds, but only as a *reachability invariant* of the store, whose proof needs a generator-uniqueness induction the paper silently assumes. We mechanize the transition system and the invariant.

2. **The soundness proof has a fully LCA-legal defeater** (§4.2): a reachable configuration of a legal add-wins MRDT — its final merge sitting at an honest LCA — on which every bottom-up peel of the paper's derivation rules is stuck. The meta-theorem must be rebuilt, not patched.

3. **The lattice contract does not transfer; a delta contract does** (§6): the lattice laws are equations over *free* tuples — the wrong instances for an LCA-aware merge. Side-idempotence $\mathsf{mergeL}(l, a, a) = a$ with l free fails for the counter $(2a - l)$, but its only DAG-honest instance has $l = a$ (the LCA of a version with itself *is* that version), where it holds trivially; and when two *distinct* histories reach equal side states, $2a - l$ is the *correct* merge — both branches' increments count — not an idempotence failure. The induced orders degenerate, and the honest associativity law has *four distinct LCAs* and holds only on feasible tuples. The moral: the algebra of an LCA-aware merge is an algebra of *causal*

histories, not of concrete states. What replaces the lattice laws is a two-law *delta contract* — redistribution identities in which the LCA slot does, by arithmetic, the job idempotence does order-theoretically for CRDTs.

4. **Eight VCs suffice** (§7–§8): three relational VCs on rc (one of them carrying a different-replica guard — same-replica pairs are ordered by program order, and demanding rc cover them is unsatisfiable for real data types), merge symmetry, three feasibility-conditioned delta laws, and one contextual *causal-delta* equation. The adequacy theorem is per *version*: every version in the store, LCAs included, linearizes.

5. **The VC set is validated on the production catalogue** (§9): the flat production MRDTs shipped in Sal — OR-set, an optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, a tombstone RGA, Peritext, the multi-valued register and the add-wins priority queue — are discharged end-to-end, kernel-checked; the sequence data types with physical deletion go through the conditioned framework and its factored discharge route (§10–§13, Part II). The sweep produces an empirical *class map* of which data types need which strength of contract, with two surprises: tombstones are a metatheoretic trivializer, and commutativity class and delta class are independent axes.

6. **The metatheorem factors** (§13): conditioning splits as a generic honest-reachability metatheorem times a per-datatype join discharge (`HonestReach`, `JoinLemma3At`), with a uniform honesty shape (a client-checkable predicate holding of each event at the fold of its causal past). Three join species are observed in production (conditioned commutation, flat commutation, direct witness), and the mergeable queue, whose conflict graph admits no rc assignment at all, is the flagship of the direct-witness species: Peepul's merge is its own linearization certificate. The same route carries Part II's embedded-chain RGA.

7. **The model meets its runtime** (§14–§16): the criss-cross gate on Merge is lifted by a recursive virtual-LCA rule whose synthesized base provably carries exactly the intersection event set, with no new per-datatype VC for join-lemma datatypes; commit GC is proved safe against a keep set read off pairwise head intersections, with a countermodel showing head-sync is load-bearing; and state-level compaction (the GC callback into the datatype) gets a six-clause stability VC whose metatheorem makes compaction observationally invisible, with the OR-set as first instance and a refuted naive gate as the shaping countermodel.

Everything stated as a theorem below is mechanized with zero `sorry`s; §41 maps statements to Lean names and files (foundations under `Framework/`, metatheorems under `Metatheory/`, negative results under `Refutations/`, the per-RDT instances under `MRDT_Instances/`, the investigation record under `Development/`; each cited theorem kernel-checked).

2 Mergeable replicated data types, by definition and example

Data type and implementation. A replicated data type of *type* $\tau = \langle O_\tau, Q_\tau, \mathit{Val}_\tau \rangle$ exposes a set of update operations O_τ, a set of query operations Q_τ, and a domain of query return values Val_τ. An *MRDT implementation* of τ is a tuple

$$\mathcal{D} \;=\; \langle \Sigma,\ \sigma_0,\ \mathsf{do},\ \mathsf{mergeL},\ \mathsf{query},\ \mathsf{rc} \rangle,$$

where Σ is the space of replica states with initial state σ_0; $\mathsf{do} : \Sigma \times \mathcal{E} \to \Sigma$ applies an *event* — an update-operation instance $e = (t, r, o)$ carrying a globally unique timestamp t, its origin replica r, and the operation $o \in O_\tau$; $\mathsf{mergeL} : \Sigma^3 \to \Sigma$ is the three-way merge, whose first argument is the state at the lowest common ancestor of the two versions being merged; $\mathsf{query} : \Sigma \times Q_\tau \to \mathit{Val}_\tau$ reads a state; and $\mathsf{rc} \subseteq O_\tau \times O_\tau$ is the *conflict-resolution* relation, consulted to order concurrent non-commuting operations (it is the data-type designer's declaration of who wins a race). Queries never change state and play no role in the metatheory; the theory below reads the implementation as $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$.

Describing an MRDT: the counter. To describe an MRDT one gives the five components and, when operations can race, says who wins. The increment-only counter of Figure 1 is the smallest complete example:

$$\Sigma = \mathbb{Z}, \qquad \sigma_0 = 0, \qquad \mathsf{do}(\sigma, (t, r, \mathsf{inc})) = \sigma + 1, \qquad \mathsf{mergeL}(l, a, b) = a + b - l, \qquad \mathsf{rc} = \emptyset.$$

All operations commute, so rc is empty — yet the merge genuinely uses its LCA argument: $a + b$ alone would double-count the history shared by the two branches, and the $-\,l$ subtracts it exactly once. This "read the shared past off the LCA" pattern recurs throughout the catalogue.

A data type with races: the OR-set. The add-wins observed-remove set stores tagged elements: add inserts a tag unique to the adding event, rem deletes exactly the tags it has *seen*. Sequentially, a remove after an add deletes it; concurrently the designer declares the add the winner: $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$. The merge keeps a tag iff both sides have it (nobody deleted it) or some side added it since the fork:

$$\mathsf{mergeL}(l, a, b) \;=\; (a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

The LCA here plays the role a tombstone set plays for the CRDT OR-set: deletion is absence *relative to* l, and the state needs no graveyard.

The generalized signature. Both examples' operations make sense on *every* state: any state can be incremented, any tag can be added. Such data types are *flat*, and for them the metatheory of §3–§9 quantifies its VCs over all states. But a data type whose operations carry *preconditions* — insert under a node that must exist, delete a node that must be live — breaks that quantification: its commutation equations are simply false at nonsense states. The full signature therefore extends the tuple with three declarations (`ConditionedMRDTSig`; formalized in §11):

$$\mathcal{D} \;=\; \langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc}, \;\mathsf{Inv}, \;\mathsf{app} \rangle \quad \text{together with} \quad \approx \;\subseteq \Sigma \times \Sigma,$$

where Inv is a state invariant (a shape over-approximation of reachability), app an applicability predicate (when an event is sensible at a state), and $\approx$ an observational equivalence (what queries can distinguish; representation residue is quotiented away). Commutation is then required only at Inv-states where both events are applicable. Flat data types take $\mathsf{Inv} = \mathsf{app} = \top$ and $\approx \;=\; =$, collapsing the general signature to the plain one — one framework, of which §3–§9 is the flat instance and §10 onwards the general case.

The running hard example: sequences with physical deletion. The datatype class that forces the general signature is the replicated list underlying collaborative text editing. Characters are nodes identified by timestamps, each inserted *after* an anchor node; production RGAs keep deleted nodes as *tombstones* because later concurrent inserts may still anchor on them, and a *tombstone-free* design deletes physically. Its operations are precondition-laden: an insert names an anchor that must exist, a delete names a live node, and the flat update layer quantifies its commutation VCs over every state, including nonsense states no execution produces, where those equations are false. Worse, what makes reachable-state commutation true is a property of the *execution that generated the operation* (the issuer saw the anchor live), not of the state the operation later meets, so it is not expressible as a $\forall\sigma$ equation over the signature alone. This is the reason the general signature exists: Inv and app name the sensible states and operations, an honesty discipline makes generation-time facts available to the proofs, and $\approx$ absorbs representation residue. Part II presents the embedded-chain RGA, the sequence datatype that holds the repository's canonical seat, together with its end-to-end discharge through the factored route of §13; a predecessor design that carried recorded ancestor paths and rehomed orphans on delete was proved RA-linearizable in the same framework and was later demoted by its sequential refutation (Part III).

3 The ternary setting

This section fixes the model: the data-type signature, the replicated store and its transitions, and the specification the metatheorem must deliver. The definitions are the paper's, with one correction flagged where it occurs.

Signature. An MRDT is $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc}\rangle$ as in §2, read flat for now ($\mathsf{Inv} = \mathsf{app} = \top$, $\approx\ =\ =$; the general case resumes in §10). We write $e(\sigma)$ for $\mathsf{do}(\sigma, e)$, and $e_1 \rightleftarrows e_2$ when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for every σ.

Executions. A configuration carries a *store*: a DAG of versions, each holding a state and the set of events that produced it, with per-replica heads. Transitions fork a replica, apply a fresh event at a head, query, or **merge**: pick heads v_1, v_2, find their LCA v_ℓ in the DAG, and install a new version with state $\mathsf{mergeL}(\sigma_{v_\ell}, \sigma_{v_1}, \sigma_{v_2})$ and event set $L(v_1) \cup L(v_2)$. Visibility $\xrightarrow{\mathsf{vis}}$ records what each event's issuing replica had seen.

RA-linearizability, per version. Write $e_1 \parallel e_2$ when neither $e_1 \xrightarrow{\mathsf{vis}} e_2$ nor $e_2 \xrightarrow{\mathsf{vis}} e_1$ (concurrency).

Definition 3.1 (linearization order, set-relative)**.** *For an event set E, the order lo^E puts e_1 before e_2 iff*

$$\big(e_1 \xrightarrow{\mathsf{vis}} e_2 \ \wedge\ e_1 \not\rightleftarrows e_2\big) \ \vee\ \big(e_1 \parallel e_2 \ \wedge\ e_1 \xrightarrow{\mathsf{rc}} e_2 \ \wedge\ \neg\,\exists e_3 \in E.\ e_2 \xrightarrow{\mathsf{vis}} e_3 \wedge e_2 \not\rightleftarrows e_3\big)\,:$$

a visible non-commuting pair is ordered by visibility; a concurrent pair whose conflict rc *resolves is ordered by* rc *— unless e_2 is already absorbed (overwritten) by a later non-commuting event of E. The paper's order* lo *is the variant whose absorber clause ranges over the whole configuration; since a* lo*-edge asserts no absorber anywhere, $\mathsf{lo} \subseteq \mathsf{lo}^E$ pointwise, for every E.*

Definition 3.2 (RA-linearizability)**.** *A list $\pi = e^1 e^2 \cdots e^n$ witnesses a version holding state s and event set E when (i) π enumerates E without repetition; (ii) π extends lo: $i < j$ implies $\neg(e^j \mathsf{lo} e^i)$; and (iii) π folds to the state: $e^n(\cdots e^2(e^1(\sigma_0))\cdots) = s$. A configuration is RA-linearizable when every version in the store — not only the replica heads; LCAs are historical versions and the merge reads them — admits a witness. In the general signature the fold clause (iii) is read up to the data type's observational equivalence, $e^n(\cdots e^1(\sigma_0)\cdots) \approx s$ — the same definition with its equality parameter generalized; flat data types instantiate $\approx$ $=$ and this section's form is exactly that instance.* `IsRALinearizable3`, `IsRALinearizable3Eq`

Why a set-relative order? One definitional correction is forced at the outset: for every internal use, the absorber clause must range over the event set of *the version being linearized*, not over the whole configuration — an event can gain absorbers from another branch, silently cancelling an order edge inside a version whose own state depends on it (the defeater of §4.2 realizes exactly this). We therefore construct witnesses respecting lo^E throughout; since $\mathsf{lo} \subseteq \mathsf{lo}^E$, every such witness also extends the paper's order, and Definition 3.2 is delivered exactly as the paper states it.

Running examples. Three data types exercise every corner of the theory:

- the **counter**: $\Sigma = \mathbb{Z}$, $\mathsf{inc}(\sigma) = \sigma + 1$, $\mathsf{mergeL}(l, a, b) = a + b - l$; all operations commute, but the merge genuinely uses its LCA;
- the **OR-set** (tagged add-wins set): elements are tagged by the adding event's timestamp; rem deletes the tags it has seen; concurrently, $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$ resolves in favour of the add. The merge keeps an element iff *both* sides have it or *some* side added it since the fork:

$$\mathsf{mergeL}(l, a, b) \;=\; (a \cap b) \cup (a \setminus l) \cup (b \setminus l)$$

  — pointwise, "if the tag is in l, require it in both branches (nobody deleted it); otherwise one branch suffices (somebody added it)." Here the LCA acts as a *tombstone set*: deletion is represented by *absence relative to l*, so the state itself needs no graveyard;
- the **enable-wins flag**: per-replica pairs (counter, flag); enabling bumps your own counter and sets your flag; disabling clears all flags; the merge compares each replica's counter against the LCA's to decide whether an enable "happened since the fork" on that replica. Its merge *compares* counters rather than accumulating sets — and that difference will be visible in the metatheory (§9).

4 Three facts the metatheory must respect

Before building anything, we record two boundary facts about the paper's development: its store-level LCA lemma is true but under-proved, and its soundness induction is refuted by a fully legal execution. Together they fix the shape of the corrected metatheory — resting on a mechanized store invariant (§4.1) and built around the join rather than the sides (§4.2).

4.1 The LCA lemma is a reachability invariant — with a gap in the published proof

Everything above silently used:

Lemma 4.1 (LCA lemma)**.** *In every reachable configuration, if v_ℓ is an LCA of versions v_1, v_2 in the store's DAG, then $L(v_\ell) = L(v_1) \cap L(v_2)$.*

This is what lets merge VCs be stated over *event sets*: the state the merge receives as l is the canonical state of exactly the intersection of the two sides' histories. The lemma is true — but it is a *global induction over the reachable store*, not a local fact. Its mechanized proof threads a store invariant (all parents allocated; event sets monotone along edges; and an *origin* invariant — each event appears first at a unique generator version) through every transition; the published proof asserts the generator-uniqueness step without the induction that sustains it. An erratum-sized gap: the statement survives, the proof as written does not. (A related boundary fact: *criss-cross* merges can defeat the *existence* of an LCA — they gate merge-enabledness, not the lemma. §14 lifts the gate.)

4.2 A fully LCA-legal execution defeats the soundness proof

The generic half of the paper's contract is a *bottom-up linearization* theorem: given linearization witnesses for the two sides of a merge, construct one for the merged version by repeatedly *peeling* a final event through mergeL using the catalogue's interchange VCs. One might hope the version DAG's discipline — every merge sitting at an honest LCA — keeps the peel supplied with peelable events. It does not: one staging replica suffices to realize the intersection of two histories as an honest version, and the merge induction is stuck.

Example 4.2 (the defeater)**.** *The data type is a one-key add-wins skeleton with tagged adds: $\sigma = (A, D) \in 2^{\mathbb{T}} \times 2^{\mathbb{T}}$ with live set $A \setminus D$; add_t inserts its tag into A; rem kills everything it currently sees ($D := A \cup D$ — the state-dependence is what makes $\mathsf{add} \not\rightleftarrows \mathsf{rem}$); concurrently $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$ (add wins); the merge is componentwise union, ignoring its LCA argument. A legal MRDT. Replica p adds (A_p), replica q adds (A_q). A staging replica s merges both one-add versions, creating v_s with $L(v_s) = \{A_p, A_q\}$. Now p removes (R_p, seeing only A_p) and then merges with v_s; symmetrically q removes (R_q) and merges with v_s. Figure 2 shows the DAG. The final merge of the two heads finds $\mathrm{LCA} = v_s$, and $L(v_s) = \{A_p, A_q\} = L(head_p) \cap L(head_q)$ — the configuration is fully legal. Now count last events. p's head lives on exactly A_q's tag, so a state-correct witness must run R_p before A_q, and neither R_p nor A_p can come last: every linearization of p's head ends in A_q. Symmetrically, every linearization of q's head ends in A_p. Yet in the merged version every tag is dead, so its linearizations must end in one of the removes. The events the induction must peel are buried in the very sides that own them; no ordering of the paper's bottom-up rules applies.*

The two removes commute ($\mathsf{rem} \not\rightleftarrows \mathsf{add}$ but $\mathsf{rem} \rightleftarrows \mathsf{rem}$), which invites a particular rescue attempt: the paper's BottomUp-2-OP rule (`inter_lca_2op`) admits the commuting case and could, one hopes, peel a remove last. It cannot, for a reason that is a one-line state fact. To peel o last from a side, that side's state must be an o-image — of the form $\mathsf{do}(\sigma', o)$ with o outermost. But in the add-wins skeleton *every* rem-output has empty live set ($\mathsf{rem}(A, D) = (A, A \cup D)$), whereas both heads carry a live tag (head_p lives on A_q, head_q on A_p — the opposite add, merged in *after* the local remove). So neither head is a rem-image, and no bottom-up rule can extract a remove from it. The tempting linearization $[A_p, R_p, A_q, R_q]$ is a valid witness of the *merged* version, but its restriction to head_q folds to the all-dead state, not head_q's actual state (live A_p): it is not assembled from a state-correct side witness, which is exactly what the induction requires. All three facts are mechanized (`InterLca2op_Defeater_Arbiter`: `awset_rem_output_empty`, `no_inter_lca_2op_rem_peel_of_defeater`, `crack1_witness`).

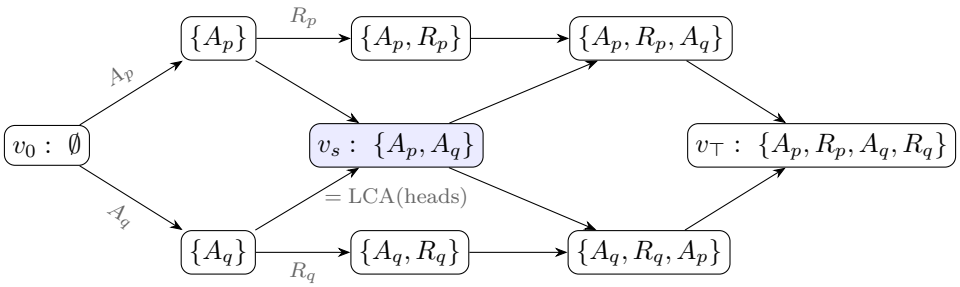


Figure 2: The defeater (Example 4.2). Nodes are versions labelled by their event sets; the shaded version v_s, created by a staging replica, realizes $L(\mathrm{head}_p) \cap L(\mathrm{head}_q)$ as an honest store version, so the final merge is LCA-legal. The removes R_p, R_q — the only events a bottom-up derivation may peel last — are buried mid-history on their own sides.

The consequence for the design: the metatheorem cannot be patched peel-by-peel — it must be rebuilt on canonical states and a Join Lemma, with merge-side VCs that are statements *about the join*, not about how either side's state factors.

5 Canonical states and the ternary Join Lemma

We now rebuild — new proof architecture, and with it a *new set of verification conditions* to replace the paper's catalogue. The lesson of the defeater is architectural: witnesses for a merged version cannot be manufactured out of witnesses for its sides, so the corrected metatheory must not traffic in witness lists at all. The rebuild has two layers. The *update layer* makes states, not sequences, the objects of the induction: it assigns to every event set E a single well-defined state, its *canonical state* (Definition 5.2 below). The *merge layer* is then one equation between canonical states — the ternary *Join Lemma* — and establishing it from per-data-type VCs becomes the entire burden of the metatheorem (§6–§7).

The update layer comes first, and a structural observation makes it cheap: this half of the corrected theory never mentions the merge. Its one theorem is stated over the update signature $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{rc} \rangle$ alone:

Theorem 5.1 (convergence)**.** *Assume the update-layer VCs (items 1–3 of §7). Then any two lo^E-respecting enumerations of a finite event set E fold from σ_0 to the same state.*

This is where the set-relative reading of §3 earns its keep: with the paper's configuration-wide order in place of lo^E, the theorem is *false* — the defeater's heads are already counterexamples, since two order-respecting replays of the three events of head_p reach different states. Convergence licenses a definition:

Definition 5.2 (canonical state)**.** *For a finite event set E, the canonical state $\sigma(E)$ is the state obtained by folding any lo^E-respecting enumeration of E from σ_0. At least one respecting enumeration exists because lo^E is acyclic (VC 2); the result is independent of the choice by Theorem 5.1; and $\sigma(\emptyset) = \sigma_0$. We write $\sigma(E)$ for the canonical state of an event set, and σ_v for the state a version v happens to hold in the store — proving that the two coincide is precisely the metatheorem's job.*

With canonical states in hand, witness lists disappear as promised: a version linearizes iff $\sigma_v = \sigma(L(v))$ — the witness, when one is owed, is any respecting enumeration of $L(v)$. The induction can therefore maintain a state-level invariant ("every version holds σ of its events") instead of constructing sequences. The one genuinely ternary statement is what the merge must produce:

Definition 5.3 (ternary Join Lemma)**.** *For backward-closed event sets E_1, E_2 of a reachable configuration,*

$$\sigma(E_1 \cup E_2) \;=\; \mathsf{mergeL}\big(\sigma(E_1 \cap E_2),\ \sigma(E_1),\ \sigma(E_2)\big).$$

By Lemma 4.1 the first argument is exactly the state the store hands the merge. The adequacy proof (§8) is then an induction over the transition system maintaining "every version holds the canonical state of its event set"; the merge case *is* the Join Lemma. What remains is the question this part exists to answer: which per-data-type VCs make the Join Lemma true?

6 Why the contract is a delta contract

Where should the laws that prove the Join Lemma come from? The obvious supplier is the CRDT tradition. For state-based CRDTs — binary merge, no LCA — merge-coherence is lattice-flavoured: associativity, idempotence, inflation of updates. None of the three transfers to the ternary merge: their free-tuple forms are the wrong equations for an LCA-aware merge, and their honest instances are too weak to power a lattice-style proof. This section works through why, because the shape of what fails to transfer reveals the contract that succeeds.

Idempotence: the free-tuple form is the wrong equation. The *diagonal* law $\mathsf{mergeL}(a, a, a) = a$ — the paper's own MergeIdempotence — holds (for the counter: $a + a - a = a$), but it is not the law a lattice-style proof consumes. The binary architecture uses idempotence to absorb a duplicated *side*, $\mathsf{merge}(a, a) = a$, with nothing tying the two copies to a common past; the ternary analogue would be $\mathsf{mergeL}(l, a, a) = a$ with l *free*. But quantifying l freely asks the merge about tuples the version DAG never produces. Merging a version with *itself* has $l = a$ — the LCA of a and a is a — where the law collapses to the diagonal and holds. And when two *distinct* histories happen to reach equal side states, feasibility puts l strictly below them and the counter's $\mathsf{mergeL}(l, a, a) = 2a - l$ is the *correct* answer: both branches' increments count. So there is no idempotence *failure* — there is no meaningful side-idempotence *law*: its honest instances are the trivial diagonal, and demanding it unconditionally would exclude precisely the LCA-sensitive data types the ternary theorem exists for.

The general principle, which the rest of this section instantiates: the algebraic properties of an LCA-aware merge are properties of the *causal history*, not of the concrete state. Two sides merge idempotently when their *histories* coincide — which is what forces $l = a$ — not when their states happen to be equal; equal states with different histories rightly merge differently. The lattice laws go wrong exactly by quantifying over states while forgetting the histories that produced them, and every law that survives below is stated over history-indexed data: canonical states $\sigma(E)$, feasible tuples (the LCA slot carrying the intersection history), downset deltas.

No usable order. The lattice-style workhorse order $x \sqsubseteq y \coloneqq \mathsf{merge}(x, y) = y$ has ternary candidates $x \sqsubseteq_l y \coloneqq \mathsf{mergeL}(l, x, y) = y$, but for the counter this relation degenerates to $x = l$;

no antisymmetry survives to power order-theoretic reasoning. The single contextual VC below is accordingly an *equation*, not an inequality.

Associativity is the wrong target. The natural guess — $\mathsf{mergeL}(l, \mathsf{mergeL}(l, a, b), c) = \mathsf{mergeL}(l, a, \mathsf{mergeL}(l, b, c))$ — assumes one LCA for all three merges. In an execution, re-associating a three-way reconciliation involves *four* distinct LCAs:

$$\mathsf{mergeL}\big(l_{(ab)c},\, \mathsf{mergeL}(l_{ab}, a, b),\, c\big) \stackrel{?}{=} \mathsf{mergeL}\big(l_{a(bc)},\, a,\, \mathsf{mergeL}(l_{bc}, b, c)\big),$$

with $l_{ab} = \sigma(E_a \cap E_b)$, $l_{(ab)c} = \sigma((E_a \cup E_b) \cap E_c)$, and so on. For the counter — whose canonical states are additive measures over event sets — the two sides come to $a + b + c$ minus the two LCA measures, and they agree because

$$(E_a \cup E_b) \cap E_c = (E_a \cap E_c) \cup (E_b \cap E_c) \quad \Longrightarrow \quad \text{both LCA-sums} = |E_{ab}| + |E_{ac}| + |E_{bc}| - |E_{abc}|$$

by inclusion–exclusion. The law holds, but only *through the set-algebra of intersections*: over four unconstrained states it is false. Honest associativity is inherently a feasible-tuple fact — and, the mechanization's pleasant surprise, **the corrected induction never needs it**. Every merge the induction forms already sits at its honest LCA; the only laws consumed are two *redistribution* identities that equate two honestly-formed merges:

Definition 6.1 (the delta contract)**.**

$$\begin{aligned} \textsc{local-redistribute}: &\quad \mathsf{mergeL}(l,\, \mathsf{mergeL}(m, x, c),\, y) \;=\; \mathsf{mergeL}(m,\, \mathsf{mergeL}(l, x, y),\, c), \\ \textsc{redistribute}: &\quad \mathsf{mergeL}\big(\mathsf{mergeL}(m, x_0, c),\, \mathsf{mergeL}(m, x_1, c),\, \mathsf{mergeL}(m, x_2, c)\big) \\ &\qquad\qquad = \;\mathsf{mergeL}\big(m,\, \mathsf{mergeL}(x_0, x_1, x_2),\, c\big). \end{aligned}$$

Read c as a *delta relative to* m (in the induction, c will be "apply event e to its causal past"). LOCAL-REDISTRIBUTE says a delta application commutes past an enclosing merge through one branch slot. REDISTRIBUTE says a delta carried by *all three* components extracts *once* — the LCA slot cancels the duplicates by arithmetic. This is the delta contract's quiet coup: cancelling the duplicated shared contribution is exactly the job idempotence performs in lattice-based convergence arguments, now done without any idempotence — which is why the counter sails through (REDISTRIBUTE for the counter is the identity $\sum x_i + 3c - 3m - (x_0 + c - m) - \cdots$ collapsing on both sides).

Where the contract holds unconditionally — and where it cannot. The two laws hold *on all states* exactly for the group-like class (the counter) and the lattice-like, LCA-blind class (grow-only structures). They are *unconditionally false* for the OR-set: take an element tag present in c and l but in neither branch — the two sides of LOCAL-REDISTRIBUTE then disagree about whether the LCA's copy survives. But such a tuple can never arise: a tag in a canonical LCA state is in *both* branches unless removed, and a removal is itself an event of the branch. The failing tuples are *infeasible*, and this is a theorem-grade phenomenon, not an inconvenience:

Finding. *For LCA-sensitive MRDTs beyond the group $\oplus$ lattice classes, the merge-coherence laws are irreducibly feasibility-conditioned: true on canonical tuples at honest LCAs, false in general. Quantifying merge VCs over all states — the published catalogue's style — is not merely inconvenient for such data types; for several laws it is wrong in principle. (The enable-wins flag even falsifies the unit law* $\mathsf{mergeL}(\sigma_0, \sigma_0, s) = s$ *on an infeasible state with a set flag and a zero counter.)*

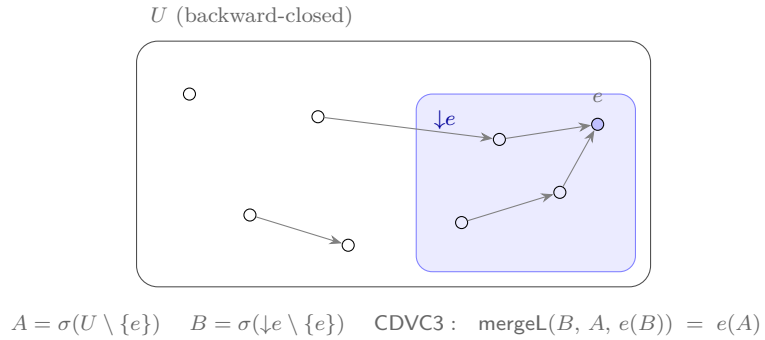


Figure 3: The causal-delta equation. The shaded region is e's causal past $\downarrow e$ (what e saw); e is lo^U-maximal. e's effect on the whole world A equals: compute e's effect on its own causal past B, then merge that delta into A *over LCA* B. The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$. Concurrent context that e never observed cannot amplify what e does.

Accordingly the delta laws enter the final contract in feasibility-conditioned form: quantified over canonical states of backward-closed event sets, with every mergeL node at its honest LCA. The unconditional form remains available as a one-line *on-ramp* (it trivially implies the conditioned form) for the group and lattice classes.

7 The eight verification conditions

The update-side obligations of §5 and the merge-side laws of §6 now assemble into the full contract — a *new* VC set, replacing the paper's catalogue of twenty-four. For an MRDT $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$, the contract is:

Update layer (unconditional, SMT-shaped).

1. **rc-non-comm (guarded)** — for distinct events *from different replicas*: they fail to commute iff rc orders them in exactly one direction. *The conflict table is the commutativity table, with a direction.* The different-replica guard is a semantic necessity, not a convenience: same-replica events are totally ordered by program order, so they are never concurrent and there is no conflict for rc to resolve. An unguarded VC would instead demand that rc order *every* non-commuting pair — unsatisfiable for real data types: the *optimized OR-set*'s same-replica same-element adds do not commute (the newer add evicts the older tag), yet they can never race, and its rc rightly ignores them. (The paper's F* artifact carries exactly this guard in its interface — `App_mrdt.fsti`: `requires distinct_ops o1 o2 /\ get_rid o1 <> get_rid o2`.)

2. **no-rc-chain** — conflict priorities do not chain: never $o_1 \xrightarrow{\mathsf{rc}} o_2$ and $o_2 \xrightarrow{\mathsf{rc}} o_3$. This is what makes the linearization order acyclic, so maximal events exist — the entire peel-and-recurse architecture stands on it.

3. **cond-comm** — a resolved conflict is invisible once an absorber runs: swapping an rc-ordered concurrent pair deep in a fold changes nothing for any later non-commuting event, across arbitrary intervening events. This is the engine of convergence.

Merge layer.

4. **merge symmetry** — $\mathsf{mergeL}(l, a, b) = \mathsf{mergeL}(l, b, a)$.
5. **feasible unit** — $\mathsf{mergeL}(\sigma_0, \sigma_0, \sigma(E)) = \sigma(E)$: merging over nothing is a no-op, on states that can arise.
6. **feasible local-redistribute** and
7. **feasible redistribute** — Definition 6.1, quantified over canonical tuples at honest LCAs.
8. **the causal-delta equation (CDVC3)** — for a lo^U-maximal event e of a backward-closed U, with $A = \sigma(U \setminus \{e\})$ and $B = \sigma(\downarrow e \setminus \{e\})$ the canonical state of e's own causal past:

$$\mathsf{mergeL}\big(B,\ A,\ e(B)\big)\ =\ e(A).$$

What an event does is determined by what it saw (Figure 3): applying e to the whole world equals computing e's delta on its causal past and merging it in over that past.

Items 1–4 are one-liners for every data type we have discharged; items 6–7 frequently hold unconditionally anyway (for the OR-set, REDISTRIBUTE is a pointwise Boolean tautology); essentially the entire per-data-type proof effort concentrates in item 8, whose discharge follows a uniform recipe: characterize $\sigma(E)$ (a "sandwich" invariant along canonical enumerations), then a *trichotomy* placing every event the maximal e interacts with either inside $\downarrow e$ or absorbed on a side that owns it.

Compare the published catalogue: 24 VCs per data type, plus five more the soundness proof uses silently. Seventeen of the 24 — the BOTTOMUP induction-scheme families — exist to drive the broken induction and are consumed nowhere in the corrected one; the paper's MERGEIDEMPOTENCE is likewise never used. The seventeen were, however, doing honest work of a different kind: they are an *automation layer*, Skolemizing "holds on feasible states" into base-and-step equations an SMT solver can discharge. Rebuilding that layer, aimed at the corrected target (CDVC3 rather than BOTTOMUP), is in our view the most valuable open engineering problem this work leaves (§40).

8 Adequacy

The contract delivers. Everything a data type owes is the eight equations of §7; everything else is proved once, for all data types, as part of the execution model.

Theorem 8.1 (soundness of the eight VCs)**.** *If an MRDT satisfies VCs 1–8, then every configuration reachable from the initial configuration in the ternary transition system is RA-linearizable, per version.* `ra_linearizable_of_core_feasible_cd3`

The proof shape: the update layer powers the set-relative convergence theorem, hence canonical states; the transition induction maintains "every store version is canonical for its event set" (fork and query are trivial; apply is the extend lemma); the merge case reduces, via Lemma 4.1, to the ternary Join Lemma, which is proved from VCs 4–8 by strong induction on $|E_1 \cup E_2|$: select a $\mathsf{lo}^{E_1 \cup E_2}$-maximal e, decompose each side containing e through $\downarrow e$ (LOCAL-REDISTRIBUTE at the honest LCA $\downarrow e \setminus \{e\}$), cancel the shared delta (REDISTRIBUTE), close the principal step

MRDT	contract tier	end-to-end theorem	σ-characterization (one line)
Grow-only set	unconditional	`GOSet_ra_linearizable3_eq`	union of adds
Grow-only map	unconditional	`GOMap_ra_linearizable3_eq`	union of (k, v) records
Incr.-only counter	unconditional	`IOC_ra_linearizable3_eq`	#events
PN-counter	unconditional	`PN_ra_linearizable3_eq`	#inc − #dec
RGA (tombstone)	unconditional	`RGAM_ra_linearizable3_eq`	grow-only nodes + tombstones
Peritext	unconditional	`Peritext_ra_linearizable3_eq`	grow-only mark records
OR-set	feasible	`ORSet_ra_linearizable3_eq`	tag live iff added, no vis-later `rem`
OR-set (efficient)	feasible	`ORSetE_ra_linearizable3_eq`	… or same-replica `add` later (eviction)
Enable-wins flag	full-closure	`EWFlag_ra_linearizable3_eq`	per key: ctr = #enables; flag iff unkilled enable
Multi-valued register	feasible	`MVR_ra_linearizable3_eq`	tagged writes × overwrite log; all-comm ⇒ $B{=}\sigma_0$
Add-wins priority queue	feasible	`AWPQ_ra_linearizable3_eq`	the OR-set pattern on A; grow-only I
Bounded counter (escrow)	all-comm	`BC_ra_linearizable3_eq`	convergence flat; the *bound* is a conditioned safety theorem (`bc_version_inv`)
FWW reservation register	all-comm	`FWW_ra_linearizable3_eq`	payload arbitration (min-ts semilattice); winner characterized at every version (`fww_version_min`)
LWW register	all-comm	`LWW_ra_linearizable3_eq`	payload arbitration (max-ts semilattice), the last-writer twin of FWW; winner characterized (`lww_version_max`); *end-to-end mechanized*, unlike the flat CRDT LWW (sorry-carrying VC bridge)
BudgetCart	or-set `rc`	`BCart_ra_linearizable3_eq`	budgeted cart, spend derived from live instances; safety *gated* on causal canonicity (OQ 40.10)
Mergeable queue (Peepul)	*direct join*	`queue_ra_linearizable3`	no `rc` exists (enqueue clique); the merge itself is the linearization witness
RGA (embedded-chain)	*direct join*	`embed_ra_linearizable3`: RA-lin at every honestly reachable configuration; canonical sorted state, fold-canonical (`e_fold_canon`); code-parametric (Part II)	
RGA (sided, two-sided)	*direct join*	`sided_embed_ra_linearizable3`: for every side assignment; the one-sided datatype is its all-R fragment (Part II)	
Peritext (embed kernel, fused)	*instantiation*	`peritextEmbed_ra_linearizable3`: one RGA at $\alpha :=$ `char` ⊕ `boundary`; deleting a character never re-formats another (`renderIds_del`; Parts II–III)	

Table 1: The Sal production catalogue. The flat data types conclude through the ONE generic conditioned metatheorem (§10–§12) at the identity instantiation ($\approx = =$, their VC bundles feeding the flat collapse `FlatGeneric_Bridge.lean`, instance theorems in `MRDT_Instances/` (per-RDT)); the direct-join instances conclude through the per-configuration join hook under honest reachability (§13). The contract-tier column is each data type's Join-Lemma discharge content (`MRDT_Instances/`, one directory per RDT); the historical flat corollaries over the raw system remain there as internal steps.

with CDVC3, and recurse. The buried-event pathology of Example 4.2 dissolves structurally: no step ever asks how a *side's* state factors — every factoring happens through the join.

Everything else one might fear owing is proved once, for all data types, as an execution-model invariant: visibility transitivity, per-version backward closure, store well-formedness, and Lemma 4.1 itself. The data type owes exactly the eight equations.

A second route, and an honest asymmetry. One discharged data type does not fit the pattern above. The enable-wins flag's merge *compares counters* against the LCA, and for such merges the Join Lemma's hypotheses as stated (backward closure under non-commuting-visibility) are provably too weak: there is a closure-legal tuple — a live LCA enable, a dead post-LCA enable inflating one side's counter, the killing disable visible only to the other side — on which the production merge computes the wrong flag. Full *causal* closure (which the store invariant actually supplies) excludes it, and the flag is discharged through a full-closure variant of the join. The finding: **required closure strength is a function of the merge's shape** — commutation-closure suffices for set-shaped merges, counter-comparison merges need full causal closure. Reunifying the two routes was expected to be routine — "full closure only strengthens what a data type may assume." Mechanization shows it is not (`JoinLemma3C`: `no_proper_back_block`, `no_proper_front_block`): the naive reunification — re-run the Join Lemma induction with full-closure hypotheses — is refuted, because no single-event (or block) peel preserves full closure. What survives is a *closure-indexed* contract in which the data type *declares* its closure strength; the soundness bridge is uniform across strengths, so no reunification into one strength is needed. In the mechanization the closure-indexed contract JoinLemma3C $\mathcal{C}$ has the single soundness bridge `ra_linearizable3_of_joinC` (store versions are fully causally closed, so the bridge applies for any $\mathcal{C}$ that full closure implies), and all nine production discharges route through it unchanged (`AllMRDTs_viaContract`): eight at weak closure, the enable-wins flag at full.

9 The catalogue, discharged

Table 1 is the empirical heart of the metatheory: the VC set is not merely adequate in principle — the data types people actually ship satisfy it. Three lessons from the sweep:

Tombstones are a metatheoretic trivializer. The catalogue's celebrated hard cases — the RGA sequence type and the Peritext rich-text type — land in the *easiest* tier. The reason is almost embarrassing: with tombstones, deletion *adds* a record; every state component is grow-only; all operations commute; the merge is a blind join. All of the RGA's genuine difficulty lives in the *read-side* traversal (turning the node soup into a sequence), which RA-linearizability of the state never touches. By contrast, tombstone-free designs (which delete physically) fall outside the eight VCs for a reason worth stating precisely: their commutation VCs are themselves reachability-conditioned (two inserts commute only on well-formed states), and the update layer above, like the paper's, quantifies over all states. The conditioned metatheory of §10 exists to host exactly this class, and the embedded-chain RGA of Part II is its canonical occupant, discharged through the factored route of §13.

Commutation class and delta class are independent axes. The multi-valued register is all-commuting, yet *not* in the unconditional tier: its merge is intersection-shaped, $(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$, and fails the delta laws on infeasible tuples exactly as the OR-set does. The boundary between tiers is drawn by *how the merge uses its LCA*, not by whether operations commute. (The compensation: all-commuting makes every punctured causal past empty, $B = \sigma_0$, so the feasible discharge needs no trichotomy at all.)

The eight VCs certify real code on its historical fault line. The enable-wins flag's repository history contains a known-broken sibling: a global-counter variant whose compositional VC (`inter_right_1op`) fails on DAG-shaped executions, the very defect per-replica counters were introduced to fix. The mechanized σ-facts prove the production merge computes exactly union-liveness — *verified on precisely the corner where the sibling fails.* This is the methodology working as intended: the metatheory does not merely bless the catalogue, it explains *why* the fixed design is right.

10 The conditioned metatheorem

The eight VCs of §7 quantify over *all* states, and §9 showed that is right for flat data types and fatal for state-dependent ones: for a sequence datatype with physical deletion, two concurrent operations commute only on well-formed states at which both are applicable, and written over all states the commutation VC is simply false. The remedy is the general signature of §2: condition every VC on the state invariant Inv and the applicability predicate app, relax the witness fold of Definition 3.2 to the data type's observational equivalence $\approx$, and prove RA-linearizability from the conditioned VCs once and for all. The developer's burden is only to describe sensible operations; the metatheorem does the rest. This section and the next two state the conditioned VCs and the metatheorem with its pen-and-paper proof; §13 then presents the factored discharge route the production instances actually use.

11 The conditioned verification conditions

A datatype presents a signature $\langle \mathrm{State}, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{Inv}, \mathsf{app} \rangle$ where Inv is a predicate on states and `app` a predicate on (operation, state) pairs (Lean: `ConditionedMRDTSig`, `Framework/MRDTSig.lean`). It must supply:

VC 1 (Discipline)**.** $\mathsf{Inv}\,\sigma_0$ *holds;* do *preserves* Inv *on applicable operations; and generation is disciplined: every operation is* `app` *and fresh at its birth state, with globally unique, monotone ids. (This is the execution model, Lean:* `ConditionedConfiguration`*,* `Framework/ConditionedExecutionModel.lean` *— kernel-clean, axiom-free.)*

VC 2 (Conditioned commutation — the swap witness)**.** *For concurrent operations* a, b *and any state* σ *with* $\mathsf{Inv}\,\sigma$ *at which both are applicable in the relevant reorder sense,* $\mathsf{do}\,(\mathsf{do}\,\sigma\,a)\,b \approx \mathsf{do}\,(\mathsf{do}\,\sigma\,b)\,a$.

VC 3 (Threadable reorder invariant)**.** *An auxiliary invariant* J *on (state, pending-event) that (i) certifies applicability/the swap precondition of VC 2 at reorder states, (ii) holds at the enabling fold of each event, and (iii) is preserved by each reorder step. Only the enabled events — those whose causal past is folded — need be certified; that scope is exactly what a linearization repair ever transposes.*

VC 4 (Merge–linearization bridge — the conditioned Join Lemma)**.** *For a reachable LCA state* l *and branches* a, b *obtained from* l *by disjoint concurrent event lists,* $\mathsf{merge}\,l\,a\,b \approx \mathsf{fold}\,l\,\pi$ *for a* lo^E*-respecting enumeration* π *of the branch events.*

For flat datatypes $\mathsf{Inv} = \mathsf{app} = \mathsf{true}$, VC 2 degenerates to unconditioned commutation, VC 3 to $J = \mathsf{true}$, and VC 4 to the ordinary Join Lemma — recovering the unconditioned methodology.

12 The metatheorem

Theorem 12.1 (Soundness — conditioned VCs ⇒ RA-linearizability)**.** *Any MRDT satisfying VCs 1–4 is RA-linearizable (Definition 3.2, read up to* $\approx$*).*

The proof is generic — it never inspects a particular datatype — and factors through two lemmas, each proved over the abstract signature with $\approx$ an arbitrary congruence for do and merge.

Lemma 12.2 (Convergence)**.** *VCs 1, 2, 3 imply that all* lo^E*-respecting admissible enumerations of a backward-closed reachable* E *fold from* σ_0 *to* $\approx$*-equal states; hence* $\sigma^*(E)$ *is well defined up to* $\approx$*.*

Proof. Any two lo^E-respecting enumerations of E are related by repeatedly transposing adjacent lo^E-incomparable events. Because lo^E is non-transitive, "respecting" is the elementwise `Pairwise` condition rather than sortedness, so this order-repair is more delicate than for a total order — it is exactly what the generic engine `eq_convergence` carries out, and we appeal to it rather than re-derive it. Each transposition preserves the fold up to $\approx$ by VC 2, provided the swap precondition holds at the transposition point; VC 3 supplies it along the whole enumeration (base at each event's enabling fold, preserved by each step, scoped to the enabled events — the only ones the repair transposes). The engine consumes only that $\approx$ is an equivalence, do is $\approx$-congruent, and a swap oracle of the shape VC 2+3 provides (Lean: `eq_convergence`, kernel-clean). □

Lemma 12.3 (Canonical adequacy)**.** *VC 4 and Lemma 12.2 imply that every reachable version state is* $\approx\ \sigma^*$ *of its delivered event set.*

Proof. Induction over the version DAG. Initial: the empty fold. Update node: appends one delivered operation to a canonical fold (definitional). Merge node with LCA l and branches a, b: by hypothesis $a \approx \sigma^*(E_a)$ and $b \approx \sigma^*(E_b)$, and $l \approx \sigma^*(E_l)$. First rewrite a, b to literal canonical folds using $\approx$-congruence of merge (so VC 4, stated for branch folds, applies); then VC 4 gives $\mathsf{merge}\, l\, a\, b \approx \mathsf{fold}\, l\, \pi$ for a lo^E-respecting π of the branch events $E_a \cup E_b$. Now E_l causally precedes those branch events (they were generated on states derived from l), so a lo^E-respecting enumeration of $E_l \cup E_a \cup E_b$ factors as (an lo^E-enumeration of E_l) followed by π; hence $\mathsf{fold}\, l\, \pi = \mathsf{fold}\, (\sigma^*(E_l))\, \pi \approx \sigma^*(E_l \cup E_a \cup E_b)$ by Lemma 12.2 (the engine is start-state generic). Backward closure keeps every enumeration admissible. □

Proof of Theorem 12.1. Immediate from Lemma 12.3: each version state $\approx \sigma^*(E_v)$, which is Definition 3.2. (Unconditioned template: `ra_linearizable3_of_joinC` plus the closure-indexed adequacy bridge, already kernel-clean with nine instances; the conditioned metatheorem generalizes it by carrying $\mathsf{Inv}/\mathsf{app}$ through the two lemmas.) □

Corollary 12.4 (Strong eventual consistency)**.** *Two versions with the same delivered set E have $\approx$ states: both $\approx \sigma^*(E)$ by Theorem 12.1, hence $\approx$ each other.*

Remark 12.5 (The flat instances)**.** *The nine flat production MRDTs (counter, OR-set, MVR, …) take* $\mathsf{Inv} = \mathsf{app} = \mathsf{true}$*: VC 2 is their unconditioned commutation, VC 3 is $J = \mathsf{true}$, and VC 4 is the ordinary Join Lemma — already discharged (`ra_linearizable3_of_joinC`). They inherit RA-linearizability from the same Theorem 12.1. The instances of §13 and Part II exercise the conditioned mechanisms proper.*

13 The factored discharge route: honest reachability and per-configuration joins

The conditioned instances in production do not each re-prove VCs 2 and 3 from scratch. What they share is an induction: thread a per-configuration honesty contract through reachability, and invoke a Join Lemma at each merge. This section presents that induction, extracted once, together with the instances that ride it: the mergeable queue (the flagship, whose conflict graph admits no rc at all), the bounded counter (conditioning for safety), and the FWW reservation register (payload arbitration). Part II's embedded-chain RGA is discharged by exactly this route.

The factorization: one metatheorem, three join species. The common structure across the conditioned instances is a theorem rather than a convention. The developments all ran the same induction — thread a per-configuration honesty contract through reachability, invoke a Join Lemma at each merge — and that induction is extracted once (`Metatheory/HonestReach.lean`): `HonestReach` D H is LTS reachability where every step leaves a configuration satisfying the contract H, and the metatheorem (`ra_linearizable3_of_honest_reach`; axioms `propext`, `Quot.sound`) states that if every H-configuration admits the Join at its core (`JoinLemma3At`), every H-honestly reachable configuration is per-version RA-linearizable. Plain reachability is the degenerate contract $H = \top$,

recovering the unconditional bridge; the queue's and the bounded counter's bespoke inductions are one-line corollaries of the extracted form.

What does *not* factorize is the join discharge, and that is the finding: the instances discharge `JoinLemma3At` by genuinely different mechanisms — *conditioned commutation* (operations commute on invariant-satisfying states, the framework's fully general quotiented route), *flat commutation* (the bounded counter: the CD route, contract $\top$), and *direct witness* (the queue: the merge is its own linearization certificate; Part II's embedded-chain family discharges the same way). Conditioning thus splits as

$$\text{generic honest-reachability metatheorem} \;\times\; \text{per-datatype join discharge},$$

and the framework's job at the boundary is to *receive* a join, not to produce one. A witness-classed generalization (`JoinLemma3AtW`, `Metatheory/WitnessClass.lean`, from the Shesha investigation of Part III) extends the hook to datatypes whose canonical states are unique only per witness-classed enumerations; and the instance layer is payload-generic where it should be (the embed instance of Part II takes its element type as a defaulted parameter, so the fused Peritext is a pure instantiation). One further regularity is worth recording: the production honesty contracts all have the shape "P holds of each event at the fold of its causal past" — anchor liveness for the sequence family, slack for the counter, head-ness for the queue — with P the signature's client-checkable `app` (or its accuracy analogue). That shape is extracted once (`GenHonest`, with `AppHonest` its `app` instance and `ra_linearizable3_of_genHonest_reach` the composite metatheorem; `Metatheory/GenHonest.lean`), and the counter's and the queue's contracts are re-derived as instantiations (`BCHonest_iff_genHonest`, `qHonest_of_genHonest`). A generic *safety* metatheorem completes the factorization; it is the subject of a paragraph below, and the counter's counting argument indeed turned out to mark where the per-datatype content sits.

The flagship direct-join instance: the mergeable queue — when no rc can exist. The queue of the certified-MRDT line of work (Soundarapandian et al., PLDI'22, *Certified Mergeable Replicated Data Types*) is the catalogue's first data type whose conflict graph is a *clique* rather than a star: two concurrent enqueues genuinely do not commute (queue order is arrival order), so VC 1's iff forces an rc-edge between them, and any orientation of a self-conflicting class composes with itself into a length-2 path — every candidate assignment violates `no_rc_chain` (Open Question 40.9). The flat engine is structurally unavailable, and the instance goes through the framework's per-configuration join hook instead (`JoinLemma3At`, `goodConfig3_merge_at`): prove the Join Lemma directly, at every reachable configuration.

The implementation is the functional queue with Peepul's merge, verbatim: the state is a list of $(tag, value)$ pairs, head first; $\mathsf{enq}\, v$ appends a fresh element tagged with the operation's own timestamp; $\mathsf{deq}\, t$ removes the element tagged t, wherever it sits; and the three-way merge keeps the LCA's elements that survive in both branches (in LCA order), then each branch's new arrivals (in branch order). The client-facing API is unchanged — dequeue() at a replica reads its local head — but the *operation* records which element that call removed: app for $\mathsf{deq}\, t$ is "t is the head of the issuing replica's state". As with the sequence family, the generation discipline is the datatype's natural client behaviour, captured rather than imposed.

The proof exhibits Peepul's merge as its own certificate. The Join Lemma asks for a delivery-respecting enumeration of $E_1 \cup E_2$ whose fold is $\mathsf{mergeL}(\sigma_0, \sigma_1, \sigma_2)$; the witness is the concatenation $\rho_0 \cdot (\rho_1 \setminus E_0) \cdot (\rho_2 \setminus E_0)$ of the given enumerations — exactly the merge's shape, no reordering.

Three facts make the fold of that concatenation compute the merge, all consequences of the closure premise (branch event sets are backward-closed under $\xrightarrow{\mathsf{vis}}$ among non-commuting pairs), timestamp uniqueness, and an honest-history contract (`QHonest`: every dequeue names a tag whose enqueue is $\xrightarrow{\mathsf{vis}}$-prior — the queue's honest-delivery contract, discharged by the `app` head-check via `qHonest_of_applicable`): a fold from the empty queue is "the enqueues, in order, minus the dequeued tags" (the fold formula, which needs honesty — a dequeue preceding its enqueue would consume nothing); a delta's tags are fresh with respect to the LCA (uniqueness); and a cross-branch dequeue is impossible — a $\mathsf{deq}\,t$ in one branch pulls its enqueue into that branch by closure, so if the enqueue of t is the *other* branch's delta, it would lie in both branches, hence in the LCA, contradicting delta-ness. An LCA element then survives the union iff it survives in both branches, which is precisely the merge's first clause.

The payoff is the specification. In the PLDI'22 development the queue was the hardest case exactly because its specification had to be written in a bespoke relational language over event partial orders, with an explicit clause recovering *which element is the head*; the merge was then verified against that ad-hoc spec. Here the specification is the same generic statement as every other instance in Table 1 — every version is the fold of a delivery-respecting linearization of its event set — and head-ness, FIFO order, and once-per-element consumption are *read off* the fold rather than specified. (One semantic caveat, present equally in Peepul: two replicas that concurrently dequeue the same head both consume that one element — the merged state removes it once. RA-linearizability makes this visible instead of hiding it: both $\mathsf{deq}\,t$ events fold over the single $\mathsf{enq}\,t$.) At ∼870 lines (`MRDT_Instances/MergeableQueue/`), the instance carries positional structure at a fraction of a sequence datatype's cost.

Conditioning for safety: the bounded counter. Conditioning first entered the framework to deliver *convergence* for state-dependent datatypes. The bounded counter (per-replica escrow: grow-only tallies ($\mathsf{incs}, \mathsf{decs}$) per replica, per-slot group merge; mirror of `Sal/CRDTs/Bounded_Counter`) shows the same mechanisms delivering *safety* for a datatype whose convergence is flat: all its operations commute, the eight VCs discharge along the counter route, and the catalogue capstone (`BC_ra_linearizable3_eq`) is an identity instantiation. What the flat theory cannot even state is the property the datatype is named after. Its CRDT development is explicit: "the bound is enforced operationally by well-behaved clients" — a client refuses to issue Dec without slack — and that discipline lives outside the formal development. Conditioned, it moves inside: Inv is the per-replica escrow invariant ($0 \le \mathsf{decs}\,r \le \mathsf{incs}\,r$), `app` is the client's check (a Dec needs slack in the *issuing replica's own* slots — positive, and checkable against the issuing replica's state), and the contract on executions (`BCHonest`: every decrement was applicable at the fold of its causal past) is the counter's honest-delivery contract. The theorem (`bc_version_inv`, kernel-clean): in every reachable configuration with honest history, *every version — heads, LCAs, everything the store registered — satisfies the invariant*; hence the counter's value is non-negative everywhere (`bc_value_nonneg`). The proof is a counting argument riding the flat machinery: every version's state is a fold of its event set (canonicity), fold slots *are* per-replica event counts (the group merge is inclusion–exclusion on counts, via the LCA lemma), and the vis-maximal decrement's honesty at its own causal past — which lies inside the version by causal closure — bounds the decrement count by the increment count. At ∼600 lines, it is also a calibration point for which conditioning machinery is generic: the same invariant-at-generation shape, none of the positional structure.

The generic safety metatheorem: causal witnesses, and an escrow route. The safety half of conditioning also factorizes, but the obvious formulation is *false*, and the refutation comes from the flagship safety instance itself. One would like: if updates preserve Inv on applicable operations and the history is honest, then Inv holds at every version — proved by inducting along the version's canonical enumeration. But canonicity constrains the enumeration only by lo, and for the bounded counter — all of whose operations commute — lo is *empty*: [dec, inc] is a perfectly canonical enumeration of an honest version, and its intermediate state violates the escrow invariant. Only *endpoints* of canonical folds are guaranteed; commuting causal predecessors may float behind the event that needs them. The repair is to restrict the witness class, not the induction: a *causal* witness additionally linearizes $\xrightarrow{\mathsf{vis}}$ (`CausalCanonical`), and its prefixes are exactly the vis-closed, future-free sets containing each event's whole causal past. Over those, the metatheorem holds (`version_inv_of_causal_canonical`; axioms `propext`, `Quot.sound`): Inv at every version, from honesty (`HonestApp`: app at some causal fold of the event's causal past — what the issuer's own state witnesses) and one per-datatype obligation `SafetyStep` (applicability transfers from the causal past to any admissible prefix, fused with Inv-preservation). For datatypes whose operations all commute the causal witness is free (`causalCanonical_of_all_comm_rc_either`, a pointwise permutation argument). The bounded counter discharges `SafetyStep` in three lines — extras in a prefix are other replicas' events and cannot touch the issuer's slots — and `bc_version_inv` is re-derived as a corollary, retiring the bespoke vis-maximal counting apparatus, which is thereby revealed as compensation for inducting over non-causal witnesses. A second, independent route generalizes that retired argument itself: for datatypes *measured* by affine observations (fold values = event-set counts), an *escrow metatheorem* (`escrow_version_inv`) yields the same conclusions with no causal witness and no Inv-preservation at all; `bc_version_inv` is derived a second time this way. The queue, by contrast, *refuses* `SafetyStep` — "t is the head" is not stable under a concurrent honest dequeue of the same head, and correctly so: the double-dequeue execution satisfies any sound obligation, and the queue's stable residue is event-level (`QHonest`), consumed by convergence rather than by a state invariant. The full analysis, including two further refuted formulations, is the pen-and-paper memo `Development/GENERIC_SAFETY_PENPAPER.md` — written, per this project's standing lesson, before any Lean. (A natural follow-up is a *conditioned step relation* — apply guarded by app at the head state — under which honesty becomes a theorem of the LTS rather than a contract; deferred.)

A small fourth instance: the FWW reservation register — payload arbitration, and what honesty cannot buy. A register claimable once (a seat, a username): sequentially the first claim wins; concurrently, the winner is the claim that is first in *Lamport* time. This is the positive complement to the metadata-free kill-test (`lww_merge_needs_timestamps`): arbitration whose key is not in the state is impossible, and arbitration whose key *is* in the state is a semilattice triviality — the state is the minimum claim $(\mathsf{ts}, \mathsf{rep}, v)$ under the lexicographic order (or unset), update and merge are min, and the entire eight-VC discharge is min-algebra (∼300 lines, the smallest conditioned instance). The characterization theorem (`fww_version_min`) is honesty-free — semilattice folds are enumeration-independent — and says: at every version of every reachable configuration the register holds exactly the minimum-timestamp claim of its event set, unset exactly on the empty set. Since timestamps respect causality, a causally later claim never displaces an installed one; the min rule arbitrates only among *concurrent* claimants, and it must, on pain of divergence. The instructive part is the generation discipline: app for

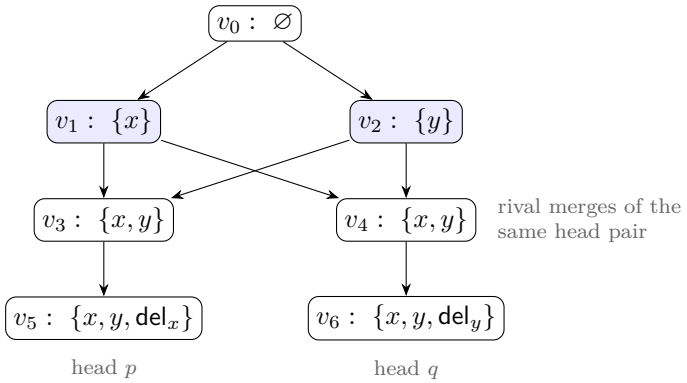


Figure 4: The criss-cross store used for the machine pins (the store of `VirtualLCA_Spot.lean`, drawn top-down; edges parent to child). Replicas p and q concurrently add x and y; each then merges the same diverged pair (v_1, v_2), creating the rivals v_3, v_4 with equal event sets; each deletes one element. The common ancestors of the heads v_5, v_6 are $\{v_0, v_1, v_2\}$ with *two* maximal elements v_1, v_2: no version is an LCA, and the gated Merge cannot fire. Note $E(v_1) \cup E(v_2) = \{x, y\} = E(v_5) \cap E(v_6)$: the antichain's union is exactly the meet, which is Proposition 14.1.

a claim is "the register is unset in my causal past", and *deliberately no theorem consumes it* — two honest concurrent claimants both see unset, both claim, and each locally wins until a merge disabuses one. "Unset" is the minimal example of an applicability check that is *not stable under concurrent honest extension* (contrast the escrow counter's own-slot slack, which is — precisely the boundary the safety metatheorem's `SafetyStep` draws). A merge-based register is a reservation, confirmed only at causal stability; it is never a mutex.

14 Virtual LCAs: merging without a common-ancestor version

The transition system of §3 gates Merge on the existence of an LCA: a version v_ℓ that reaches both heads and dominates every common ancestor. In a *criss-cross* configuration the common ancestors of two heads have two or more maximal elements, no version satisfies the predicate, and Merge is disabled. This is not an exotic corner. The shape's routine genesis is two replicas each merging the *same* pair of diverged heads: both create a version with the same event set, neither reaches the other, and every later pair of descendants has both rivals as maximal common ancestors (Figure 4). The JS runtime built on this model makes the gate explicit, and its randomized testing shows how routine the shape is under honest peer-to-peer syncing: 592 gated sync attempts in 220 randomized trials. The gap is exactly where git reaches for its *recursive* merge strategy: merge the maximal common ancestors into a virtual base, then merge the heads against that. This section develops that extension inside the metatheory and reports what it costs: nothing, for join-lemma datatypes.

The rule. Write $\mathrm{MCA}(v_1, v_2)$ for the maximal common ancestors of two versions (maximal in the reachability order, which is a partial order because a version's identifier is its allocation rank). $\mathsf{VirtualLCA}(v_1, v_2)$:

- $M := \mathrm{MCA}(v_1, v_2)$. It is nonempty (the pinned root is a common ancestor), and when the heads are incomparable every member is a strict ancestor of both.

- If $M = \{m\}$: return m. This is the existing rule, and Lemma 4.1 applies unchanged.
- Otherwise sort M ascending by rank (deterministic: version ids are allocation ranks) as $m_1 < \cdots < m_k$ and fold: start with m_1; at each step, merge the accumulator with the next member m_i through the recursively resolved LCA of the sub-pair, materializing a *scratch node* whose state is that ternary merge and whose event set is the union so far. Return the final accumulator.

The head merge then runs the ordinary ternary merge with the returned state in the LCA slot. The scratch nodes exist so the nested MCA queries are well posed; nothing references them afterwards, so they are never ancestors of real heads and cannot perturb later queries. Termination is unconditional: measuring a call on (a, b) by the number of allocated versions in the pair's joint ancestry, every version in a sub-pair's support is a common ancestor of the original heads while v_1 itself is not, so the measure strictly drops at each nesting level, and within a level the fold index drops. In the mechanization the extension lands as a conservative *new* step relation `Step3V`, with a base case embedding every old step and `mergeVirtual` the one new case; widening `Step3` in place was tried and rejected, because it breaks every exhaustive case analysis over the old relation in instance files. Only the final virtual node is allocated; the store invariant extends across the new step.

The covering proposition. The Merge rule's semantic requirement on the LCA slot is Lemma 4.1: its event set must be the head intersection. The virtual construction meets it exactly, on the strength of invariants the store already carries.

Proposition 14.1 (covering)**.** *Let S be a finite nonempty set of allocated versions and w an allocated version; let* $\mathrm{MCA}(S, w)$ *be the maximal elements of $\{x : x$ reaches some $u \in S$ and reaches $w\}$. Under the store invariant,*

$$\bigcup_{m \in \mathrm{MCA}(S,w)} E(m) \;=\; \Big(\bigcup_{u \in S} E(u)\Big) \cap E(w).$$

mca_events_cover

With $S = \{v_1\}$, $w = v_2$ this covers the head pair; with S the support of a scratch node it covers every sub-pair of the recursion; with a singleton MCA it degenerates to Lemma 4.1. The subset direction is event-set monotonicity along reachability, twice. The superset direction is where the store invariant earns its keep: an event in both sides has, by the *origin* invariant of §4.1 (each event appears first at a unique generator version), a single birth version that reaches every version whose event set contains it; that generator is a common ancestor, every common ancestor reaches a maximal one, and monotonicity carries the event up into it. Since the fold unions event sets, the returned virtual node has event set $\bigcup_i E(m_i) = E(v_1) \cap E(v_2)$: exactly what the Merge rule requires. An earlier forecast (recorded in the GC analysis, §15) had guessed the union approximates the intersection from below; under the carried invariants the approximation is *exact*, even though each antichain member individually sits strictly below the meet.

One delimitation is worth stating, because it marks the model boundary the proposition rests on. The origin invariant is load-bearing: in a model that can apply the *same* event at two incomparable versions (op-based redelivery, with no store-wide timestamp freshness), the event has two incomparable birth sites, neither a common ancestor, and the union strictly undershoots

the intersection. Countermodel: apply e independently at the root on both branches; then $E(v_1) \cap E(v_2) = \{e\}$ while the sole MCA is the root with empty event set. The transition system excludes this by store-wide timestamp freshness (an op is born at exactly one version), so the covering invariant is not a new obligation: it is `StoreInv.origin` plus monotonicity, already carried and maintained.

Canonicity: one fold induction, and the VC surface does not move. The adequacy induction of §8 consumed two facts about the LCA slot: its event set is the intersection (now Proposition 14.1) and its registered state is canonical for that event set. The virtual construction re-supplies the second from the one per-datatype hypothesis the framework already centers on, the per-configuration Join Lemma:

Claim 14.2 (virtual join)**.** *If every allocated version's state is canonical for its event set and `JoinLemma3At` holds at the configuration, then every scratch node's state is canonical for its union event set; in particular the returned virtual state is canonical for* $E(v_1) \cap E(v_2)$*.* *`virtualLCAState_canonical`*

The proof is an induction over the ascending-rank fold: the antichain members are allocated, hence canonical by hypothesis; at each step the sub-pair's inner LCA slot is canonical by the inner recursion plus Proposition 14.1, and the Join Lemma yields canonicity at the union. Backward closure is preserved by union and intersection, so all intermediate sets stay in the hypothesis class; and canonicity is a property of a (configuration, event set, state) triple, not of allocated versions, so scratch states need no allocation to carry it. The same induction instantiates at the witness-classed join (`virtualLCAState_canonicalF`), and the merge case of the honest-reachability metatheorem gains the sibling case (`goodConfig3_mergeVirtual_at`, `ra_linearizable3_of_honest_reachV`). The headline is what did *not* change: `Framework/VC_Set.lean` is untouched. For any datatype that discharges its join at every configuration, the criss-cross gate lifts with **no new per-datatype verification condition**; the per-datatype VC surface does not move.

Two statement-hygiene facts, both machine-witnessed. First, determinism: the fold order is fixed (ascending rank), and for join-lemma datatypes order-insensitivity of the *returned* state is a theorem, since canonical states of a fixed event set are unique. Git's own recursive strategy folds in commit-date order and is known to be order-sensitive on conflicting merges; rank fold plus canonicity is what removes that defect here. Second, the insensitivity claim must be scoped to the result: on a directed three-member antichain the *intermediate* scratch states of the ascending and descending folds genuinely differ (each is canonical for its own, different, union), and only the final states coincide. Any mirror lemma stating canonicity of intermediates across fold orders is false.

The single-MCA shortcut, machine-refuted. The tempting implementation shortcut (pick *some* maximal common ancestor and use it as the LCA; no recursion, no scratch nodes) is not a simplification but a soundness bug, and the store of Figure 4 pins it. Resolving the head pair (v_5, v_6) through the virtual base folds v_1 and v_2 over the root into the state with both adds visible; the head merge then sees each delete on its own side against a base that *contains* the deleted element, and both deletions survive: the merged state is empty, the meet's fold. Picking the single MCA v_1 instead puts a base without y in the LCA slot, so y on v_6's side reads as a fresh add and the deleted y *resurrects*; picking v_2 dually resurrects x (`t1f_pick_u_resurrects_y`,

`t1f_pick_v_resurrects_x`, `Metatheory/VirtualLCA_Spot.lean`; the harness twin is the `T1F` fail companion of `virtual_lca_check.py`). Every single-MCA pick is wrong on this store; the recursion is not dispensable. Nor is its depth boundable in practice: in a dense randomized probe (200 trials, 6 replicas, sync-heavy), criss-crosses arose in 176 of 200 trials with resolution nesting up to depth 7, because rivalry propagates upward (rival joins of rival joins); a depth-limited implementation would simply re-erect the gate.

The quotient layers, and the rehoming stress test. The claim above serves the raw layer. The framework's quotient layers (the $\approx$-indexed contracts of §12) each thread their own merge case, and their bill was probed on the hardest instance available: the rehoming RGA, the one production datatype whose proof route runs through an equivalence quotient with a reachability-anchored honesty stack (its only role here; it is the generality stress test, nothing more). Two findings. First, the dividing line at antichain unions is exactly the per-event versus prefix-state line: honesty components that speak about an event's *own* causal past (born accuracy, dependency-prefix correctness) restrict to every scratch union for free, and the probe confirms them at every union of 590 randomized trials; components that speak about *applicability along an enumeration* (`noopFeasible`, applicable delivery) are *false* at antichain unions, in three named shapes (an insert whose anchor a concurrent kill has already removed fired 390 times across the probes; stale delete and insert paths fired 68 and 80 times), while every one of those states remained canonical for its union. So union-level obligations must be generation-discipline shaped, never applicability shaped. Second, the bill collapsed on execution: exactly *one* fold induction was owed beyond the raw layer (at the H layer, whose merge case consumes the registered slot's layer canonicity); the Eq-quotient layers contain no step destructs and re-thread mechanically (`rga_ra_linearizable3_eq_V`, and the flat production catalogue over `Step3V` via `FlatGeneric_Bridge_V`); and the NF layer is precisely the refuted `noopFeasible` route with no production consumer, so its mirror was *deliberately not built*. The refuted predicate marks a dead proof route, not a defect of the construction.

The queue, as a corollary. The mergeable queue of §13 is the instructive consumer: its join is already per-configuration (`q_join_at`), which is exactly the hook the virtual fold induction consumes, so lifting the criss-cross gate costs the queue nothing beyond restating reachability over the widened LTS. The capstone `queue_ra_linearizable3_V` (RA-linearizability at every honestly reachable configuration with virtual merges enabled) is ten lines, all of them plumbing. This is the factored architecture of §13 paying out: the framework extension multiplied across instances at the join boundary, with zero new per-datatype content.

15 Commit GC: the keep set and the GC-safety theorem

The version store only ever grows, and merges arbitrarily far in the future reach back into it for their LCA state, so a runtime cannot reclaim "everything below the heads." The GC question is which store entries may be dropped without changing any future behavior; the answer is a keep set computable from the current heads, and a forward simulation showing pruning outside it is invisible.

The keep set. Fix a configuration C_0 (the prune point) with current heads H. Over allocated versions define

$$\mathrm{Keep}(C_0) \;=\; \big\{\, v \;:\; \exists\, h_1, h_2 \in H \;(h_1 = h_2 \text{ allowed}),\quad E(h_1) \cap E(h_2) \subseteq E(v) \,\big\} \;\cup\; \{0\},$$

the *upward event-set closure of the pairwise head intersections*, with the root 0 pinned separately (the model fixes $\mathrm{ver}\,0 = (\sigma_0, \varnothing)$ forever, and fresh replicas are created there). Taking $h_1 = h_2$ shows every head is kept, and with it everything event-set-above any single head. Pruning drops the store *payload* only: entries outside the keep set become unallocated, while the replica cores and the DAG skeleton (parents, a map of ids, not states) are untouched. Retaining the skeleton is forced, not a convenience: restricting the parent relation to kept versions changes the LCA predicate itself, since removing common ancestors makes the domination clause easier to satisfy, so new LCA triples can appear whose event sets violate Lemma 4.1, and the pruned structure would fail its own invariant. The reclaimable memory is the states and event sets, which is where the memory is; the skeleton stays.

The theorem. Safety is a forward simulation, built from three lemmas. *Monotonicity* (disjunctive): along any run from C_0, every current head either event-set-dominates some prune-time head, or its only prune-time ancestor is the pinned root. The disjunction is forced by replica creation: a fresh replica's head is the root, and its lineage grows from empty until it first merges with a dominating lineage, at which point it becomes dominating itself. *Intersection containment*: if a future merge fires with heads v_1, v_2 and allocated LCA v_ℓ, then v_ℓ was not dropped; when both sides dominate prune-time heads h_1, h_2, Lemma 4.1 gives $E_0(h_1) \cap E_0(h_2) \subseteq E(v_1) \cap E(v_2) = E(v_\ell)$, which is membership in the keep set, and when either side is virgin the LCA can only be the pinned root. *Prune commutation*: every step of the unpruned run is matched, with the same label, on the pruned store; apply reads only the issuing head and a fresh slot (never dropped: dropped slots were allocated at prune time and allocation is permanent), and merge reads the two heads and the LCA, which the previous lemma keeps.

Theorem 15.1 (GC safety)**.** *For every run $C_0 \to \cdots \to C_n$, the pruned configuration admits a run with the same label sequence, and every replica read at every point is unchanged.* `gc_safety`

One scope note, recorded because it is a genuine asymmetry: the converse simulation is deliberately not mechanized. A pruned store can exhibit steps the unpruned one cannot (a freed slot can be re-allocated; store-wide timestamp freshness quantifies over fewer entries), and the asymmetry is unobservable only via a reachability invariant (every allocated version sits below some current head) that is not part of the configuration structure. The direction GC-safety needs (nothing is lost) is the mechanized one.

The head-sync countermodel. Head-sync is the discipline that future merge arguments are current heads; the Merge rule enforces it syntactically. The keep set is calibrated to exactly that discipline, and the calibration is tight: a merge from an old interior version genuinely needs a version outside the keep set. The countermodel, worked concretely (replicas A, B; events e_1, e_2, e_3 on A, e_4 on B):

version	created by	events	parents	kept?
0	root	$\varnothing$	[ ]	pinned
1	A applies e_1 at 0	$\{e_1\}$	[0]	**dropped**
2	A applies e_2 at 1	$\{e_1, e_2\}$	[1]	kept
3	B merges A (LCA 0)	$\{e_1, e_2\}$	[0, 2]	kept
4	A applies e_3 at 2 (head)	$\{e_1, e_2, e_3\}$	[2]	kept
5	B applies e_4 at 3 (head)	$\{e_1, e_2, e_4\}$	[3]	kept

The pairwise head intersection is $E(4) \cap E(5) = \{e_1, e_2\}$; version 1, with $E(1) = \{e_1\}$, contains no pairwise intersection and is dropped, strictly (the store genuinely returns nothing at 1 after pruning). The future head merge is safe: the LCA of $(4, 5)$ is version 2, kept. But version 1 satisfies the LCA predicate for the *interior* pair $(1, 5)$, so any widened Merge that accepted a non-head argument (a rewound replica, a checkpoint restore, a replica missing from the head map) would demand the dropped entry. Every out-of-band way of resurrecting an old version breaks the keep set; head-sync is a hypothesis of the metatheorem, not pessimism.

The revision under virtual merges. §14 widens what a merge reads: where no exact LCA exists, the resolution reads the maximal common ancestors, whose event sets sit strictly *below* the pairwise meet, while the keep set above retains versions *above* some pairwise meet. Under the widened rule the original keep set prunes away exactly what the recursion needs. The revision: let mcasClosure(H) be the least superset of the head set closed under pairwise MCA (finite: ranks only decrease), and keep the upward event-set closure of its members (`mcasClosure`, `keepSetV`). This contains the old keep set, and covers every recursion read: sub-pair MCAs are contained in real-pair MCAs, iterated. One might hope the one-layer seed (the pairwise MCAs of the current heads, without the fixpoint) suffices; it is one closure layer short, machine-witnessed: on a directed nested criss-cross, a depth-2 resolution reads the MCAs of the MCAs, pruning by the one-layer rule kills them (the sync raises a pruned-store error), and pruning by the closure rule keeps them while still pruning a genuinely dead interior commit, with the post-GC sync equal to the no-GC control. Future-proofing (the analogue of intersection containment) needs one new reachability fact, the *gateway* lemma: after the prune point, every ancestry path from a prune-time version to a post-prune version passes through a prune-time head or the pinned root, so a prune-time version that a future virtual resolution reads is an MCA of prune-time heads (layer one of the closure), and deeper reads stay in the closure. The widened theorem `gc_safetyV` keeps the headline shape of `gc_safety` unchanged; the gateway forced no statement change.

Bounded state: dropping the skeleton by a clock swap. The theorem above reclaims the payload but keeps the parent map, so the pruned store still costs O(commits ever minted): it retains history's shape, not its states. The skeleton was forced only for the parent-relation representation, where restricting parents mints new LCA triples that falsify lca_events (the argument of the keep-set paragraph). A change of representation removes it. The commit graph is a delta-compression of event sets, so a version's ancestry role is already carried by its event set used as a *clock*: ancestry is $\subseteq$, and the LCA of a head pair is the kept version at the *clock meet* $E(v_1) \cap E(v_2)$, which is exactly what lca_events says the structural LCA carries. The bounded store keeps only the kept payloads and sets parents $\equiv$ [ ]; with no edges, structural reachability collapses to equality and lca_events holds trivially, so there is no history to store and the state is $O(|\text{Keep}|)$ (`clockPrune`, `gc_safety_bounded`, `GC_BoundedState.lean`). Completeness is free

(`clockLCA_complete`: every head pair with a structural LCA has it recovered at the clock meet, kept by `lca_not_dropped`), so the swap never disables a merge the skeleton enabled.

The swap has exactly one hazard, and it is the lineage collision that §16 isolates. Resolving the LCA by clock alone is sound while the payload is a function of the event set, and unsound once datatype compaction breaks that: a same-clock version from an unrelated lineage, used as a merge's L, resurrects a dropped instance. The structural IsLCA sidesteps it by resolving within shared ancestry; a bare clock lookup cannot, so it is guarded by `ClockCoherent` — two kept versions with the same clock carry the same state, i.e. the payload is a function of the clock on the keep set, which is precisely the R_S-compatibility of §16. Under it every clock LCA carries the structural LCA's state (`clockLCA_sound`), so the bounded LTS Step3C resolves each merge at the clock meet and reads identically to the unpruned run. The boundary is machine-checked both ways: a PASS SPOT where a future head-pair merge finds its LCA at the clock meet and reads correctly, and a FAIL SPOT (`spot_lineage_resurrection`) exhibiting the unrelated-lineage collision that resurrects a dropped instance, with ClockCoherent on that configuration provably false. One case is deliberately not covered: a criss-cross antichain of MCAs has no single clock meet, so the bounded form of a *virtual* merge needs the retained lineage among the MCA closure, and is left owed.

One frontier, three consumers. The meet of the current heads, $\bigcap_{h \in H} E(h)$, is the *stable cut*: the events every replica has already incorporated, below which no future merge can ever again place a concurrent event. Three parts of this document consume that one frontier. The garbage collector reads it from *above*: the keep set retains the upward closure of the pairwise meets, and what pruning reclaims is exactly the versions no future LCA lookup can name. The stability VC of §16 reads it from *below*: facts certified at the cut stay true under all future merges, which is what licenses compaction. And the storage layer of Part II (§24) consumes it as the epoch boundary for coordinate re-coding, where the in-flight-anchor argument needs visibility of all current heads. A runtime that tracks the frontier once serves all three consumers from the same bookkeeping, at pairwise rather than global granularity where GC needs it.

16 The stability VC: verified GC callbacks

Commit GC (§15) reclaims *versions*; it never touches the states themselves. But states also accumulate redundancy: an OR-set holding two add instances of the same element needs only one once both are known everywhere, and the embedded-chain RGA's coordinates carry dead history that a re-coding epoch could shed (§24). The runtime therefore wants to call back *into the datatype* so the concrete state can compact. The question is what the datatype must prove for such a `compact` to be sound, and what exactly the runtime must promise it. Both halves have sharp answers, and both were shaped by countermodels.

The naive gate is unsound: the discriminating remove. The obvious runtime promise ("the cut S is contained in every current head") does not suffice, by a near miss that shapes everything after it. Adds of element a at stamps $t_1 < t_2$, both below every head. Replica A compacts, dropping the older instance t_1. But a remove, minted long ago by a replica that had observed *only* t_2 (the remove is concurrent with the add t_1), is still at or below some head and arrives later. In the full execution the remove kills only t_2 and the element survives through t_1; in the compacted execution t_1 is gone and the element is absent. Reads diverge (Figure 5). The

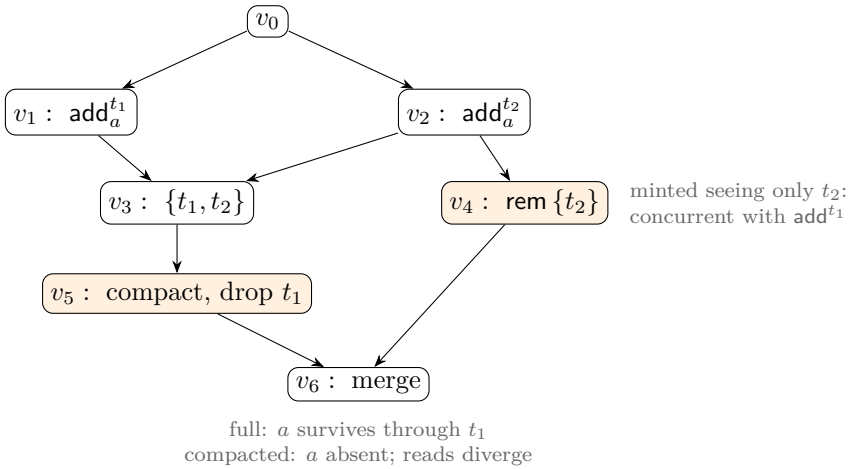


Figure 5: The discriminating remove (top-down; edges parent to child). Both adds of a sit below every head of the compacting replica, yet an in-flight remove that observed only t_2 can still arrive. Dropping the older instance t_1 under the naive "below every head" gate changes the read: the remove kills t_2 alone, and only the full state still holds a. The repair is SettledAt: compaction must wait until every event concurrent with the cut is already absorbed.

countermodel fires mechanically in the validation harness under the naive gate, and once (seed 2026, trial 218) it was found by the randomized twin runs rather than injected.

The interface: SettledAt. The repair is a version-level condition. A cut S is *SettledAt* version v when (i) $S \subseteq E(v)$ and S is causally downward closed, and (ii) every event concurrent with an event of S *that exists anywhere* is already in $E(v)$. Clause (ii) looks like a global quantification the runtime could never check, and in the design note it was carried as a hypothesis the runtime would somehow certify. It is now a *discharged theorem*: on an honestly reachable configuration, the purely local, checkable predicate AllHeardSince(C, v, S) — v has absorbed, for every registered replica j, a commit c_j with $S \subseteq E(c_j)$ — *implies* SettledAt(C, v, S) (`settledAt_of_allHeard`, `EvidenceDischarge.lean`). The argument is the evidence one made rigorous: an event e concurrent with some $s \in S$ was minted by a replica j before j saw s; the absorbed evidence commit c_j has $s \in E(c_j)$, so e precedes c_j at j and $e \in E(c_j) \subseteq E(v)$. The impossible case (were e still outside $E(v)$) would place the cut event s in e's minter's past, contradicting concurrency. This is causal stability in the sense of Baquero, Almeida, and Shoker, transplanted from op-based delivery to the commit DAG: there the argument rides on causal delivery order, here on downward closure of event sets. Two facts make it a genuine discharge rather than a relabeled assumption. The frontier the runtime maintains — the per-replica latest-absorbed commit, meeting to the largest certifiable cut — is tracked *axiom-free*: no choice, no classical step enters the version-level payoff. And createReplica is the *one* breaker: a replica that joins after a compaction has heard nothing since S, so it falsifies AllHeardSince until it catches up, which is exactly the open-membership gate (the deferral of §16 made a side condition of the theorem, not an unproved promise). Two operational sharpenings from the mechanization. The gate is SettledAt *and* all-heard: settledness alone is not stable under future mints by replicas that have not yet heard of S, so the all-heads frontier (the same one §15 tracks) keeps it stable, and replica creation after a compaction is policy-gated (the open-membership deferral made op-

erational). And evidence is *commit*-shaped, not event-shaped: a pull that brings no new events can still bring the evidence commit that opens the gate, so a runtime that skips event-subsumed pulls silently starves the callback.

Two species of compaction. *Drop* (OR-set, MVR, remove-record GC): `compact` removes records; under a live-set merge rule a removed record propagates exactly like a deletion, so compaction spreads through merges with no protocol. *Remap* (the embedded-chain RGA's coordinate re-coding, §24): `compact` rewrites coordinates by an order isomorphism below the cut, and mixed states need translation on ingest. One VC skeleton covers both, with the simulation relation R_S instantiated to "differ only by S-redundant records" for the drop species and to the graph of the coordinate isomorphism for the remap species.

The six VCs. Fix a conditioned MRDT, a cut S, a compaction $\mathrm{compact} : \Sigma \to \Sigma$, and a simulation relation $R_S \subseteq \Sigma \times \Sigma$ (full state on the left, compacted on the right). The bundle (`StabilityVC`):

S1 *Entry.* At a gated version (SettledAt plus all-heard), $\sigma_v \; R_S \; \mathrm{compact}(\sigma_v)$.

S2 *Observation.* $\sigma \; R_S \; \tau$ implies every query of the read interface agrees on σ and τ.

S3 *Step preservation.* R_S is preserved by applying any op minted with S in its causal past (all future ops, by stability). The design note carried a second class here, in-flight ops referencing dropped records; in the state-based ternary model those effects arrive only through merge branch states, so the class is absorbed into S4 and the step VC covers new mints alone.

S4 *Merge congruence, argumentwise and reachability-indexed.* mergeL respects R_S in each argument independently, so that mixed executions (a compacted head merging against a full LCA payload, or against a replica that never compacted) stay related. Argumentwise is forced, because mixed is the normal case. And the congruence is *not* free: unconditioned, it is false for the OR-set instance (a related LCA pair, an A past a remove that killed the kept twin, and a B still carrying the dropped instance live make the two merges read differently). The offending triple is unreachable (any state descending from the compacted LCA sheds the drop at its first absorption merge), so the VC is stated over triples arising at genuine merge premises under an auxiliary pair-invariant threaded through the simulation, not as a free equation.

S5 *Contract preservation.* R_S preserves Inv, the applicable set up to redundant references, and the honesty contract the capstone consumes. This is the clause that composes compaction with the conditioned framework, and the one with no analogue in unverified systems.

S6 *Epoch coherence, observational.* Compacting at S and then at $S' \supseteq S$ lands in the same observational class as compacting at S' directly, so staggered and repeated compactions across replicas stay related. Observational is the correct strength: as a same-R-class statement it is false (a tag dropped at the first epoch whose second-epoch keeper has since died is kept by the direct coarser compaction; reads still agree).

The metatheorem, and the stuttering-self-merge trick. Lifting R_S to configurations (versions pointwise related, DAG shape shared), the six VCs plus the runtime contract give: every step of the execution model preserves the lifted relation, compaction itself included, and every version of every reachable configuration reads identically in the compacted and uncompacted executions (`stability_simulation`, `stability_reads_equal`); the compacted system inherits RA-linearizability up to $\approx$ observationally from the uncompacted capstone (`stability_ra_inherited`). The proof device worth recording: compaction projects, on the full side, to a *stuttering self-merge*. The LCA predicate is reflexive and $E \cup E = E$, so merging a head with itself is a legal step that allocates a new version with the same event set; the full run performs that stutter exactly where the compacted run compacts, and the twin DAGs stay *literally identical* in shape, so no version-map bisimulation is ever needed. That one trick is why the metatheorem's axiom bill is {`propext`, `Quot.sound`}: no choice, no classical reasoning, just the step relation walked in lockstep.

One consequence deserves its own sentence, because it constrains future representation work: *compaction makes the payload no longer a function of the event set.* Resolving an LCA by event set alone is sound before compaction and unsound after (a same-event-set version from an unrelated lineage used as the LCA slot resurrects a dropped instance); the framework's structural LCA already resolves within shared ancestry, so the model is right as it stands, but any representation swap that identifies versions by clocks or event sets must retain enough lineage, or restrict lookup to R_S-compatible payloads.

First instance: the OR-set, and the refuted static drop. The redundancy predicate: among the *live* instances of an element, the adds in S that are not the kept witness. Under SettledAt every remove that could discriminate between two S-instances is already applied (that is Figure 5 inverted), so the surviving S-instances live and die together from here on, and dropping all but a live kept witness preserves every read. Two mechanization findings. The drop set must be *state-dependent* even under the settled gate: a static drop set (computed once from the cut) is unsound when the kept twin has already died before compaction fires, machine-refuted as `static_drop_unsound`; the sound callback is the twin-liveness-guarded drop, and newest-first is not needed for soundness (any live kept witness works). And the interesting S4 case is a remove minted elsewhere against a full state whose kill set names a dropped instance: on the compacted side the kill is a partial no-op, and the states stay related because the dropped instance's fate is determined by the kept one. The instance discharges all six clauses (`orStabilityVC`, `ORSet_stability_reads`); the remap species' standing cut hypothesis is discharged by the same gate (`eRecode_settled_bridge`, the bridge §24 consumes).

Part II
The embedded-chain family

The embedded-chain RGA (replicated growable array) is a replicated list that keeps no tombstones, no ledger, and no event log: its entire state is the live nodes. It is a sequence MRDT in Part I's sense: state-based, reconciled by a three-way merge against the branches' lowest common ancestor version. A deleted element survives only as $\Theta(\log \delta)$ bits inside its descendants' sort keys, δ being the Lamport-timestamp gap between the element and its own insertion anchor (1 under continuation typing); that is the log of the event gap the insertion crossed, a race or a

cursor jump, paid once, in space. Concurrent inserts provably cannot tie, so the merge contains no repair and never decides an order; the state is a canonical function of the event set, and convergence is literal equality. Observationally the datatype *is* the published RGA with the tombstone set compressed into coordinate entropy: a kernel-checked read-equivalence theorem, with machine lockstep at every step as its tested form. All of this follows from one idea: an element's sort key is its *birth chain*, the timestamps from the root of the birth tree down to the element, dead ancestors included. The chain is written at birth, never edited, and carried in an order-preserving self-delimiting code. Separating the datatype from its encoding makes the metatheory nearly definitional: correctness is three one-line facts about chains, plus a single faithfulness theorem about the code. The design is machine-validated (the L1–L25 anomaly battery, except L19, RGA's own, discharged by the sided generalization of §21; 420+ randomized version-DAG executions; 120/120 lockstep), and the central theorem, RA-linearizability at every honestly reachable configuration, is a kernel-checked Lean theorem (§27) discharged by the direct-join route of §13. (Working names in the validation artifact: `embed-tree` for the unary code, `embed-code` for the delta-coded one.)

17 The datatype: sort keys are birth chains

Timestamps $t \in \mathbb{N}^+$ are Lamport stamps, globally unique, and double as element identities. Causality gives: the anchor of every insert carries a smaller timestamp than the insert itself.

The birth tree. Every insert names an anchor. The event set therefore induces a tree: the root is the front sentinel 0; the *birth parent* of each element is its anchor. The tree is append-only and never edited: an element's place in it is fixed at birth.

Birth chains. The *birth chain* of u is the sequence of timestamps from the root down to u (sentinel elided): $\mathrm{chain}(u) = \mathrm{chain}(a) \cdot u$ when u was inserted after anchor a, and $\mathrm{chain}(u) = \langle u \rangle$ at the front. Dead ancestors are *not* elided: their timestamps remain in their descendants' chains. The chain is the sort key, and everything in this section is stated in terms of it; nothing here depends on how chains are stored (§18).

Type. A state is a finite map over *live nodes only*:

$$\sigma \;:\; u \;\mapsto\; (\,\mathrm{chain}(u),\ \mathrm{char}_u\,).$$

init. $\sigma_0 = \varnothing$.

do (the operations). Two operations, with $\mathsf{rc} = \mathsf{Either}$ everywhere (§19):

- $\mathsf{ins}(x \leftarrow a,\ c,\ \pi)$: insert element x (its timestamp) with character c after anchor a ($a = 0$ for the front). The payload $\pi = \mathrm{chain}(a)$ is the anchor's birth chain, carried *for the proof alone* (§19). Application on a state where a is live (every reachable application, by honesty):

  $$\sigma[x] := \big(\mathrm{chain}_\sigma(a) \cdot x,\ c\big), \qquad \pi \text{ unread.}$$

  No head-tracking, no reading of siblings.

- $\mathsf{del}(d)$: remove d's record. Nothing else changes: survivors' chains are untouched; in particular, d's descendants still carry d's timestamp in their sort keys. If d is absent, no-op (concurrent double delete).

read. Sort survivors by *chain-lex*: compare chains timestamp-by-timestamp with *larger first*; a proper prefix precedes its extensions (a live ancestor displays immediately before its descendants). Among same-parent siblings this is exactly RGA's newest-first rule; equivalently, the canonical order is the RGA traversal of the birth tree restricted to survivors.

merge. merge(L, A, B), where L is the LCA (the branches' lowest common ancestor version, the merge context the MRDT model stores and supplies). The merge is *survival* (OR-set): live iff in all three, or born in exactly one branch. There is no second step at this layer: survivors' chains come with their insert events, so every input that holds a survivor agrees on its chain. The merge never compares timestamps and never decides an order.

Inv / applicable / honesty. The honesty contract is the standard RGA generation discipline: an insert is *generated* at a replica where its anchor is live (you insert after a character you can see), and $\mathsf{ins}/\mathsf{del}$ are *applied* on states where they are applicable (anchor live; delete of a live or already-dead node). On every reachable state, application never consults π. Canonicity (Theorem 17.1) doubles as Inv; at the representation layer, Theorem 18.1(iii) is its stored form.

$\approx$. Equality. The state is a canonical function of the event set (Theorem 17.1), so no quotient is needed: convergence is literal state equality.

The metatheory, which is nearly definitional. At this layer the design's theorems all but evaporate; we record them because they are the specification the representation must meet (§18).

Theorem 17.1 (chain-level metatheory)**.** *In every reachable state:*

(i) Canonicity: $\sigma = \{\, u \mapsto (\mathrm{chain}(u),\ \mathrm{char}_u) \,\}$ *over survivors u; the state is a function of the event set.*

(ii) Birth constancy: $\mathrm{chain}(u)$ *is fixed at u's insert and never edited by any later transition.*

(iii) Totality (no ties): chain-lex is a total order on every state's survivors.

Proof. (i) Survivorship is OR-set, hence a function of the event set, and a survivor's chain is determined by its insert event alone: by induction, ins writes $\mathrm{chain}_\sigma(a) \cdot x = \mathrm{chain}(a) \cdot x$ (the anchor is live at application, by honesty, and its stored chain is canonical), while del and the merge only delete or select records, never rewrite them. (ii) is (i) read off per node: no transition writes to an existing record. (iii) Distinct elements' chains are distinct (they end in distinct timestamps); if neither is a prefix of the other, the first difference compares two distinct timestamps in $\mathbb{N}$, and uniqueness of Lamport stamps *is* the no-tie property; prefix pairs are ancestor/descendant, ordered by the prefix rule. □

The one-line separation, and the conservation law. The natural first attempt at tombstone-freedom is the *rehoming* RGA: keep flat (element, anchor) records, and let a delete splice its node out, re-parenting the orphans to the nearest live ancestor (the *climb*). We built and verified that design before this one (it is RA-linearizable, in the same Lean framework), and it fails the L1 delete-reorder litmus (the $[b, a, c] \to [c, b]$ anomaly, Figure 6): the climb re-sorts a rehomed node by its own timestamp, so *its sort key is the birth chain truncated at each rehoming*. This design keeps the chain whole. That is the entire difference between the two datatypes, and it is the crisp form of the conservation law that our design sweep kept re-imposing: *the sort key must retain dead ancestors' timestamps*. Every design that forgets them has a named machine-checked countermodel (§26). What is negotiable is only how many bits the retained timestamps cost; that is the subject of §18, and the answer is: the log of each element's anchor gap, and nothing else.

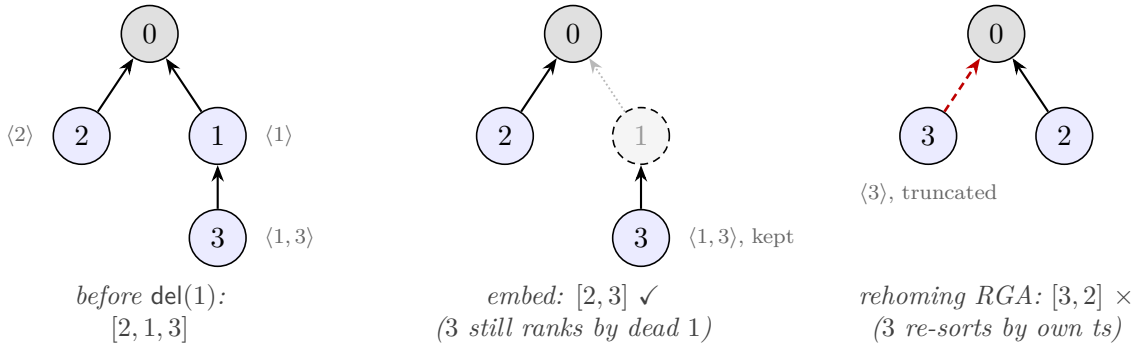


Figure 6: The one-line separation on the L1 delete-reorder litmus: insert 1 and 2 at the front, 3 after 1, delete 1. The rehoming RGA's climb (red) re-parents 3 and re-sorts it by its *own* timestamp: the truncated key $\langle 3\rangle$ beats $\langle 2\rangle$, and 3 jumps over 2. The embedded chain $\langle 1, 3\rangle$ keeps ranking 3 by its dead parent's timestamp: $[2, 3]$, the delete-order-preserving read. Arrows point from a node to its anchor; dashed grey = dead.

Worked example (Figure 7). 6 and 10 concurrently insert at the front: chains $\langle 6\rangle$ and $\langle 10\rangle$; display $[10, 6]$, no arbitration, at every replica and merge order. Type 22, 16 under 6: chains $\langle 6, 22\rangle$, $\langle 6, 16\rangle$. Delete 6: the records of 22 and 16 are untouched and still carry 6's timestamp. A third party concurrently inserts 8 at the front: $\langle 8\rangle$ sorts below $\langle 10\rangle$ and above $\langle 6, 22\rangle$, so the display is $[10, 8, 22, 16]$ whether 8 arrives before or after 6's death (machine-checked as the *credential countermodel*, the shape that kills every timestamp-oblivious variant, §20).

18 The representation: chains as entropy-coded coordinates

Everything above is the datatype; everything below is bits. A naive carrier would store a full timestamp at every chain level (each $\sim \log$(document age) bits, even for purely sequential typing) and compare whole chains at every read. This section stores the chains compactly without disturbing their order.

Vocabulary. A *codeword* is the bit string $C(\delta)$ encoding one chain level: one element's timestamp, stored as its differential δ to its birth anchor (step 1 below). A *coordinate* is the bit

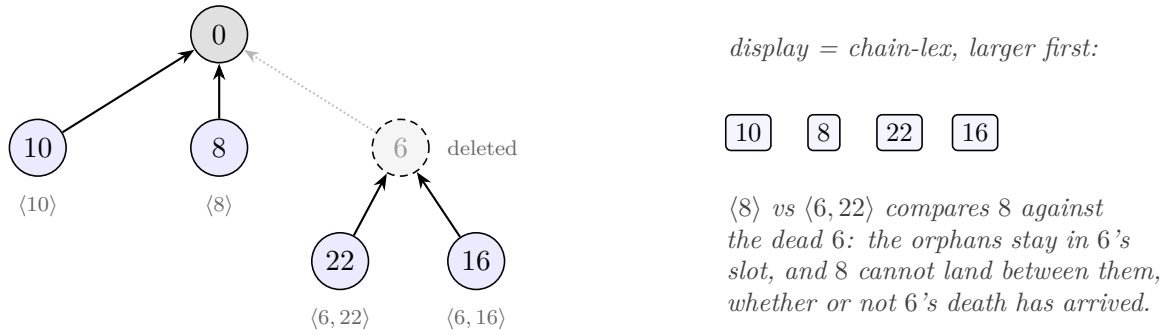


Figure 7: The credential countermodel as a birth tree. 6 and 10 race at the front; 22, 16 are typed under 6; 6 dies; 8 arrives at the front concurrently. Survivors sort by whole birth chains, so the read is $[10, 8, 22, 16]$ at every replica and merge order. Every timestamp-oblivious variant flips this shape (§26).

string encoding a whole chain: the concatenation, root to element, of one block per level (the level's codeword plus three framing bits fixed by the mint construction below), with nothing else marking the boundaries. We write $|s|$ for the length in bits of a bit string s. Read as a binary fraction, a coordinate names a dyadic interval in $(0, 1)$; codeword concatenation becomes interval nesting, and the interval is the $\alpha(u)$ of Theorem 18.1. Bit string and interval are two views of one object; we use whichever is clearer.

What the coding is for. Two jobs, only one of which is about correctness. *Job 1: cost.* The datatype layer already forced sort keys to retain dead ancestors' timestamps (the conservation law, §17); the only remaining question is how many bits that retention costs. Without entropy coding, "tombstone-free" would be bookkeeping: a full timestamp per level grows with document age however calm the history. The *unary control* (`embed-tree`: the identical datatype with a unary code in the mint, codeword length linear in the value, kept as the experiment's cost-control arm; step 2 and the entropy-law table) makes the gap measurable: 61 KB against the coded design's 0.5 KB on the table's 1000-element chain scenario. The coding is what makes the headline claim true: the dead survive as the entropy of the anchor gaps they were born across, and not more. *Job 2: faithfulness.* The chains must be stored so that plain lexicographic bit comparison reproduces chain-lex exactly, in every reachable state, forever. This is the single obligation the representation owes §17 (Theorem 18.1). Correctness lives upstairs; this layer is an implementation that must not lie. Job 2 needs no entropy optimality at all: it needs exactly two structural properties of the per-level code, stated next.

The contract. The stored form must support four operations: *mint* (insert: produce the new codeword from the two timestamps alone, coordination-free, reading no siblings); *compare* (read: plain lexicographic comparison, with no tie rule); *fold* (delete: splice a dead node out by concatenating its block into each child's stored string, exactly); and *refold* (merge: recompute survivors' coordinates by the same concatenation). Each guarantee is bought by a specific property of C:

- **Monotone** ($\delta < \delta' \Rightarrow C(\delta) <_{\text{lex}} C(\delta')$) buys correct *compare*. A chain-lex first difference always compares two same-anchor entries, where timestamp order and δ order coincide

(step 1); bitwise comparison of coordinates therefore equals timestamp comparison at the first differing level precisely when C is monotone.

- **Prefix-free** (no codeword is a prefix of another) buys three things at once. *Alignment*: in a comparison of two coordinates, the first differing bit falls inside the first differing level's codewords, so levels cannot be confused across boundaries and no delimiters are needed; this is also what keeps *fold* and *refold* exact splices. *Geometry*: by the Kraft correspondence, prefix-free codewords occupy pairwise-disjoint dyadic cells, which is Theorem 18.1(i) and the reason a concurrent front-insert can never land inside a dead sibling's span (Figure 9). *No ties*: cell disjointness plus Lamport uniqueness means sibling coordinates never tie, so the merge never decides an order and the read needs no tiebreak.
- **Universal, with a cheap floor** buys Job 1: C is defined for every $\delta \geq 1$, $|C(1)| = O(1)$ so continuation typing costs a constant per level, and $|C(\delta)| = \Theta(\log \delta)$ so a level pays the log of its anchor gap, whether the gap is a race or a cursor jump (step 1), and nothing else.

Monotone plus prefix-free is all of Job 2: any such code satisfies Theorem 18.1, which is why the Lean capstone is parametric in the code (§27) and why the unary control proves the same theorems at absurd cost. Entropy coding enters only in *which* monotone prefix-free code is chosen; the choice sets the constant in the bits-per-level bill and nothing in the correctness story. One requirement cuts across all four operations: the arithmetic is exact, never rounded (see the binary64 refutation below).

Step 1: differential coding. By causality $\delta = x - a \geq 1$. A chain-lex first difference always compares two entries with the *same* anchor (the shared prefix), and for a fixed anchor, ordering by x and by δ coincide, so each chain level may store the differential instead of the timestamp. Call δ the *anchor gap*: the events elapsed between the anchor's mint and the insert's. Continuation typing (each character anchored at its predecessor) has $\delta = 1$ at every level regardless of how long the document has lived. Two things widen the gap, and they pay the same bill: a genuine race with timestamp gap g costs $\log_2 g$ bits, and so does a *cursor jump*, the first character typed at an old anchor; its successors anchor at fresh characters and are back to $\delta = 1$. The differential is an encoding choice, not semantics: the mathematical object remains the timestamp.

Step 2: entropy coding. For $\delta \geq 1$ let L be the bit-length of δ and

$$C(\delta) \;=\; \underbrace{1\cdots1}_{L-1}\;0\;\big(\delta \text{ with its leading bit removed}\big) \qquad (|C(\delta)| = 2L-1),$$

an order-preserving prefix-free code: exactly the contract. The length accounting: prefix-freedom makes each codeword self-delimiting, and a self-delimiting code over unbounded δ must spend bits twice, once on the value and once announcing its own length. C pays each bill at $\sim \log_2 \delta$. The header $1^{L-1}0$ is a unary announcement "$L{-}1$ payload bits follow" (L bits); the payload is δ's trailing $L{-}1$ bits, since the leading bit of an L-bit number is always 1: once the header has said L it carries no information and is dropped. Hence

$$|C(\delta)| \;=\; \underbrace{(L{-}1)+1}_{\text{header}} \;+\; \underbrace{(L{-}1)}_{\text{payload}} \;=\; 2L-1 \;=\; 2\lfloor \log_2 \delta \rfloor + 1,$$

e.g. $C(1) = 0$, $C(2) = 100$, $C(3) = 101$, $C(5) = 11001$. This is Elias γ with its header flipped: γ announces the length in *zeros*, which is prefix-free but order-reversing across length classes: at the first divergence the longer, hence larger, number shows a header 0 against the shorter's leading payload bit 1. Writing the header in ones makes that same position decide in favor of the longer code, so lexicographic order on codewords is numeric order on δ. The γ-flip is the building block, not the last word: only the *value* bill is forced at $\log_2 \delta$, and the *length* bill can itself be entropy-coded. Applying C to the length field (then the payload) gives the order-preserving flip of **Elias** δ, prefix-free and monotone by the same across-length-class argument, at $\log_2 \delta + O(\log \log \delta)$ per level; *this is the canonical mint. Any* order-preserving prefix-free code works here, and the artifact exists in three proved encodings of one datatype (`embed-tree` unary, `embed-code` the γ-flip, `eliasDeltaCode` the δ-flip); the capstone theorem is inherited at each verbatim, with zero new proof content (§27): the code-parametricity claim, machine-validated. Two honest footnotes: per level the δ-flip wins only from $\delta \geq 32$ and loses at $\delta \in \{2, 3\} \cup [8, 15]$ (kernel-checked length comparisons), and measured editing tails are thin, so on real traces the two flips are a wash; the dominance shows in race-heavy shapes (the entropy-law table below) and in the asymptotics.

The mint. Writing $b = 1 \cdot C(\delta)$ (a leading 1 keeps coordinates positive) and $w = 2^{-|b|-2}$, the mint is the second quarter of b's dyadic cell:

$$M(\delta) \;=\; \big(\, 0.b + w, \;\; 0.b + 2w \,\big) \;\subset\; \big[\, 0.b, \;\; 0.b + 4w \,\big).$$

The quarter placement leaves a gap below and headroom above inside the cell, and the open space $(0.b + 2w,\ 0.b + 4w)$ is never minted: it exists so that the interval has room to *contain* the node's own subtree without touching the next cell. This also pins down the per-level *block* promised in the vocabulary: $0.b + w$ written as a string is $b\,\texttt{01}$, so the block is $1\,C(\delta)\,\texttt{01}$, the codeword in its three framing bits (the leading 1 for positivity, the `01` that selects the second quarter). The block is what a node stores; read as a binary fraction it is the lower endpoint of $M(\delta)$, and flipping the trailing `01` to `10` gives the upper. Worked out for small deltas:

δ	$C(\delta)$	block $= 1\,C(\delta)\,\texttt{01}$	$M(\delta)$	bits/level
1	`0`	`1001`	(9/16, 10/16)	4
2	`100`	`110001`	(49/64, 50/64)	6
3	`101`	`110101`	(53/64, 54/64)	6
4	`11000`	`11100001`	(225/256, 226/256)	8
5	`11001`	`11100101`	(229/256, 230/256)	8
10	`1110010`	`1111001001`	(969/1024, 970/1024)	10
100	`1111110100100`	`1111111010010001`	(65169/65536, 65170/65536)	16

Three things to read off. The endpoints are consecutive dyadics at scale $2^{-(|b|+2)}$ (numerators n and $n{+}1$): the mint is one tick wide at its own scale, with the tick below it (the gap) and the two ticks above it (the subtree headroom) never minted. The rows are strictly increasing as fractions: monotonicity made visible. And the bit count is $2L + 2$ for an L-bit δ: four bits at $\delta = 1$ (the sequential-typing constant of the entropy law), sixteen at $\delta = 100$.

The stored state. The carrier stores each live node parent-relatively:

$$\sigma \;:\; t \;\mapsto\; (\,\text{par} \in \mathbb{N}, \;\; s \in \{0,1\}^*, \;\; \text{char}\,),$$

Figure 8: Mints $M(\delta)$ for $\delta = 1..4$ inside the parent's unit interval (cell boundaries dotted). Larger deltas sit strictly higher; distinct deltas are disjoint with gaps; each mint is a quarter of its cell, leaving room for the node's own subtree. Drawn for the γ-flip C; the construction reads verbatim over any order-preserving prefix-free code, in particular the canonical Elias-δ mint (step 2).

where s is a block ($1\,C(\delta)\,\texttt{01}$ at birth; after folds, below, a concatenation of blocks) *relative to the parent*, and par $= 0$ denotes the root sentinel (which owns $(0, 1)$ and is never stored). The record holds no interval; the interval is the string's decoding,

$$(lo, hi) \;=\; \big(0.s,\;\; 0.s + 2^{-|s|}\big) \;\subset\; (0, 1),$$

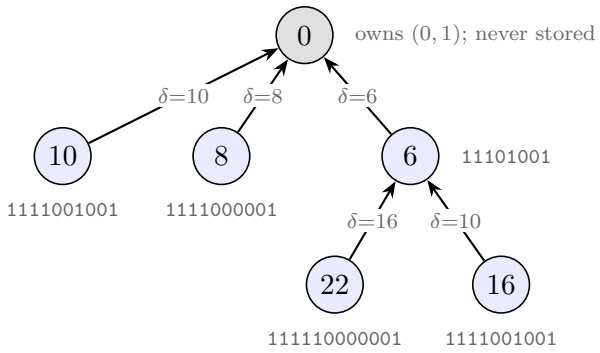
consecutive dyadics at s's own scale, one block at a time exactly as in the mint table. The maintenance below and Theorem 18.1 are stated on the intervals, and every operation they perform on an interval is a string operation on s: composition is concatenation, comparison is lexicographic. (The validation model stores the decoded pair directly, as exact fractions.) A node's *absolute* interval is the affine composition of the intervals along its parent chain. For a node v with mint $M(\delta_v) = (lo_v,\; lo_v + w_v)$, let

$$\varphi_v : (0, 1) \to M(\delta_v), \qquad \varphi_v(x) \;=\; lo_v + w_v \cdot x,$$

the unique increasing affine bijection of the unit interval onto v's mint; it maps a subinterval (a, b) to $(lo_v + w_v a,\; lo_v + w_v b)$, and on strings it prefixes v's block: $\varphi_v(0.y) = 0.(s_v\, y)$. Write

$$\alpha(u) \;=\; \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}\big(M(\delta_u)\big)$$

for the composition along u's birth chain $c_1 \cdots c_k\, u$ (δ_v is v's delta to its birth parent): $\alpha(u)$ is the *encoding of* chain(u), and at the string level it decodes the concatenation of the blocks along the chain. Worked on the tree of Figure 7, redrawn before del(6) with the delta (child minus anchor) on each edge and the stored block under each node:



Writing $(n, n{+}1)/2^k$ for the interval $(n/2^k,\; (n{+}1)/2^k)$ and $\approx$ for its lower endpoint:

node	δ	stores (par, s)	decode of s	α (absolute)	$\approx$
10	10	(0, 1111001001)	$(969, 970)/2^{10}$	$(969, 970)/2^{10}$	0.94629
8	8	(0, 1111000001)	$(961, 962)/2^{10}$	$(961, 962)/2^{10}$	0.93848
6	6	(0, 11101001)	$(233, 234)/2^{8}$	$(233, 234)/2^{8}$	0.91016
22	16	(6, 111110000001)	$(3969, 3970)/2^{12}$	$(958337, 958338)/2^{20}$	0.91394
16	10	(6, 1111001001)	$(969, 970)/2^{10}$	$(239561, 239562)/2^{18}$	0.91385

The root's children store their absolute intervals already (par = 0). The grandchildren compose: 22's absolute string is its parent's block followed by its own, 11101001 111110000001 (20 bits, hence the 2^{20}), and likewise 16's (18 bits). Both absolute intervals sit inside $M(6) = (0.91016, 0.91406)$ while 8's sits above it, so the descending-α read is $[10, 8, 22, 16]$ and 8 cannot land between the orphans. Nodes 16 and 10 store the same string, both being $\delta = 10$ births; records are parent-relative, so this is no collision. After $\mathsf{del}(6)$ the fold rewrites the two records to (par 0, the concatenated s) and the α column is unchanged bit for bit. Figure 9 draws these intervals (widths schematic).

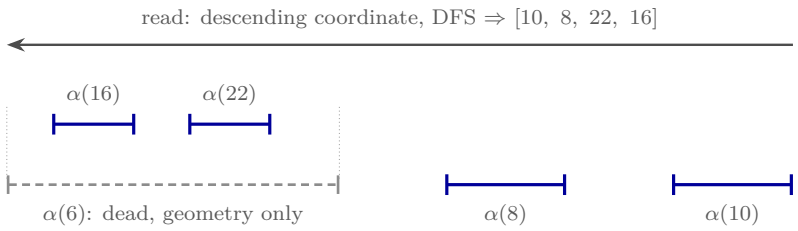


Figure 9: Figure 7 downstairs (widths schematic; true mints are the dyadic cells of Figure 8). Deleting 6 removes its record, but $\alpha(16)$ and $\alpha(22)$ are compositions *through* $M(6)$: the dead node survives as the dashed container, the tombstone compressed into geometry. 8's interval can never land inside $\alpha(6)$'s span: distinct deltas occupy disjoint cells (Theorem 18.1(i)), which is the credential verdict, now a fact about intervals.

Maintenance. How the stored form tracks the datatype:

- $\mathsf{ins}(x \leftarrow a)$ mints $M(x - a)$ relative to a: the new record depends only on the two timestamps.
- $\mathsf{del}(d)$ *isometrically folds* d into its children: each child c is re-parented to d's parent and d's record is substituted into c's. On the stored strings this is concatenation, $c.s := d.s\, c.s$; read as intervals, $c.\text{frac} := d.\text{frac} \circ c.\text{frac}$, where $I \circ J$ is the image of J under the increasing affine bijection of $(0, 1)$ onto I (the φ of the stored state, with I in place of a mint): for $I = (lo,\ lo + w)$,

$$I \circ J \ = \ (\, lo + w \cdot lo_J, \ \ lo + w \cdot hi_J \,).$$

  The substitution is exact. Upstairs, deletion changes nobody's sort key (§17); the fold is the parent-relative storage scheme *re-normalizing* while every absolute interval stays fixed (Figure 10): representation maintenance, not datatype semantics. On the running example, $\mathsf{del}(6)$ removes 6's record and substitutes it into both children: 22's record

$(6,\ \texttt{111110000001})$ becomes $(0,\ \texttt{11101001 111110000001})$, whose decode is

$$\begin{aligned}(233, 234)/2^8 \circ (3969, 3970)/2^{12} &= (233 \cdot 2^{12} + 3969,\ 233 \cdot 2^{12} + 3970)/2^{20} \\ &= (958337, 958338)/2^{20},\end{aligned}$$

and likewise 16's becomes $(0,\ \texttt{11101001 1111001001})$, decoding $(239561, 239562)/2^{18}$. Both are exactly the α column of the worked table: the absolute intervals did not move, only the parent-relative bookkeeping did.

- The merge *refolds*: for each survivor, compute its absolute interval by folding its record to the root inside the input that holds it (a record's stored parent is always live in its own input, so the chain stays in one input; by Theorem 18.1(iii) the choice of input cannot matter, asserted at runtime and never fired); re-parent to the nearest *surviving* ancestor; re-relativize.
- The read is depth-first from the root, children in descending order of *lo*. No tie rule is needed (Corollary 18.2).

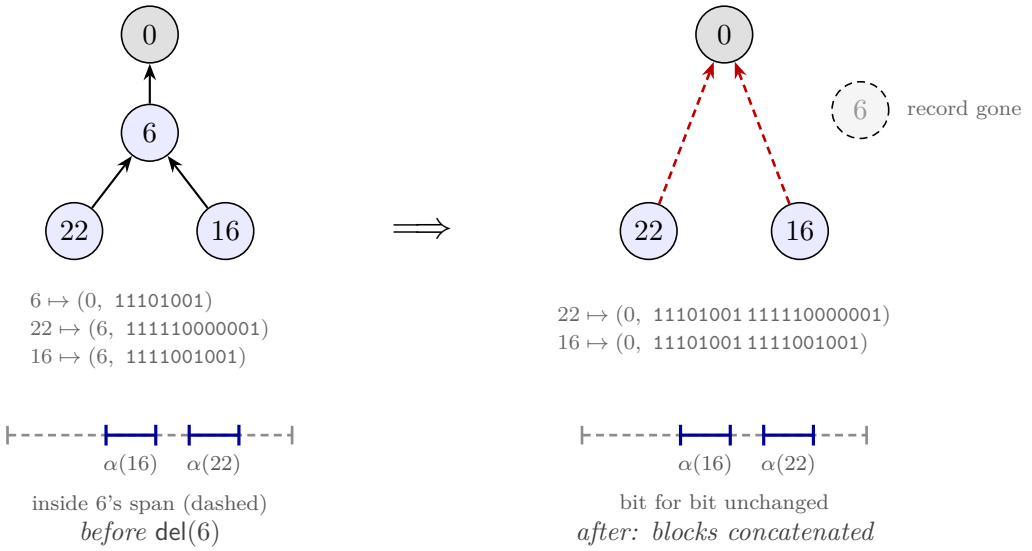


Figure 10: The isometric fold on the running example. del(6) deletes 6's record and concatenates its block `11101001` onto the front of each child's, re-parenting both to the root (red: the re-parenting, a storage move only). The stored states change: 22 and 16 now carry two-block strings and par 0. The absolute intervals $\alpha(22) = (958337, 958338)/2^{20}$ and $\alpha(16) = (239561, 239562)/2^{18}$, and with them the display, are bit-for-bit unchanged: upstairs, the sort keys still read $\langle 6, 22\rangle$ and $\langle 6, 16\rangle$. The dashed span is 6's old interval, surviving as pure geometry.

Why the codeword must announce its own end. Before the theorem, the countermodel that motivates it. A single codeword in a field of known extent needs no header, and the length looks recoverable from the string itself; plain binary shows what breaks, once per purchase in the contract. *Geometry:* $B(2) = \texttt{10}$ prefixes $B(5) = \texttt{101}$, so $B(5)$'s cell sits inside $B(2)$'s;

containment means ancestry, so the δ=5 sibling lands amid the δ=2 sibling's descendants and non-interleaving dies with everything alive. *Parsing:* once a fold erases the structural boundary between blocks, `101100…` reads as $B(2)$ then `11…` or as $B(5)$ then `00…`, and nothing outside the string can say. *Ties:* a binary fraction cannot see trailing zeros, so `10` and `100` both denote $\frac{1}{2}$; distinct siblings mint touching intervals, and the id tiebreak wakes into an arbiter that orders by id, not chain-lex. Nor can the header bits be dodged: prefix-freedom demands $\sum_\delta 2^{-\ell(\delta)} \le 1$, and plain binary puts 2^{L-1} values at every length L, so every prefix-free code pays a length announcement of order $\log L$ somewhere (out-of-band boundary tables cost the same bits and forfeit the string as a self-contained key; database key encodings inline it for the same reason). And the fold names where the boundary went: the tree node that carried it structurally is the deleted tombstone, whose ordering verdict becomes the payload and whose boundary becomes the header. "Tombstone compressed into coordinate entropy" is literal at the bit level.

The one obligation. The four theorems of the earlier drafts collapse into a single statement about the encoding.

Theorem 18.1 (encoding faithfulness)**.** *(i) Order-embedding at one level: for $\delta \neq \delta'$, $M(\delta)$ and $M(\delta')$ are disjoint with a gap; $\delta < \delta' \Rightarrow M(\delta)$ lies entirely below $M(\delta')$.*

(ii) Homomorphism: α realizes chain concatenation as affine composition and embeds chain-lex in the interval order: for distinct u, v, if $\mathrm{chain}(u)$ *is a proper prefix of* $\mathrm{chain}(v)$ *then $\alpha(v) \subset \alpha(u)$ (and, if both are live, the two are never read-time siblings); otherwise $\alpha(u) \cap \alpha(v) = \varnothing$, with the chain-lex-greater element's interval strictly higher.*

(iii) Exact maintenance: in every reachable state, every live node's stored records compose to exactly $\alpha(u)$.

Proof. (i) C is prefix-free: codes of lengths $2L-1 < 2M-1$ differ at position L (the shorter's header terminator 0 against the longer's header 1); equal-length codes differ in the payload. Prefix-free codes have pairwise-disjoint dyadic cells. C is monotone as a binary fraction: within a length class the payload is δ's low bits in order; across classes the position-L bit decides in favor of the longer code. Disjoint cells ordered by their codes, and mints interior quarters of their cells, give both claims. (ii) If neither chain is a prefix of the other, they first differ on distinct deltas relative to the *same* birth ancestor (§18, step 1); (i) separates them there, and affine images of disjoint intervals are disjoint and order-preserved (each φ is increasing). If $\mathrm{chain}(u)$ prefixes $\mathrm{chain}(v)$, then $\alpha(v) \subseteq \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}(\varphi_u((0,1))) = \alpha(u)$ by construction; and if both are live, v's nearest live ancestor is u or deeper, so they never share a current parent, and sibling groups are pairwise disjoint. (iii) By induction over transitions. ins establishes it by definition. del's fold replaces $(\mathrm{par}, \mathrm{frac})$ pairs by their exact composition, leaving α unchanged. The merge recomputes precisely $\alpha(u)$ (folding a record chain to the root) and re-relativizes against another α; ties cannot exist by (i) and (ii), so no step of any transition ever rewrites an interval by anything but exact composition. □

Corollary 18.2 (display ≡ chain-lex; no tie rule)**.** *The read (DFS by descending lo) enumerates survivors in exactly chain-lex order: per level, larger delta first, so among same-parent siblings, exactly newest first; survivors rank by their own timestamps, folded heirs by their dead founder's. Sibling lo-values never tie; the read's id tiebreak is dead code. Machine checks: pairwise sibling*

disjointness asserted at every read of 320 *randomized executions, all clean; over* 19,531 *multi-member sibling groups, own-timestamp order violated* 3,481 *times (exactly the post-fold heir groups; forcing own-ts there is the refuted dissolution behavior),* ***chain-lex violated*** 0 ***times.***

The entropy law. A coordinate is the concatenation of the codes along the birth chain: $\Theta(\sum_i \log \delta_i)$ bits, the entropy of the chain. Continuation typing ($\delta = 1$ at every level) costs $\sim$4 bits per level under every code here; a level minted across an event gap g, by a race or by the first character after a cursor jump, costs $\log_2 g + O(\log \log g)$ bits under the canonical Elias-δ mint ($2 \log_2 g + 1$ under the γ-flip). *The state pays for the magnitude of edit non-locality, concurrent or not, and for nothing else.* Measured (1000 elements; the numbers are *order metadata alone*, survivors' coordinate bits read off the dyadic denominators; characters are excluded, since data costs are identical across all designs):

scenario	quarter carve (refuted)	unary mint	γ-flip mint	**Elias-δ mint**
chain then delete-all (δ=1)	2^{2001}	2^{502501} ($\sim$61 KB)	2^{4001} ($\sim$0.5 KB)	$\mathbf{2^{4001}}$ ($\sim$0.5 KB)
1000 root siblings (δ=t)	— (ties)	max 2^{1003}, 125 KB	max 2^{23}, 5 KB	**max $\mathbf{2^{20}}$, 4 KB**

The carrier. The natural representation is a **parent-relative bit string per node**: the fold is concatenation, comparison is lexicographic ($O(1)$-ish with structure sharing); the rationals are the semantics of the strings, kept for the proofs. Nothing here computes $M(\delta)$: the endpoints $0.b + w$ and $0.b + 2w$ are the strings $b\,\texttt{01}$ and $b\,\texttt{10}$, so the mint emits a block and the fold concatenates, and *hi* need not be stored at all. The Lean mechanization drops even the framing bits: its coordinate is the bare codeword concatenation on `List Bool` (`mint` appends $\Gamma.\mathrm{enc}(t-a)$ to the anchor's coordinate), with the prefix rule stated directly on strings; the quarter framing serves the interval realization alone, buying positivity and subtree interiority. The rational $M(\delta)$ is computed in exactly one place, the validation model `embed_tree.py`, as exact fractions; the entropy tables read their bit counts off those denominators. IEEE 754 binary64 is machine-refuted as the carrier, a faithfulness failure rather than a design failure: mints degenerate at $t = 52$, depths underflow at 44, and rounding voids Theorem 18.1(iii) (6/120 PBT executions die). The obstruction is a pigeonhole (chains carry $\Theta(\sum \log \delta)$ bits; a fixed-width word holds 64), but binary64 is sound as a *comparison cache* with exact fallback on float equality.

Timestamps in practice. The unique stamps in $\mathbb{N}^+$ are Lamport's 1978 construction, flattened: each replica ticks a counter per local operation, advances it to the max on merge, and breaks ties by replica id, packed as $t \cdot 2^k + r$ with k id bits. Uniqueness and causality ($t_a < t_x$ gives $\mathrm{packed}_a < \mathrm{packed}_x$ regardless of ids, so $\delta \geq 1$) both survive the packing; the proof layer only *assumes* uniqueness (the Lamport-structural part of honesty), and this construction is the witness that the assumption is realizable. Two entropy caveats. *(i)* Packing inflates every delta by 2^k, costing $\sim 2k$ extra bits per chain level, which turns the 0.5 KB sequential run above into $\sim$3 KB at $k = 10$. Cheaper: extend the mint's code to the pair, $C(\Delta t)$ followed by a k-bit id. The pair code is still order-preserving prefix-free (lex on $(\Delta t, r)$), and costs k bits only where the level is minted. A further option, unvalidated against the battery and recorded as a design note, is to order sibling ids *relative to the anchor's* (any deterministic per-anchor id order is a legitimate tiebreak; convergence needs agreement, not a uniform order), making "same author as my anchor", i.e. sequential typing, cost $\sim$1 bit, so only genuine cross-author races pay $\log R$.

(ii) The clock must be *dense logical time, never physical*: wall-clock or HLC stamps make δ measure elapsed time rather than elapsed events: a keystroke every 100 ms over μs-resolution stamps mints $\delta \approx 10^5$, $\sim$34 bits per level for a purely sequential edit. The entropy law holds precisely because the counter ticks once per event, making δ an event-count gap; if wall-clock order is ever wanted, it belongs in a display-layer tiebreak, never in the minted coordinate.

What the storage layer now delivers. Dead chain entries concatenated into survivors' strings are the conservation law's irreducible content (§17); at the time of the original design note, reclaiming them under *causal stability* (renumber once every replica has seen the delete, the only condition under which rewriting geometry is safe, since no verdict involving the dead can be contested again) was the deepest of the open cost items. That item is now closed, and the answer occupies its own section (§24): a verified re-coding theorem cluster at settled cuts (rank renumbering and spine fusion, gated by §16's SettledAt), and, one level further down, the run table, a lossless re-representation of the live chains that collapses the per-record cost by one to two orders of magnitude with *no* stability gate at all. The residual headroom (per-anchor context codes, Steiner-sharing dead prefixes across co-heirs, the Elias-δ constant of step 2) remains catalogued in the companion assessment `whiteboard/order-coding-compression-notes.md`.

19 The insert prefix: for the proof, and the proof alone

The insert payload π exists for verification, not execution. This section says why it must exist, and why an implementation may drop it from the wire.

Why rc must be Either. The verification framework (Part I) resolves a concurrent pair of non-commuting operations by consulting a declared conflict-resolution order rc; any *oriented* entry for an insert/delete pair, however, incurs a base commutation obligation that is machine-refuted for this design space: ordering $\mathsf{ins}(x \leftarrow a)$ against $\mathsf{del}(a)$ (in either direction) is undischargeable as a conflict entry, because rehoming is non-transitive across chains of deletions. So rc = Either: the proof must show the two *commute*, in both orders, on all applicable states.

The commutation, and where the prefix earns its keep. Order 1, $\mathsf{ins}(x \leftarrow a)$ then $\mathsf{del}(a)$: x is born with chain $\pi \cdot x$, and the delete touches no chain (downstairs: the fold re-homes x's record with composed coordinates; by exactness $\alpha(x) = \alpha(a) \circ M(x - a)$). Order 2, $\mathsf{del}(a)$ then $\mathsf{ins}(x \leftarrow a)$: a is gone; application must still be *total* and land x on the same sort key. This is what the carried prefix $\pi = \mathrm{chain}(a)$ (equivalently, the anchor's coordinate) provides: apply sets $\mathrm{chain}(x) = \pi \cdot x$ (downstairs: $\alpha(x) = \pi \circ M(x - a)$, attached to the nearest live ancestor). The two orders produce *equal* states: commutation to equality, not merely up to $\approx$, courtesy of canonicity (Theorem 17.1). Figure 11 draws the square on the smallest instance.

Ghost, in the strict sense. On every reachable state the anchor is live at application (honesty), and the application reads only the state and the two timestamps: π is never consulted. The Lean artifact carries this as `ins_prefix_ghost` (`Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean`): on accurate ops the always-π transition and the state-reading transition are literally the same function: the two conventions of this section coincide on every honest state. An implementation may therefore drop π from the wire entirely;

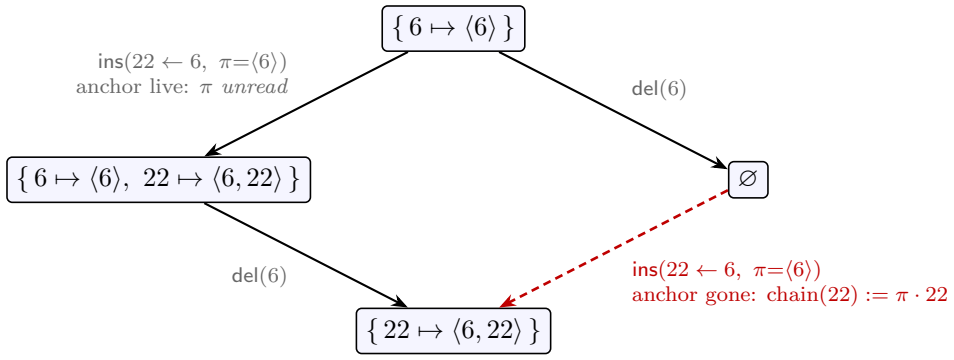


Figure 11: rc = Either discharged: ins(22 ← 6) and del(6) commute *to equality*. Down the left, the anchor is live and application never consults π (the reachable, honest order). Down the right, 6 is already gone: the state cannot supply the anchor's chain, so the carried $\pi = \text{chain}(6)$ does, landing 22 on the same sort key. The right-hand state is unreachable under honesty; the verification condition quantifies over it anyway, and π exists exactly to make that corner total.

the *proof* may not, because the verification conditions quantify over reorderings that visit unreachable states, and there application without π is partial. The prefix costs $O(\sum \log \delta)$ bits, the same entropy as the coordinate itself, and exists at the proof layer alone.

20 Validation

All numbers reproduce from `whiteboard/litmus/` in the accompanying Sal repository (`embed_tree.py`; harnesses `litmus.py`, `pbt.py`). Notation: $\text{RGA}_\dagger$ is the published tombstoned RGA (Roh et al.; §26); the *rehoming* RGA is the convergence-verified flat tombstone-free design of §17 (since demoted repo-wide: the L1 row below was generalized into the kernel-checked sequential refutation of Part III, §33, and this design took the canonical seat). The L1–L25 battery is the litmus suite accumulated over this project's design sweep; each entry is the named countermodel that killed at least one of 16 prior candidate designs. The one exception below, L19, is discharged by the sided generalization of §21.

check	verdict
litmus battery L1–L25 (incl. every countermodel that killed 16 prior designs: ceiling escapes, tie inheritance, three-branch topology, frame mixing, fold-verdict inheritance)	clean, except one-sided L19 (backward-run interleaving, the published RGA's own anomaly)
credential countermodel (tie heirs vs mid-ts third party, death-before vs death-after topologies)	converges, no flips; = $\text{RGA}_\dagger$'s reads
subordination-escape and retroactive-subordination CEs	survive
randomized version-DAG PBT (FLIP / CONV / LIVE / DUP)	120/120 and 300/300 clean
per-read sibling-disjointness assertion sweep	320/320 clean
chain-lex instrumentation (19,531 sibling groups)	0 violations
lockstep vs the published tombstoned RGA	**read-equal at every apply/merge/read, 120/120**
L1 delete-reorder vs the verified rehoming RGA	embed $[2,3]$ ✓ vs rehoming $[3,2]$ ×

The lockstep row is the identification: *the embedded-chain RGA is observationally the published*

tombstoned RGA with the tombstone set compressed into coordinate entropy, machine-checked on 120 executions and now a kernel-checked theorem (the compaction theorem, §27): on every honest event set, any enumeration of the tombstoned RGA's visible ids in its own visible order *equals* the embed read, element for element. The L1 row is the separation from the verified rehoming RGA, and it is exactly the one-line difference of §17: that design's climb re-sorts a rehomed node by its own timestamp (the $[b, a, c] \to [c, b]$ anomaly), i.e. its sort key is the birth chain *truncated at each rehoming*; this design keeps the chain whole.

21 The sided generalization: two-sidedness as a parameter

The one blemish in the battery is L19: two replicas that each type a *backward* run at the same place (repeatedly "insert before the current head") interleave, because every insert-before anchors at the same left neighbor and both runs land in one sibling group. The fan is not a traversal artifact: no read order of the same birth tree un-fans a fan, so any fix must change where insert-before *anchors*. The Fugue family's answer is a second side: insert-before anchors at the *right* neighbor as an L-entry, so a backward run is a chain, exactly as a forward run already is.

The L19 example, concretely. It is the battery's own trace (Figure 12). The LCA document is $\langle 1 \rangle$ (one insert, stamp 1, anchored at the start marker $\diamond$). Then, concurrently, replica A types a backward run at the head, three inserts each anchored "before the current first element":

$$\mathsf{ins}(10 \text{ at head}), \quad \mathsf{ins}(30 \text{ at head}), \quad \mathsf{ins}(50 \text{ at head}),$$

so A locally reads $\langle 50, 30, 10, 1 \rangle$; replica B does the same with stamps $21, 41, 61$ and locally reads $\langle 61, 41, 21, 1 \rangle$. (The stamps interleave numerically, $10 < 21 < 30 < 41 < 50 < 61$, because the two replicas' Lamport clocks advance together.) In the one-sided datatype, "insert at the head" can only anchor at the left neighbor, which is $\diamond$ for every one of the six inserts: all six become R-children of $\diamond$, one sibling group, and the sibling tiebreak (newest first) merges the two runs by stamp:

$$\langle\, 61, 50, 41, 30, 21, 10, 1 \,\rangle$$

after the merge, the two sentences shuffled together. No traversal of that tree can do better; the information "these three form one run" is simply not in a sibling fan whose stamps interleave.

The sided datatype, and why it prevents the interleaving. The generalization does not build a second datatype. An insert names an anchor *and a side*; chains become sequences of (side, timestamp delta) entries; and the display rule generalizes from the prefix rule ("a prefix precedes its extensions") to the in-order rule ("L-extensions, then the node, then R-extensions"). One marker trick makes this ordinary lexicographic comparison again: a node's sort key is its coordinate followed by a virtual terminator, and the sided symbol alphabet is ordered

$$(R, 1) < (R, 2) < \cdots < \mathsf{marker} < \cdots < (L, 2) < (L, 1),$$

with the L band *mirrored* (among L-siblings the newer sits adjacent to the node). Marker above the R band puts a node before its R-extensions; marker below the L band puts L-extensions before the node.

On the example, the Fugue anchoring rule (the generation-policy paragraph below) sends each "insert at head" to the *successor* as an L-entry, so each run is an L-chain (Figure 12, right). Writing chains as entry sequences with deltas relative to the parent's stamp ($10 = 1 + 9$, $30 = 10 + 20$, …):

$$\begin{aligned} &\text{chain}(1) = (R,1) && \text{chain}(10) = (R,1)\,(L,9) \\ &\text{chain}(30) = (R,1)\,(L,9)\,(L,20) && \text{chain}(21) = (R,1)\,(L,20) \\ &\text{chain}(50) = (R,1)\,(L,9)\,(L,20)\,(L,20) && \text{chain}(41) = (R,1)\,(L,20)\,(L,20) \end{aligned}$$

and the marker-lex comparisons (keys descending in display order) work out as follows, writing M for the marker:

- *L-extensions precede the node*: $\text{key}(10) = (R,1)(L,9)\,M$ vs $\text{key}(1) = (R,1)\,M$ first differ at position 2, $(L,9)$ vs M, and the L band sits above the marker: so 10 displays before 1. Likewise 30 before 10, 50 before 30: each backward keystroke lands immediately left of the previous one, which is the typed intent.
- *The two runs are decided once, not per pair*: key(10) vs key(21) first differ at position 2, $(L,9)$ vs $(L,20)$, and the L band is mirrored, so $(L,20) < (L,9)$: A's run displays before B's. Now *every* node of A's chain carries the prefix $(R,1)(L,9)$ and every node of B's carries $(R,1)(L,20)$, so *every* cross-pair first-differs at that same position 2 with that same verdict. The blocks cannot interleave:

  $$\langle\, 50, 30, 10,\ \ 61, 41, 21,\ \ 1 \,\rangle.$$

  This is the datatype-level theorem `schain_subtree_convex` (a subtree is an in-order interval; §27): *any* policy that keeps a run inside one subtree gets non-interleaving from the datatype, for free.

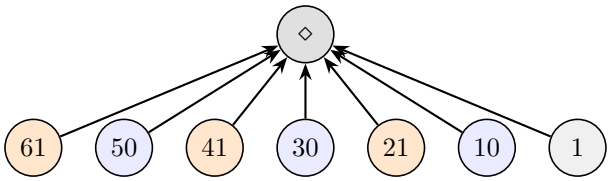
The current datatype is the all-R fragment of this order, *exactly*: with L uninhabited the in-order rule degenerates to the prefix rule, and the existing key of §18 (the sym-mapped bits plus terminator) is literally this construction with the L band empty. That is a theorem, not a slogan: the mechanization's fragment theorem (§27) proves all-R chains order exactly as one-sided chains.

How the encoding realizes it, for zero extra bits. The one-sided block of §18 spends a constant leading `1` per level whose only job is keeping coordinates nonempty; repurpose it as the side bit. The L band needs an order-*reversing* prefix-free code, and the bit complement $\overline{C}$ is exactly that at identical lengths (flipping every bit flips every first-difference verdict and preserves both lengths and prefix relations), so the block becomes

$$\text{block}(\text{side}, \delta) \;=\; \text{sideBit} \,\|\, \big(C(\delta) \text{ if } R,\ \overline{C(\delta)} \text{ if } L\big),$$

prefix-free per band, with no framing change and no new bits. On the example, with the binary delta code ($C(1) =$ `0`, $C(9) =$ `1110001`, $C(20) =$ `111100100`, so $\overline{C(9)} =$ `0001110`, $\overline{C(20)} =$ `000011011`), the stored coordinates are the concatenated blocks:

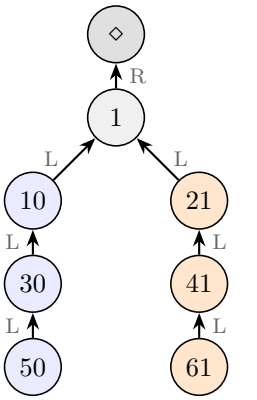
node	chain	stored coordinate (side bits marked)
1	$(R,1)$	1̲0
10	$(R,1)(L,9)$	1̲0 0̲0001110
21	$(R,1)(L,20)$	1̲0 0̲000011011
30	$(R,1)(L,9)(L,20)$	1̲0 0̲0001110 0̲000011011
50	$(R,1)(L,9)(L,20)(L,20)$	1̲0 0̲0001110 0̲000011011 0̲000011011



every edge R: one sibling group

⟨ 61 50 41 30 21 10 1 ⟩

one-sided: runs interleaved (L19)



⟨ 50 30 10 61 41 21 1 ⟩

sided, Fugue policy: blocks contiguous

Figure 12: L19 under the two anchoring rules, the battery's own trace. LCA document $\langle 1 \rangle$; replica A (blue) types backward $10, 30, 50$ before the head while replica B (orange) concurrently types backward $21, 41, 61$. One-sided (left): every insert-before anchors at the same left neighbor $\diamond$, both runs land in one R-sibling group, and the timestamp-sorted fan interleaves them. Sided under the Fugue policy (right): insert-before anchors at the *successor* as an L-entry, each run is an L-chain hanging off 1, and the in-order read keeps the blocks contiguous: L-extensions precede the node ($50, 30, 10$ before their parents' line ending at 1), and among the L-siblings $10, 21$ of node 1 the older run displays first (the L band is mirrored). Both display strips are machine outputs of `embed_sided.py`.

To compare two coordinates, parse: read a side bit; if `1`, decode one C codeword as an R-band symbol (R, δ); if `0`, decode one $\overline{C}$ codeword (decode C on the flipped bits) as an L-band symbol (L, δ); repeat, then terminate with the marker. Prefix-freedom per band delimits every block, so no length fields or framing are needed, exactly as one-sided. The complement is what makes the mirror free: within the L band, plain descending comparison of the *stored* bits is already the mirrored delta order ($\overline{C(20)} = \texttt{000011011} < \overline{C(9)} = \texttt{0001110}$ bitwise, matching $(L, 20) < (L, 9)$), so the carrier stores entropy payload only and the marker-lex of the previous paragraph is read off the bits. Faithfulness of this parse (unique decodability plus order embedding of the sided alphabet) is machine-checked by the chain-model/code-model lockstep below.

Two honest cost notes. Runs absorb the side bit like every other run-constant (paid once at the head). On *this* trace the sided coordinates are longer than the one-sided fan's (the runs' deltas are 20 because the two replicas' Lamport clocks interleave); the compression claim is about the common case, continuation-stamped backward typing ($\delta = 1$ per keystroke, an $\texttt{0}\|\overline{C(1)} = \texttt{01}$ block per level), where today's design pays a growing-δ fan. Two-sidedness buys the semantics always and the compression exactly when backward runs are locally stamped.

Side selection is a generation policy, not a datatype choice. If insert merely "takes a side," convergence and RA-linearizability hold for any sides whatsoever: those proofs never consult the order's intent. What depends on *which* side is picked is the ordering intent. Fugue's non-interleaving needs the rule "right child of the left neighbor when it has no right children, else

left child of the successor," and that rule lives at generation time, where the framework already has a slot: honesty, the same layer that polices the mint. The architecture is therefore one sided datatype, verified once, policy-free (canonicity, fold exactness, the Join, RA-linearizability), with intent theorems per generation policy: under always-R, read-equivalence to the published RGA (the fragment theorem composed with the compaction theorem); under the Fugue rule, non-interleaving (a subtree is an in-order interval). RGA and Fugue become two disciplines over one kernel rather than two datatypes.

Validation. `whiteboard/litmus/embed_sided.py`: the chain semantics and the bit carrier as lockstep twin models, both run under both policies. Every design prediction survived the first full run:

check	verdict
full gauntlet under the Fugue policy (L1–L25, credential family, subordination CEs, stale forks)	clean, **including L19** (out = $\langle 50, 30, 10, 61, 41, 21, 1\rangle$)
same gauntlet under always-R	clean except L19: the one-sided fan, reproduced
chain semantics $\sim$ bit carrier, both policies	lockstep-EXACT (unique decodability + order embedding)
all-R fragment $\sim$ the one-sided embed model	lockstep-EXACT, every scenario
randomized version-DAG PBT, all four models	120 + 300 clean

The Lean mechanization of both layers (the sided chain-lex kernel and the sided conditioned instance, capstone included) has landed kernel-clean; §27 records it.

22 Maximal non-interleaving, closed: the traversal theory and Theorem 9

The sided kernel of §21 hosts the Fugue family, and the Weidner–Kleppmann paper (TPDS'25) supplies that family's ordering specification. This section states the specification natively, presents the theory that discharges it, and records the closure: all three conditions of the paper's Definition 4 are kernel-checked for the FugueMax variant, which is, to our knowledge, the first fully mechanized maximal non-interleaving theorem for a sequence datatype. Along the way three findings recur on one theme: **the dead are load-bearing**, in stating the specification, in achieving it, and in proving it.

The specification, natively. Fix an execution. The *left origin* of an element is the element directly preceding its insertion position at the time of insertion (or a start sentinel); its *right origin* is the element directly following the left origin *in the list including tombstones* at that moment (or an end sentinel). The paper's Definition 4 (*maximal non-interleaving*) demands, over all reachable list states:

(1) *forward*: if A is the left origin of B and B displays earliest among elements with left origin A, then A and B are consecutive;

(2) *backward, with exceptions*: if B is the right origin of A and A displays latest among elements with right origin B, then A and B are consecutive, unless the exception holds: A and B

have different left origins and some C in the list state sits strictly between $\mathrm{lo}(A)$ and B without being a descendant of $\mathrm{lo}(A)$ in the left-origin tree;

(3) *arbitration*: same left and right origins order by lower identifier first.

Both origin definitions and both "earliest/latest among" quantifiers range over the list *including tombstones*: that is the first of the three places the dead carry the statement. The conditions are formalized over reachable configurations of the FugueMax kernel variant (the recorded origins are the generation policy's, the display order the kernel's), with the conclusion "consecutive" read over the *visible* document, which is the property an editor's user sees.

The traversal theory. The paper's proofs run through a tree traversal (its Lemma 7: the list order of a forward non-interleaving algorithm is a depth-first traversal of the left-origin tree; its Lemma 8 computes origins by walking the algorithm's own tree). On the embedded-chain family the algorithm's tree is implicit in the chains, and the marker theorems of §21 already state that the display order *is* the in-order rule on chains. What the discharges consume is therefore not an emission function but an *interval* form of traversal theory (`Sided_Traversal.lean`): subtree convexity (a chain prefix's extensions display as one contiguous block around it), the side lemmas (an extension displays after the node iff its first added entry is R-headed, before iff L-headed), prefix strip-and-lift, and transitivity of the chain order. A recursive emission function was designed and deliberately not built: every consumer needs only the interval lemmas, and the emission would add a fueled recursion with no downstream client; the executable emission lives in the Python twin (`traversal_check.py`), checked against the descending-key sort on both policies.

The bridge from recorded origins to chain geometry is one shape invariant, the chain form of the paper's walk-up rule. *loShape*: every minted element x satisfies

$$\mathrm{chain}(x) \;=\; \mathrm{chain}(\mathrm{lo}(x)) \cdot \langle \text{R entry} \rangle \cdot \ell s, \qquad \ell s \text{ all-L}.$$

R mints satisfy it with ℓs empty; for L mints the content is that the tombstone-visible successor of an anchor with an R-child is itself an all-L descendant of that R-child, proved by an argmax argument (an R entry inside the rest would exhibit a minted element displaying between anchor and successor, beating the successor's maximality). loShape is a *generation-discipline* fact, not a kernel fact: a forged state that stores an L entry under the wrong parent displays fine (the total order does not care) but breaks the walk-up rule, and condition (1) then genuinely fails; the honesty clause excluding it is precisely that side and parent come from the policy at mint time, never from op payload.

The forward discharge. With loShape in hand, the paper's "first node the traversal visits among A's right descendants" is replaced by a *descent*: if a minted c is an R-headed extension of A, then some minted D with $\mathrm{lo}(D) = A$ displays at or before c (strong induction on chain length, comparing the two prefix decompositions of $\mathrm{chain}(c)$). Condition (1) follows: any live c strictly between A and its earliest left-origin child B lands in A's subtree by convexity, is R-headed by the side lemma, and the descent produces a D with $\mathrm{lo}(D) = A$ displaying before B, contradicting B's minimality (`fuguemax_forward_ni`, kernel-clean; the one new invariant clause, the all-L successor shape for L mints, transported through insert, delete, and sync like the main bundle). Condition (3) was already the positive twin of the Fugue refutation (`fuguemax_same_origin_low_first`, §27). Everything so far never consults the sibling

order, and the same proof discharges the *plain Fugue* policy's forward condition as stated (`fugue_forward_ni`, `SidedRGA_Fugue_ForwardNI.lean`): the link layer was rebuilt as an external invariant over an untouched `FugueReach`, with roughly 85 percent of the FugueMax machinery transferring verbatim. Plain Fugue's conditions (2) and (3) remain false, as the paper itself proves; the machine-checked Fugue-versus-FugueMax separation of §27 stands.

The backward condition: the live-witness refutation. The Lean statement of the exception, as first adapted, required the witness C to be *live*. That reading is false, and the countermodel is the paper's own Figure-7 execution followed by one delete. In that execution (elements here named by their ids) the display is $[3, 6, 7, 4, 5]$ with $\mathrm{ro}(6) = 5$ and $\mathrm{lo}(6) = 3 \neq \mathrm{lo}(5)$; delete 4, so the visible document is $[3, 6, 7, 5]$. The premise pair $(A, B) = (6, 5)$ holds: 6 is the only minted element with right origin 5, hence trivially display-latest among them. The pair is not consecutive: 7 sits between. The exception's first half holds. But no *live* witness exists: the only elements strictly between $\mathrm{lo}(6) = 3$ and 5 are 6 and 7 (both descendants of 3 in the left-origin tree) and the *dead* 4, which is exactly the C the paper's own proof produces for this execution. The paper's Lemma 5 quantifies C over the list state including tombstones, so the corrected definition drops the one liveness conjunct and nothing else. Second load-bearing role of the dead: without them the specification itself is not a theorem of FugueMax (the refutation fired on the directed families and on 3 of 4500 randomized delete-enabled states; the earlier insert-only sweeps could never see it). Two remarks. The corrected reading is strictly paper-faithful and strictly weaker as a guarantee: a pair excused by a dead witness is a pair the visible document may interleave; that is forced, not chosen. And the *conclusion* keeps its live reading (no live element strictly between): the visible-document property is the one proved. Per this project's standing rule, the goal was corrected in the open, forced by a countermodel and matching the paper's own quantification, never silently renamed.

Replacing causal minimality. The paper's condition-(2) proof exhibits its exception witness by choosing an in-between element "not causally later than any other," then consumes that minimality three times. A reachability-invariant formulation cannot replay this: the invariant carries per-record facts about the current knowledge, not a well-founded causal order to minimize over. The replacement is geometric. Three mint-time clauses on R-mint records (a supplementary invariant bundle, transported exactly like the existing ones): a root-anchored R mint records no successor; a recorded successor displays after the anchor; and the two *RSucc* clauses pin, for an R mint whose anchor is itself an L or R mint, a stable witness (the anchor's parent, a later L-sibling, or the anchor's own recorded successor) whose key bounds the recorded successor's. With those, a deterministic, state-definable witness function replaces the causal choice: it inspects the in-between element's chain, walks at most two steps (four routes, exercised as α 253 times, β_1 22, β_0-direct 2, β_0-ladder 2 across 4500 validation states, the β_0 routes reachable only by directed construction), and returns a minted element strictly between $\mathrm{lo}(A)$ and B that the left-origin walk can never reach. The methodological point: a *universal* fact about the minter's knowledge does not transport to later states, but an *existential* witness does, because minted records, chains, and keys are stable under knowledge growth. Third load-bearing role of the dead: the constructed witness may itself be a tombstone (it is, in the Figure-7 family), and the intermediate lemmas are deliberately stated over minted, not live, elements so the ladder case can call them.

The capstone. The discharge (`SidedRGA_Backward.lean`) closes the arc: `fuguemax_maximally_noninterleaving` delivers all three Definition-4 conditions at every replica of every reachable FugueMax configuration, kernel-clean on {`propext`, `Classical.choice`, `Quot.sound`}. The reduction theorem and the forward file recompiled with zero edits after the definition correction, as the design phase predicted. SPOTs pin the witness values on all four delete-bearing directed variants, PASS and FAIL shaped.

The convergence capstone, and the TagsOK finding. Ordering intent is only half the variant's bill. The FugueMax realization writes records over a *different block format* (R entries carry the mint-time successor's key as an order-reversing tag before the complemented delta code), so RA-linearizability for the tagged datatype was owed separately and is paid (`fuguemax_ra_linearizable3`, `SidedRGA_FugueMax_RA_Lin.lean`): same canonical sorted-list state, same mergeable-queue route, fold canonicity as `f_fold_canon`. The finding worth a name: **the right-origin tag is convergence-load-bearing**, not merely intent-load-bearing. Unique decodability of the variant coordinate format (`fmCoordOf_inj`), which key injectivity and hence fold canonicity run through, is only unambiguous on *wellformed* tags, so the honesty contract carries a `TagsOK` clause alongside chain positivity; the generation layer supplies it for free, but a hand-forged tag would break convergence itself, not just the display intent. Convergence is otherwise insensitive to what the coordinates contain, exactly as the policy/datatype split predicts: coordinates are injectively minted, immutable, and consumed only through key comparisons.

22.1 Where the family sits in the published table

The Weidner–Kleppmann paper's Table 1 is the nearest published designs × anomalies matrix: roughly ten list CRDTs classified by hand against the interleaving conditions. The placement this repository contributes is not a new row but a change in how rows are occupied: **one kernel datatype, verified once, whose generation policy selects the row, each placement by theorem**.

policy / variant	row occupied	by which theorem
always-R	RGA	fragment theorem + compaction theorem: read-equal to the published RGA, anomalies *included* (L19 inherited by theorem)
Fugue policy	Fugue	forward NI as stated (`fugue_forward_ni`); *not* maximally non-interleaving: the paper's own separation, machine-checked on its Figure-7 execution
FugueMax variant	FugueMax	Theorem 9 mechanized (`fuguemax_maximally_noninterleaving`); convergence separately (`fuguemax_ra_linearizable3`)

Two readings of the table. First, the one-sided embedded-chain RGA occupies the RGA row *by theorem*: the compaction theorem (§27) makes it the published RGA's reads verbatim, so every property Table 1 records for RGA, favorable or not, transfers; the design does not get to keep the good rows and disown L19. Second, the convergence column is policy-independent (sides and tags are payload to the merge; the sided capstone quantifies over every side assignment) while the interleaving column is per-policy, which is the policy/datatype split of §21 rendered as table structure. What no policy can do is leave the family's *delete-insensitive* rows altogether: display

keys are immutable and deletion is pure removal for every policy, so a policy selects among the retention-faithful orders and nothing else. The row a splice-based design would occupy is unreachable by policy, and the next section proves that unreachability is not an accident of this kernel but an information-theoretic fact.

23 The chain price: what delete must remember

The conservation law of §17 (the sort key must retain dead ancestors' timestamps) was stated as an empirical regularity of the design sweep: every design that forgets them has a named counter-model. This section upgrades it to a characterization. The question that forced it (KC, 2026-07-18): the embed RGA works, but its proof rests on keeping a deleted node's memory alive inside live descendants' coordinates; could a *sibling-edge* implementation, which splices a dead node out and lifts its children a level up, be shown equivalent to the embed RGA with no such retention? The answer is no, by a four-event fooling pair (`Refutations/SiblingSplice_Fooling.lean`), and the counterexample names exactly the information the splice destroys.

The mechanism. In the embed order, a lifted child displays at its dead ancestor's rank: the subtree keyed by the ancestor's stamp holds its *slot*, and the child sits in that slot. Splice-and-lift erases the ancestor and re-ranks the child among its new siblings by the child's *own* stamp. Whenever a concurrent sibling's stamp lies strictly between the dead ancestor's and the child's, the two rankings disagree, and no trace of the disagreement survives in the spliced state.

The fooling pair, walked. Timestamps only; slot keys shown. Both worlds share the LCA and the concurrent branch B; only branch A differs, and its two variants land on *equal* spliced states.

	world 1	world 2
shared LCA	ins a (stamp 1) at the root	same
shared branch B	ins c (stamp 2) at the root, concurrent with all of A	same
branch A	ins b (5) *under* a; del a (6)	del a (4); ins b (5) at the root
spliced A-state	the one-node forest $[\,b(5)\,]$	$[\,b(5)\,]$: **equal**, machine-checked
b's slot key (embed)	1 (dead a's slot)	5 (its own)
owed read after merge	$[\,c,\,b\,]$ $(1 < 2)$	$[\,b,\,c\,]$ $(5 > 2)$

The knob is the stamp crossing $a < c < b$ with b and c on *different* branches, and it rides on concurrency: B saw only the LCA, so its stamp 2 is Lamport-honest, and every stamp on A exceeds everything its issuer observed ($1 < 5 < 6$; $1 < 4 < 5$). Sequentially the crossing is impossible (a later insert has a larger stamp than everything visible, the dead anchor included), which is why no sequential test can see this and why the pair had to be built on a version DAG. The impossibility is then three lines: a ternary merge function on the splice representation receives *identical* arguments in the two worlds (the LCA and branch B verbatim, the A-states by the machine-checked equality) and owes *different* answers, so there is none (`sibling_splice_no_merge_function`, quantified over every $f : \mathrm{St} \times \mathrm{St} \times \mathrm{St} \to \text{read}$; the PASS pins are the two hand-derived embed reads, the FAIL pin their inequality).

The retention characterization. The fooling pair is a *lower* bound: any state representation that forgets a lifted child's dead slot key admits the pair, so no merge over it can reproduce the embed's reads. The capstone is the matching *upper* bound: keeping the whole dead chain suffices for everything (`embed_ra_linearizable3`, the compaction theorem, the sequential spec, delete-order preservation). Between them: *the chain of dead slot keys is exactly the information content of delete*. Deletion's semantic content in this family is "the subtree keeps its slot," and the slot chain is both necessary and sufficient to honor it. What remains negotiable is only *where* the memory lives (coordinate prefixes here, carried spine paths in the validated sibling-edge design, tombstone records in the published RGA: three encodings of the same bits) and *how many bits* it costs (§18, §24); whether it lives is not.

The two-by-two. Retention crossed with display rank sorts the four named designs of this document's arc:

design	key retains the dead chain?	lifted child ranks by	verdict
embed RGA	yes: coordinate prefix	dead ancestor's slot key	capstone + seq. spec
$\mathrm{RGA}_\dagger$ (tombstoned)	yes: tombstone records	dead ancestor's slot key	read-equal to embed (compaction theorem)
rehoming (flat TF)	truncated at each rehoming	own stamp, after the climb	RA-lin; seq. spec *refuted* (Part III)
Shesha (splice)	no	own stamp, after the splice	no merge function (the fooling pair)

Reading down the rows: full retention with slot-key display gives the two read-equal designs (tombstone-free and tombstoned differ only in encoding); truncating the chain at each rehoming preserves convergence but breaks the sequential specification; discarding it altogether leaves nothing for any merge function to work with. The verdicts degrade exactly as the retained information does.

The delete-sensitivity separation. §22.1 showed the generation policy selecting rows of the interleaving table. Here is what no policy can select. For every policy over the retaining kernel, display keys are immutable and deletion is record removal, so a delete never moves survivors (`eUpdate_del_sublist` is policy-free): the whole family is *delete-insensitive*. Shesha's row is delete-*sensitive* (the splice moves survivors), and the fooling pair shows the sensitivity is not a stylistic difference but an information-theoretic one: the splice state cannot support the retention family's reads at all. Policies span the interleaving axis; nothing spans the retention axis. The two degrees of freedom are orthogonal, and only one of them is free.

No coding escapes. Could a cleverer coordinate code rescue the splice? No, in both directions. Downward: the impossibility theorem quantifies over *every* merge function on the splice states, and an encoding is a function of the state, so any coded splice inherits the fooling pair verbatim (the two worlds' encodings are equal bit for bit, being encodings of equal states). Upward: the positive theorems are parametric in the code (§27: one proof, three instantiated codes), so the entire coding freedom provably lives in what retention *costs*, never in whether it happens. Coding moves bits; it cannot manufacture information the state has discarded. That is the chain price: the storage layer of the next section spends considerable effort making the price small, and none trying to evade it.

24 The storage layer: re-coding, compaction, and the run table

The datatype's costs were settled upstairs (§18: pay the entropy of the anchor gaps at mint time) and its obligations downstairs (§23: the dead chain must be retained). This section is the layer between: what a running system does about accumulated weight. The arc has three rungs, and stating them up front keeps the bookkeeping honest. *Chains carry the semantics*: birth constants, never edited, the objects of every correctness theorem. *Runs carry the representation*: an invertible re-encoding of the current state, valid with no hypotheses at all. *Stability is needed only for forgetting*: epoch maps that genuinely discard dead history rewrite geometry, and rewriting is safe only at a settled cut (§16). Every measurement below reproduces from the cited harnesses, and each table's accounting model is stated with it.

24.1 Re-coding at a settled cut: the theorem cluster

The epoch problem. Coordinates are birth constants and never shrink: a live record whose birth chain passed through nine deleted ancestors carries all nine dead codewords as a prefix forever. Tombstone-freedom removed the dead *records*; their *codewords* survive inside every descendant, so long editing accumulates coordinate weight proportional to total history, not to the live document. A GC epoch should re-code: assign live records fresh, short coordinates inducing the same display order, and let new mints extend the fresh coordinates.

Lazy translation, and why it is the only nontrivial formulation. Replicas cannot switch codes in lockstep: a message minted against old coordinates may be in flight while the receiver has already compacted. The model is therefore *lazy translation*: the re-map is a pure function on coordinates, applied once to the state at rest and to each incoming record or op minted in the old code (the op's carried anchor prefix is translated; its delta codeword is not touched). Formally, fix a set S of *stable* coordinates, downward closed under the ancestor relation. Downward closure is not a design choice but a fact about cuts: a coordinate's proper codeword-prefixes are exactly its ancestors' coordinates, ancestors causally precede their descendants, and any causally downward-closed notion of stable therefore contains the ancestors of everything it contains. Given a compaction ρ on S, the induced re-map factors every coordinate at the cut:

$$\hat{\rho}(c) \;=\; \rho(\mathrm{stab}\ c) \,\|\, \mathrm{rest}\ c,$$

where stab c is the longest stable prefix (it is exactly one chain prefix, again by downward closure) and the unstable tail is kept verbatim. The theorems consume two hypotheses, both relativized to the coordinates *at hand* (the cut state's records plus the mints applied beyond the cut), never to all bit strings: **H2**, order preservation ($\hat{\rho}$ preserves the fold's key comparator, stated as a Boolean equation so it carries both directions), and **H3**, extension commuting ($\hat{\rho}(\pi \,\|\, \mathrm{enc}\ d) = \hat{\rho}(\pi) \,\|\, \mathrm{enc}\ d$ for mints beyond the cut, which is automatic for a genuine stable-prefix map). Injectivity is free: H2 already implies it on its domain, so the Lean bundle carries H2 and H3 and derives H1 (`StablePrefixMap`).

The cluster (`EmbedRGA_Recoding.lean`, payload- and code-generic, kernel-clean):

- **T1, fold congruence, cut-parametric** (`eRecode_applySeq`): folding translated ops over the translated cut state is the translation of the untranslated fold.
- **The global-collapse negative** (`eRecode_ext_global_collapse`): one might hope to state T1 over the whole history (map every op ever issued, re-fold from the initial state).

That statement forces H3 at every historical mint, and H3 at every mint provably collapses the re-map to a constant-prefix prepend that compacts *nothing*. Re-coding is inherently an epoch operation on states, not a history rewrite; the lazy, cut-parametric formulation is not a convenience but the only nontrivial one. A finding, mechanized.

- **T2, reads identical** (`eRecode_reads_identical`): the element sequence of the translated fold equals the untranslated one, verbatim; compression is invisible to every observer. The contentful half hides in T1: the translated side re-sorts by the *new* keys, so equality of reads is exactly the statement that the new keys induce the old order, H2 doing its job. The negative control earns its keep here: an injective, H3-satisfying but order-breaking map genuinely flips the read, pinning H2 as the load-bearing hypothesis.
- **T3, transport** (`eRecode_fold_canon`, `eRecode_sorted`, `eRecode_version_sorted`, `eRecode_ra_transport`): canonicity, sortedness, and the capstone's per-version witness statement all transport along the isomorphism. Honest scope note: the transport is at the level of states and witnesses, not configurations; the re-coded history is *not* itself an honest configuration of the same datatype (compacted coordinates deliberately forget dead chain segments, so the chain-generation honesty clause fails for them). Re-basing honesty modulo an order-embedding, so the capstone can be re-invoked natively in the new epoch, is the deferred protocol half, together with agreement on a common cut; the cut hypothesis itself is discharged by §16's SettledAt gate (`eRecode_settled_bridge`).

Measurement 1: the microbenchmark. Directed, hand-computed worked example (unary code; history: insert a at the root, b under a, c under b, delete a and b; then, beyond the cut, insert d under c and e at the root). Accounting: the numbers are *coordinate bit lengths of live records*, order metadata alone, identical convention to the entropy tables of §18; characters are excluded.

live record	original coordinate	re-coded
c (two dead codewords carried)	6 bits	2 bits
d (minted beyond the cut, under c)	10 bits	6 bits
e (minted beyond the cut, at the root)	8 bits	8 bits
total	24 bits	16 bits

Reads are identical on both sides (hand-checked and pinned as SPOTs); the dead-chain share of c and d's bits is gone entirely, and e, which never passed through the dead region, is untouched. The same harness's delete-heavy directed case (a 200-deep chain, binary code) drops live coordinate weight from 200 bits to 1 bit, the **200**× figure quoted in the arc's motivation; 500 randomized histories (random cut, two replicas forked beyond the cut with concurrent ops and a merge at the end) pass reads-identical and the T1 record-for-record correspondence at every step, and the order-breaking negative control flips the read exactly as predicted.

24.2 The verified concrete compaction, and what real traces say

compactEliasDelta. The theorem cluster is parametric in the re-map; the concrete v1 compaction instantiates it (`EmbedRGA_CompactEliasDelta.lean`): *(a) drop dead ranges* (a stable chain is carried forward iff a live descendant or a declared in-flight coordinate passes through it;

everything else is dropped and its sibling rank reclaimed) and *(b) rank-renumber* (in each sibling group the surviving deltas are renumbered to $1..k$ order-isomorphically and re-encoded with the same ordered prefix code). Its H2 discharge runs the per-group order isomorphism through the chain-lex theorem, with renumbered ranks dominated by Lamport-fresh future deltas; the headline (`compactEliasDelta_settled_reads`) discharges everything at a settled cut of an honest configuration.

The in-flight wrinkle, pinned both ways. Renumbering a sibling group is only an order isomorphism against the deltas it can see. If an op minted in the old code is still in flight with a delta in that group, dense renumbering can slide a surviving sibling past it: the FAIL SPOT pins exactly this (an in-flight root insert; the unguarded renumbering sends a surviving delta $9 \mapsto 2$, and the delivered op lands on the wrong side: the pinned read flips). The v1 resolution is the guard `skipAt`: sibling groups with a known in-flight child delta are skipped this epoch, and the PASS SPOT pins that the guarded construction skips exactly that group, preserves the read, and still compresses. Both pins are hand-derived, PASS and FAIL shaped.

AnchorsFactorBeyond: the residue is a theorem. The bridge from SettledAt to the cluster left one named residue: every op minted with the cut in its causal past must carry an anchor prefix that factors at the map. This is *not* dischargeable from bare honest reachability (countermodel: the chain-generation clause constrains the minted coordinate, not the carried split, so a dishonest split satisfies it while defeating the factorization). It *is* a theorem of the generation discipline: under the framework's honest generation (the same hypotheses the capstone's honesty discharge consumes), every insert's carried prefix is the stored coordinate of an actual anchor record in the fold of its causal past, so the factorization sits on a codeword boundary by construction (`anchorsFactorBeyond_of_anchored`), and the bridge is restated residue-free (`eRecode_settled_bridge_honest`). Scope, honestly delimited: the single-epoch statement, with cut-side data addressed by coordinates and chains, never by event ids resolved through prefix sums (the runtime twin's erratum: renumbered coordinates no longer telescope to ids, so id-addressed cuts break after epoch one). Concurrent divergent compactions (an epoch DAG with common-refinement translation) remain the deferred protocol half.

Spine fusion, mechanized. Rank renumbering alone leaves the *spine depth*: typing runs mint δ=1 chains, and every level costs at least one codeword even at ordinal 1 (Measurement 2 quantifies this). Fusion removes the dead levels themselves. A *fusible spine* is a maximal chain of settled dead nodes, each with exactly one child branch (counting every known coordinate, in-flight prefixes included) and no in-flight op anchored on it; it collapses to a single level at the spine head's codeword, so a typing-run-then-delete chain of depth k costs one codeword instead of k. Composed after the rank pass this is the full iteration-two map, and reads survive it (`compactThenFuse_reads`, `EmbedRGA_Fusion.lean`, kernel-clean). Order preservation is a three-class H2 argument: (1) within a fused block the common prefix is replaced wholesale, relative order untouched; (2) a block against the spine head's siblings is decided at the head's level, whose codeword fusion keeps; (3) the anchor-versus-extension corner is vacuous, since dead spine nodes hold no records and no key ends at a fused-away level. The mechanization sharpens class 2 into triviality: because fusion runs *after* the rank pass, the head codeword it sees is already the ordinal, so the fusion map keeps the head verbatim and drops only the strict interior, and the class-2 comparison is unchanged by construction rather than by a separate

"block inherits the rank" argument. The guard is conservative (any known in-flight branch or anchor blocks its node from every spine); through-traffic below a blocked node still fuses.

The merge clause, discharged (VC-S4 for the remap species). The stability bundle's argumentwise merge congruence (§16, VC-S4) is, for the coordinate re-map, the statement that lazy translation commutes with the ternary merge: merging three re-mapped states equals the re-map of the merge (`merge_remap_congr`, `EmbedRGA_MergeCongr.lean`). The "at hand" domain hypothesis is needed only on the two *merged branches*; the LCA argument enters the live-set rule solely through its id set, so no hypothesis on l is required. This is precisely what lets a compacted head merge against a full LCA payload, the mixed case §16 calls the normal one.

Multi-epoch composition. A single re-coded configuration is not a native honest configuration (T3's gap), so a *second* compaction cannot naively re-invoke the cluster. The data-plane half is now closed (`EmbedRGA_MultiEpoch.lean`, kernel-clean). The semantic residue of one epoch is a StablePrefixMap; two compose into one (`StablePrefixMap.comp`, axiom-free: H2 composes because order-isomorphisms do; H3 composes at the boundary, since an epoch-1 mint (π, d) once remapped is an epoch-2 mint $(F_1\pi, d)$ with the delta codeword untouched; H1 is derived), and the n-fold form gives the headline — after arbitrarily many settled-cut compactions, any beyond-all-cuts continuation, its lagging ops translated through every epoch map, reads exactly as the uncompacted run (`multiEpoch_settled_reads`). The subtlety that makes this non-trivial is *rank reclaim*: naive composition on the first epoch's domain is unsound, because if epoch 2 reclaims the rank of a record that died between the epochs, the dead coordinate and the reclaimed live coordinate collide under $F_2 \circ F_1$ (pinned as `naive_composition_collides`). The fix carries the composite's own surviving domain, coordinate-addressed; the companion erratum `id_addressing_breaks` records why id-addressed cuts fail after epoch one. Freshness across epochs is representation-agnostic (`rED_le_self`, `rED_fresh_dominates`: a delta fresh against original sibling deltas stays fresh against renumbered ordinals without inspecting their meaning), so the order-iso half composes whether stored deltas are timestamp differences or epoch-k ordinals.

The one remaining compaction obligation, named. Two residues above — the single-epoch honesty re-basing (T3's gap: the re-coded history is honest only *modulo* the coordinate order-embedding) and the multi-epoch CompatChain residue — are the *same* obligation. Both its causal-stability half (`nested_cuts_settled`, from settledAt_of_allHeard and cut-antitonicity of the frontier, `EmbedRGA_CompatChain.lean`) and its order-iso freshness half (`rED_hocc_fresh`) are closed. What is left is one lemma, which we call $(*)$: exhibit an epoch-2 configuration honest *modulo* the order-embedding F_1, so that anchored-existence re-derives at the epoch boundary. Discharging $(*)$ closes both halves at once; it is the whole remaining *single-replica* compaction protocol, and everything downstream (agreement on a common cut, concurrent divergent compaction across replicas) is the multi-replica layer proper (Open Questions 40.7, 40.8).

Measurement 2: real traces, and where the bits actually sit. The full epoch stack (rank renumbering plus *spine fusion*: collapse each maximal settled dead chain with exactly one child branch and no in-flight anchor into a single level) was run over the standard collaborative-editing replay corpus (the josephg traces). Accounting: per record, the sum over its kept-chain

levels of $|D(\delta)|$ bits, D the flipped Elias-δ code of §18; totals divided by live characters; character payloads excluded; "fused" is after the renumber-plus-fusion epoch at a settled cut.

trace (one-sided)	chain b/ch, raw	chain b/ch, fused	ratio
friendsforever	1622.5	856.5	1.9×
clownschool	2488.7	1471.0	1.7×
seph-blog1	2974.9	917.2	3.2×
automerge-paper	2304.5	1279.4	1.8×

Three findings, in causal order. *(i)* Rank renumbering alone recovers only 1.1–1.8×: typing runs mint δ=1 chains, and every level costs at least one bit even at ordinal 1, so the residual cost is *spine depth*, which is what fusion owns. *(ii)* Fusion then removes essentially everything removable: surviving dead levels cost 3 to 76 bits per character in total, and the fused column equals the code cost of the *live tree shape* alone, verified by a per-level attribution that accounts for every bit. *(iii)* The few-bits-per-character hope fails for a reason worth the number: the live shape is itself expensive. Real traces anchor typing runs in *live* chains of mean depth 835 to 1468 levels at about one bit per level, and 97 to 99.8 percent of the post-fusion bits sit on live ancestor levels that no epoch map may touch (they are the retained semantics, §23). History independence holds three ways (staggered cuts, one final cut, and from-scratch compaction land on bit-identical totals), and the fused display spells the trace's final content on every sequential trace. Conclusion: the epoch stack removes the dead-history share; what survives is a *representation* cost, which is the run table's brief (§24.3).

Measurement 3: the sided instantiation. The same stack, band-aware (renumber within each parent-band group; fusion carries the head's band and codeword), replayed under the Fugue policy. Same accounting, plus one side bit per level, which is precisely the point:

trace (sided, Fugue)	chain b/ch, raw	chain b/ch, fused	ratio
friendsforever	2582.0	1735.7	1.5×
clownschool	4058.5	2988.1	1.4×
seph-blog1	5559.1	1919.8	2.9×
automerge-paper	4448.0	2583.7	1.7×

The ratios sit *systematically below* the one-sided 1.7–3.2×, and the naive expectation that the ratio would be preserved is falsified by its own arithmetic: adding the same flat per-level side cost to numerator and denominator pulls the quotient toward the *level-count* ratio, which is below the payload ratio. Where the sides pay: the fused sided state costs about twice the fused one-sided state, and the premium is *entirely the flat side flag*; the sided delta payload exceeds the one-sided payload by only 1.0 to 4.7 percent (on these traces the Fugue tree is payload-equivalent to RGA's). The flag costs a flat 1.000 bit per level against a true side entropy of 0.19 to 0.27 bits (L-mints are 3.0 to 4.6 percent of mints under Fugue on real traces): a **4–5× overpay** on the side channel, load-bearing for lexicographic band separation, and 94 to 98 percent of the side bits sit on live levels no epoch map can touch. Both residues (live depth, flat flags) point at the same lever: a different representation of the same immutable chain data.

24.3 The run table: the representation lever

Runs. Work over the *kept tree*: the live records plus their dead ancestors (every other node has been removed outright; tombstone-freedom is unchanged). Call the edge from node p to child c *fusible* when (1) c is the unique kept child of p, (2) $\delta(c) = 1$, (3) c's side is R (vacuous one-sided), and (4) c and p have the same liveness. A *run* is a maximal chain of fusible edges. Fusibility is a per-edge predicate and condition (1) gives each node at most one fusible out-edge, so the runs are a canonical partition of the kept tree into chains: a function of the state, independent of arrival order. A table entry is one run, with header (liveness flag, parent entry, offset in parent, side, head delta, length); members after the head implicitly have delta 1 and side R, and liveness is uniform along a run. A record's address is *(run id, offset)*. Dead entries carry no records, only the structure live descendants' coordinates pass through. The design's starting observation: what typing mints *is* exactly a fusible chain, so real documents decompose into few long runs.

Two properties before any numbers. *No stability gate*: the run table is an invertible re-representation of the current state, whatever that state is, unsettled suffixes included, and in particular the freshly typed hot end of the document where the depth cost is being minted; the PBT exploits this by rebuilding the table mid-flight between merges after every event. *Lossless in the strong sense*: from the table alone one recovers every kept node's delta (head delta from the header, 1 elsewhere), side, liveness, and pre-epoch timestamp (deltas summed along the contracted path), hence every coordinate, hence the state; the harness gates exactly this, with zero tolerance, on every record of every trace.

The tail-attachment lemma. *In the canonical table, every entry attaches at its parent entry's last member.* (If an entry attached at an interior member m, then m's run successor and the entry's head would be two kept children of m, contradicting fusibility of m's outgoing run edge.) Three consequences: the "offset in parent" header field is derivable (always parent length minus one; the accounting still counts it and reports it recoverable); the cross-run comparator only ever diverges at a tail or inside a shared run, so it consults entry chains and header fields only and *never materializes a coordinate*; and it is cheap to assert, which the harness does on every table built.

The comparator. Within a run, display order is offset order (each member is the R delta-1 child of its predecessor, hence displays directly after it). Across runs, compare the two entry chains from the root; by the lemma every step exits at a tail, and at the first difference either one chain is a prefix of the other (the deeper record is inside the shallower's continuation subtree; one-sided the shallower displays first, sided an L branch displays first) or two branch entries hang at the same tail and the branch headers decide (one-sided newest first; sided by the banded order). The $(\text{ts}, \text{agent})$ concurrency tiebreak is *not* encoded in either representation's counted bits; the tie oracle rides outside the representation on both sides and is charged to neither, preserving parity with the chain measurements. The cost of a comparison is the *contracted* depth: one entry per run on the path instead of one level per node.

Mutation, and the forced coalesce. Splits mutate the table, never the semantics: stored chain data is immutable, only the partition changes. Insert at anchor a: if a is interior, split its entry after a; attach the new record as a singleton; then *coalesce* (a fusible singleton that is the tail's unique attachment fuses into the parent entry), which is how typing extends a run in $O(1)$ table work. Delete of x: split to make x a tail; if x still has kept children, carve it

out as a dead singleton (a liveness split); childless, it simply vanishes, dropping newly unkept dead tails upward and re-coalescing the parent with a now-unique fusible child. Coalesce is the inverse of split *and it is not optional*: without it, deleting a concurrent interloper leaves the two halves of the old run split forever and the table drifts from the canonical partition (found during design, pinned by a directed case and by the PBT's incremental-equals-canonical-rebuild gate after every event). Delivery under a vanished anchor (the anchor chain passed through nodes deleted while childless here) re-materializes the missing levels as dead entries from the op's carried coordinate: the run-table cost of tombstone-freedom, paid with information the embed op already carries. Merges deliver the other side's events through the same two rules; epochs rebuild the table over the compacted tree (sound because the table is a function of the state).

The accounting model. Applied identically to both representations; every pointer and every header bit counted, no hidden structure. D is the flipped Elias-δ code; $W = \lceil \log_2(N_{\text{entries}} + 1) \rceil$ is the id width.

$$\begin{aligned} \text{per record:} &\quad W + |D(\text{offset} + 1)| \\ \text{per entry:} &\quad 1 + W + |D(\text{poff} + 1)| + |D(\text{head } \delta)| + |D(\text{len})| \quad (+ \text{ 1 side bit, sided}). \end{aligned}$$

Totals are per live character, reported with the full per-field breakdown so every number is auditable back to a field. Not charged on either side: the tie oracle (above). Two fields are counted although the canonical table makes them recoverable, and the breakdown lets a reader subtract them: the parent offset (the tail-attachment lemma) and the per-record run id when records are stored grouped by run (it is then positional).

Measurement 4: the run table. Same traces and replay as measurements 2 and 3. Columns: the raw chain representation ("chain"), the chain after its best epoch ("fused," the previous subsection's current best), the run table over the *raw uncompacted* chains with no epoch and no settled cut ("rt"), and the run table rebuilt after the epoch ("rtc").

trace	family	chain	fused	rt	rtc	fused/rtc
friendsforever_flat	1-sided	1622.5	856.5	22.13	22.30	38.4×
friendsforever_flat	sided/F	2582.0	1735.7	22.54	20.98	82.7×
clownschool_flat	1-sided	2488.7	1471.0	22.42	22.64	65.0×
clownschool_flat	sided/F	4058.5	2988.1	24.11	21.20	140.9×
seph-blog1	1-sided	2974.9	917.2	26.00	23.16	39.6×
seph-blog1	sided/F	5559.1	1919.8	27.21	24.99	76.8×
automerge-paper	1-sided	2304.5	1279.4	23.76	23.92	53.5×
automerge-paper	sided/F	4448.0	2583.7	25.21	24.58	105.1×
friendsforever	1-sided	1243.2	856.5	20.92	22.30	38.4×
friendsforever	sided/F	2189.2	1735.6	22.36	20.98	82.7×
clownschool	1-sided	1863.1	1471.0	20.85	22.64	65.0×
clownschool	sided/F	3421.3	2988.1	22.29	21.20	140.9×

All values in bits per live character under the accounting model above; the `_flat` traces are the concurrent traces replayed single-author, their unsuffixed twins the concurrent originals. The verdict: **20.9 to 27.2 bits per character, 38 to 141 times below the fused chains**, confirming the design prediction (per-record cost collapses from $\Theta(\text{depth})$ to $O(1)$ amortized plus a log-sized offset) with margin. Where the remaining bits sit, from the audited breakdown:

- The per-record *run id* now dominates (10 to 14 of the 21 to 27 bits, the flat id width): the deep-chain cost did not shrink to the offset code, it shrank to *one pointer*. An implementation storing records grouped by run makes this field positional and free, putting totals at 9 to 14 bits per character; the model charges it because the pre-registered accounting does.
- Offsets cost 5.2 to 12.1 bits per character; all header fields together amortize to 0.6 to 6.4.
- The side channel collapses from one flat bit per live level (867.8 to 1494.1 bits per character after fusion) to one bit per *entry*: **0.061 to 0.174 bits per character**, three to four orders of magnitude, and below even an entropy-coded per-level flag (H(side) times 850 to 1500 levels); the explicit L-entry-list alternative is computable from the reported entry counts and is dearer at these sizes.
- The epoch stack becomes *bit-wise marginal* under this representation: the composed column beats raw by at most 2.9 bits per character (sided) and loses to raw by up to 1.8 (one-sided, where longer post-renumber runs make offsets dearer than the headers they save). The run table alone, with no settled cut at all, already delivers the collapse. What the epoch still buys is *structure*, not bits: entries drop up to 7×, the id width shrinks, raw cursor-jump boundaries vanish under renumbering, and the contracted depth drops up to 11.8×.
- The honest limit: the *bit* cost per record is $O(1)$ amortized plus the offset code, but the comparator's *time* is still Θ(contracted depth) per cross-run comparison (mean 19.6 to 261 runs per path against chain depths of 850 to 1500). Collapsing comparison time further (an order index over the contracted tree) is a separate, orthogonal layer; nothing here needs it for the metadata claim.

Gates, all passing: display identity on all six traces, both families, raw and composed (walk order equals the chain display; text equals the final content); about 198k sampled pairwise comparator verdicts matching display positions with antisymmetry checked; losslessness exact on every record; the tail-attachment invariant on every entry of every table; and a 150-trial PBT (8,186 events, three replicas, concurrent edits and merges) gating walk identity, the all-pairs comparator, and incremental-equals-canonical after every event, while exercising 872 merges, 315 mid-run splits, 438 liveness splits, 551 coalesces, and 26 vanished-anchor materializations.

The Lean mechanization of this subsection, owed in the previous revision, is now in place (`RunTable.lean`, state-level and hypothesis-free except where noted). The load-bearing kernel is the tail-attachment lemma (`tail_attachment`). The representation isomorphism is a genuine *bijection*: tableOf and stateOf are mutual inverses on canonical tables (`canonical_tableOf`, `stateOf_tableOf`, `tableOf_stateOf`), stated over labels, sides, and liveness — *not* timestamps, the exact correction the implementation forced (rank ordinals forget time, so a timestamp-preserving iso is false after the first epoch). The comparator equals the fold's key order (`cmpTable_eq_keyLt`), the structural walk equals the display (`walk_display`, via `walkE_sound`/`walkE_complete`), and the mutation rules are machine-checked perturbation lemmas: mid-run split, liveness split, vanished-anchor materialization, and the forced coalesce (the delete-inverse a paper proof would most plausibly hand-wave; its necessity is pinned by `spot_d4_coalesce_necessity` and canonicalization under mutation by `tableOf_congr`). Only the epoch-composition clause (`runTable_epoch`) carries a stability hypothesis. The sided run table is under way: its structural core is in place (`SidedRunTable.lean`: `stail_attachment`, an entry type with an explicit side field, and the band-order comparator

seed `sided_keyLt_iff_schainBefore`); the full sided repr iso and walk mirror the one-sided development and are the remaining owed piece (Open Question 40.14).

24.4 From projection to shipped bytes, and the cross-system question

Two external facts close the representation arc; both are measured, and both are laid out with their full methodology in the performance section (§38), so only the headline belongs here. First, the run-table projection of §24.3 is no longer merely measured-in-model: task #104 ships a lossless run-table serializer (`runtime/src/serialize.js`), whose `accountingBits` routine reproduces the projection's bit totals *bit-for-bit* on every trace, while the shipped bytes land *below* the projection (the model charges the recoverable positional fields — per-record run id and offset, and the tail-attachment parent offset — that a real encoder drops). Over a settled-cut compacted state the serializer reaches 1.1 to 1.3 bytes per character on the josephg traces, at production save size and an order of magnitude below the packed absolute-chain estimate. Second, placed against production CRDT libraries this closes the save-size gap to rough parity with the smallest state-only production saves, and it confirms the design's one categorical storage win: tombstone-freedom means the save *shrinks* on delete where the history-carrying systems grow. The full cross-system matrix — per-character apply latency, merge and sync, load, all four save representations, sync payload, and the grows-on-delete reproduction — is developed in §38.

25 Comparison with Eg-walker

Eg-walker (Gentle & Kleppmann, EuroSys 2025) is the closest system in spirit, and the two designs can be read as the *operational* and *denotational* realizations of one thesis, **pay for concurrency, not for history**:

	Eg-walker	embedded-chain RGA
model	op-based; replicas exchange events	MRDT: three-way merge with an LCA
durable state	**full event graph on disk**, deletes included, retained while concurrent ops may arrive; document state metadata-free	**live nodes only**; no event log anywhere; dead survive as $\Theta(\log \delta)$ bits inside live coordinates
merge mechanism	*replay*: walk the event graph, rebuild a transient CRDT structure (tombstones reappear transiently), discard at causally-linear points	*refold*: O(survivors) coordinate recomputation; no replay, no transient structure, no order decisions
sequential-editing cost	$\approx$ zero (plain edits; no metadata)	$\approx$ zero order metadata ($\sim$4 bits/level for continuation typing; a cursor jump pays log-gap once, at the run head)
concurrency cost	replay time proportional to the concurrent region, at every affected merge	coordinate bits proportional to log(ts gap), once, in space
ordering semantics	Yjs-variant (FugueMax-adjacent); maximal non-interleaving *conjectured*, not proved; two-sided (no L19 analogue)	$\equiv$ published RGA (machine lockstep); one-sided: L19 backward-run interleaving inherited; removed by the sided generalization (§21), RGA kept as the all-R fragment
correctness artifact	informal proof (Appendix C) + property testing	L1–L25 battery + DAG PBT + lockstep vs RGA$_\dagger$; **RA-linearizability kernel-checked in Lean** (§27); the design was chosen so the merge proves itself (no order decisions to verify)
op payload	integer origin positions (compact, human-meaningful)	two timestamps + ghost prefix (droppable on the wire; §19)

Four sentences of honest positioning. *(i)* Eg-walker achieves its clean document state by keeping the entire history and re-deriving order on demand; the embedded-chain RGA achieves it by making order derivation unnecessary: the sort keys are written at birth and never edited, and the coordinates are their encoding (§18), precomputed at mint time and exact forever. *(ii)* Eg-walker's ordering target was stronger where L19 is concerned (Fugue-style two-sidedness removes backward interleaving, while the one-sided design inherits RGA's anomaly by construction, being read-equal to RGA); the sided generalization of §21 adopts exactly that two-sidedness as a parameter, closing the gap while keeping the one-sided datatype as its all-R fragment; the FugueMax variant's maximal non-interleaving, which Eg-walker's shipped ordering only conjectures, is here a kernel-checked theorem (§22). *(iii)* The verification gap runs the other way: Eg-walker's replay-transform-clear loop is algorithmically sophisticated and only informally proved; here the merge performs no arbitration at all, the metatheory is one-line at the chain layer plus a single faithfulness theorem at the encoding layer, and RA-linearizability is a kernel-checked theorem in a framework that had already verified a dozen-plus replicated datatypes end-to-end (§27). *(iv)* The cost models part company at cursor jumps: a single-user history that hops back to type at

an old anchor is free for Eg-walker (nothing concurrent to replay) but mints log-gap bits here, once per jump, at the run head; on measured single-user editing traces the jump tail is thin (code bits $\approx$ 1.4–1.8 per level; see the companion note of §18). In the MRDT setting there is also a model fit: Eg-walker must reconstruct merge context by replay because the op-based model gives it none; the MRDT model hands the merge its LCA, which is exactly the context replay rebuilds.

26 Related work

The verified sweep of nearby systems lives in `whiteboard/litmus/related-work.md` (per-claim verdicts, primary sources); this section positions the datatype.

Retention taxonomy. What nearby systems keep for the dead: tombstone records (RGA with cemetery under causal stability; WOOT; Yjs's per-deletion GC structs, kept forever; Fugue, "we cannot remove a deleted element's node entirely"; Peritext, "the positions must be remembered"); the full history (Eg-walker's event graph, Chronofold/causal trees where the log *is* the text); consensus-gated compaction (Treedoc's flatten requires Paxos-commit); or **dead identifiers inside live position keys** (Logoot/LSEQ, Weidner's position-strings/list-positions, the Fugue id space). The embedded-chain RGA is in the last class *by information content* (dead ancestors' timestamps persist inside survivors' sort keys, at live-reachable scope), but with three deltas to its classmates: the retention is *delta-coded at the entropy of the anchor gaps* (a sequential history costs a constant per element where Logoot-family keys grow with allocation policy), the *birth tree kept whole* gives forward non-interleaving and delete-order preservation (position-key designs interleave, the PaPoC'19 anomaly, and Figma-style fractional indexing is unsound under concurrency by its own account), and the semantics is pinned to the best-studied sequence CRDT by a machine-checked lockstep rather than being a new point in anomaly space.

The impossibility backdrop. Attiya et al. (PODC 2016) prove $\Omega(D)$ metadata for push-based protocols meeting even the weak list spec: the published form of "ordering requires remembering the dead." This design does not escape the bound; it *relocates and minimizes* the memory: $\Theta(\log \delta)$ bits per dead-chain element inside live coordinates (plus the MRDT version store, which the model provides). The sharper, machine-witnessed form from our design sweep is the *conservation law* (§17): the sort key must retain dead ancestors' timestamps (the verdict information survives every encoding change: stamp, spine, tombstone, or denominator bits), and ties force separate *metadata* only under timestamp-oblivious minting; the timestamp-faithful mint carries the same bits inside the coordinate. Eight timestamp-oblivious mutation variants each have a named machine-checked countermodel (fold-verdict erasure; repair non-locality); the credential countermodel separates this design from all of them.

Verified datatypes. Isabelle RGA (Gomes et al. OOPSLA'17, op-based convergence), OpSets (list semantics specified by a total order of identified ops: arbitration-by-identity as specification), VeriFx (SMT, convergence properties), MET (TLA+, list CRDT bugs). In the MRDT line (Kaki et al. OOPSLA'19; Peepul PLDI'22; RA-lin OOPSLA'25, this framework), no verified sequence with sequence-CRDT-grade ordering guarantees existed; our verified rehoming RGA (§17) is RA-linearizable against its own fold yet fails the sequential delete-order spec, since RA-linearizability certifies convergence to the datatype's *own* semantics and ordering intent must be specified and

proved separately. The embedded-chain RGA delivers the missing artifact: RA-linearizability is a kernel-checked Lean theorem (§27); delete-order preservation, display stability and forward non-interleaving are proved at the model layer; and the RGA read-equivalence (the compaction theorem, §27) closes the intent question against the best-studied sequence semantics.

Treedoc's lesson, inverted. Treedoc re-balances geometry and must pay consensus for it; the principle that emerged from our ledger-based sibling design study (*reprojection without consensus is safe iff geometry has been demoted to a rendering*) is here taken to its fixed point: geometry is never reprojected at all. It is immutable, so it may safely be the only thing.

27 Mechanization: the conditioned discharge and the compaction theorem

The verification framework is Part I's metatheory; file paths below are relative to the Sal repository.

The capstone is proved (2026-07-15, kernel-clean on {propext, Classical.choice, Quot.sound}):

$$\texttt{embed\_ra\_linearizable3} \;:\; \mathrm{EReach}\ \Gamma\ C \to \mathrm{IsRALinearizable3}\ C$$

i.e. RA-linearizability per version at every honestly reachable configuration, *parametric in the code* Γ: the unary code, the binary delta code, and the flipped Elias-δ code are all proved `OrderedPrefixCode` instances, three encodings on one proof. The factoring is exactly this part's design–encoding split, which is itself evidence for the split:

1. **Model layer** (`Sal/MRDTs/RGA_Embed/`, six files, 0 sorry, plus the read-equivalence order core and the sided kernel noted below): the code-parametric kernel; commutations *to state equality* under `rc` = `Either` (`rc_non_comm'`); `ins_prefix_ghost`; coherence and chain-generation closures; display stability, with step stability (S2, delete-order preservation at its `del` instance) proved *without any axioms* and merge pairwise stability (S4, the adopted contract); unique decodability (`coordOf_inj`); **the chain-lex theorem** (`display_iff_chainBefore` = Corollary 18.2); non-interleaving as subtree convexity (`subtree_convex`); Theorem 18.1(i) for all three codes (`binEnc_mono`/`binEnc_prefixFree`, `dEnc_mono`/`dEnc_prefixFree`, cross-validated against the Python codes by kernel `decide`); `native_decide` SPOTs where the rehoming RGA's delete-reorder witness gets the opposite verdict.

2. **Conditioned instance** (`Sal/ConditionedMRDTs/MRDT_Instances/EmbedRGA/`, nine sections, 0 sorry, plus the Elias-δ inheritance corollary `EmbedRGA_EliasDelta.lean`), proved via the *mergeable-queue route* of §13: a generic join lemma (`JoinLemma3At`: the merge exhibits its own linearization witness) plus an honest-reachability metatheorem (`HonestReach`), rather than the rehoming RGA's 55-file bespoke chain. The route applies exactly when states are canonical per event set, which is Theorem 17.1(i). Landed: the canonical sorted-list state; strictly-sorted extensionality (`esorted_ext`: sorted lists with the same members are equal, replacing any canonical-list formula); **fold-canonicity** (`e_fold_canon`: well-formed enumerations of one event set fold to the *same* state; Theorem 17.1(i) mechanized); the honesty core and its bridge to well-formedness; the merge

membership characterization (`e_mergeL_mem`, OR-set survival with honesty closing the cross-branch-delete corner); the Join (`e_join_at`: the merge is its own linearization witness); the capstone; the honesty discharge from the framework's generation discipline (`eHonest_of_applicable`: applicable generation forces the honesty core, so per-op honesty needs no bespoke oracle); and the instance intent theorems: delete-order preservation *is* sublist stability (`eUpdate_del_sublist` is literally `List.filter_sublist`: a delete's successor state is a sublist of its predecessor, so the surviving display order is untouched; the merge preserves each input's survivor order on both sides).

The compaction theorem is proved (2026-07-15, kernel-clean): read ∘ embed = read ∘ $\mathrm{RGA}_\dagger$ on honest executions, as `rga_read_eq_embed_read` (`EmbedRGA_ReadEquiv.lean`): for every well-formed enumeration of an honest event set, *any* enumeration of the tombstoned fold's visible ids in the published RGA's own relational order (`visible_lt`) equals the embed fold's id sequence, with element agreement (`embed_rec_readSeq`). The proof is *against the published definition*: the order core (`Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean`) shows `visible_lt` coincides with chain-lex on the shared birth tree (soundness by induction on derivations, completeness by realizing every chain prefix as an `afters`-reachable ancestor), and en route proves the published relational order total and asymmetric on the birth tree, facts the published side never had. This converts the design into a theorem *about* the published RGA: it admits a strictly-live compaction preserving every read. The lockstep rows of §20 are this theorem's tested form.

The sequential specification is proved, and the design holds the canonical seat (2026-07-16). The campaign of Part III proved `embed_seq_sound`: on every sequentially honest single-replica history the canonical sorted state is the naive insert-at-position text buffer, record for record, with the adjacency lemma (`chainBefore_snoc_iff`: a fresh insert's delta beats every existing child's by sum-telescoping) as the geometric heart (§32). The published $\mathrm{RGA}_\dagger$ inherits the same buffer spec through the compaction theorem (`rga_seq_read_eq_buffer`), and the same statement is refuted for the rehoming design (§33), which demoted it repo-wide: the embedded-chain family is the repository's canonical sequence RDT, and the canonical fused Peritext was re-based onto this kernel (`Peritext_Embed/`: capstone by pure instantiation of the payload-generic instance; `renderIds_del`, deleting a character never re-formats another, the theorem the rehoming kernel provably lacks; and the tier-4 editor theorem `peritextEmbed_seq_sound`). One caveat is recorded as Open Question 40.12: per-replica soundness and convergence do not compose into "the merged read is the naive buffer on some ordering of the union"; the dead-anchor corner (§35) is where stamp-sorted replay diverges from the fold.

The sided generalization is proved (2026-07-15, kernel-clean), in the same two layers as §21 presents it. The sided chain-lex kernel (`Sal/MRDTs/RGA_Embed/Sided_ChainLex.lean`): the sided marker theorem `sdisplay_iff_schainBefore` (display comparison of sided coordinates IS the in-order rule), totality of the display relation *without any axioms*, unique decodability (`sidedCoordOf_inj`: the side is recoverable from the first symbol's band, the delta by prefix-freedom within the band), the all-R *fragment theorem* `schainBefore_liftR` (all-R chains order exactly as one-sided chains: "RGA is the all-R fragment," mechanized), and in-order-interval subtree convexity (`schain_subtree_convex`, the datatype half of the L19 discharge). The conditioned instance (`MRDT_Instances/SidedRGA/`, nine sections mirroring the one-sided instance with the sided key): fold-canonicity (`s_fold_canon`), the Join by the same mergeable-queue

route, and the capstone

$$\texttt{sided\_embed\_ra\_linearizable3} \;:\; \mathrm{SReach}\ \Gamma\ C \to \mathrm{IsRALinearizable3}\ C$$

for every side assignment: sides are payload to convergence, which is the policy/datatype split of §21 as a theorem, with `sHonest_of_applicable` discharging honesty from the generation guard exactly as one-sided. The per-policy intent theorems have since landed, and the policy story is a machine-checked three-act arc (2026-07-16/17). *Act one*: `sFold_liftOp`, the all-R sided fold IS the one-sided fold, unconditionally (the sort keys coincide definitionally), so every one-sided theorem transports; and `sided_fold_subtree_convex`, subtrees display as contiguous blocks in any chain-generated fold, so runs that chain never interleave (`fugue_reachable_runs_no_interleave`). *Act two*: the Weidner–Kleppmann maximal-non-interleaving Definition 4 was formalized over reachable configurations (`SidedRGA_Fugue.lean`) and REFUTED twice for this Fugue policy on reachable traces (the recency tiebreak reverses the paper's lowest-ID-first; their Figure-7 execution): the paper's own Fugue-vs-FugueMax separation, landing on this implementation. *Act three*: FugueMax realized as a kernel variant (`SidedMax_ChainLex.lean`): its sibling order depends on the right ORIGIN, not expressible by any re-banding of deltas (the mirror control machine-witnesses this), so R-entries carry the mint-time successor's key as an order-reversing tag; condition (3) is proved in full (`fuguemax_same_origin_low_first`, the positive twin of the refutation), and full Theorem 9 has since been **closed** (§22: the traversal theory discharges condition (1), the corrected backward exception discharges condition (2), and `fuguemax_maximally_noninterleaving` is kernel-clean). The sided sequential spec also landed (`sided_seq_sound`: a sentinel-rooted two-sided buffer, R-inserts splice after their anchor, L-inserts immediately before). The tag has a COST: at contested inserts the coordinate embeds the successor's key, $\Theta(|\text{key}|)$ where plain Fugue pays zero, and whether that retention is a lower bound for tombstone-free maximal non-interleaving is an open question with a Myhill–Nerode shape (task #89; Open Question 40.17): the entropy law of §18 meeting the ordering intent.

Provenance. `whiteboard/litmus/embed_tree.py` (the design, the timestamp-oblivious control, the credential countermodel, the IEEE 754 measurements; `python3 embed_tree.py` / `fp` / `code`); `litmus.py`, `pbt.py` (battery and DAG harness); `contest_tree.py` (lockstep harness, CE regressions); `whiteboard/retention-countermodel.pdf` (the retention arc that forced this design); `whiteboard/delta-tree-design.pdf` (the ledger-based sibling design and the invariants I1–I3); `whiteboard/litmus/related-work.md` (verified source list: Roh et al. JPDC'11; Attiya et al. PODC'16; Kleppmann et al. PaPoC'19; Weidner & Kleppmann TPDS'25 (Fugue); Gentle & Kleppmann EuroSys'25 (Eg-walker, arXiv:2409.14252); Preguiça et al. ICDCS'09 (Treedoc); Grishchenko & Patrakeev PaPoC'20 (Chronofold); Litt et al. CSCW'22 (Peritext); Gomes et al. OOPSLA'17; OpSets 2018; Kaki et al. OOPSLA'19; Peepul PLDI'22; RA-lin OOPSLA'25; VeriFx ECOOP'23; Figma/Wallace fractional indexing; Weidner position-strings).

Part III
Sequential specifications: convergence is not correctness

Replication-aware linearizability certifies that every version of a replicated data type is the fold of some delivery-respecting linearization of its events. The reference point of that guarantee is the datatype's *own* fold: a bug in `do` appears identically on every replica, converges perfectly, and passes every convergence theorem. This part records the campaign that closes the gap. For every production RDT in Sal there is now a kernel-checked theorem that the fold of any single-replica history equals the output of a straightforward sequential program, the one a programmer would write with no replication in mind. The campaign's unifying observation is the *witness discipline*: sequential interfaces address state positionally (`dequeue()` is nullary, `insert` names an index), replicated operations cannot, and every replicated operation therefore carries the identity of what its issuer *observed* the positional referent to be; the honesty side conditions of the theorems are exactly the faithfulness of those witnesses. The campaign's negative results carry as much information as its theorems: the rehoming tombstone-free RGA is refuted at this specification on a four-op single-replica trace, its fused Peritext inherits the refutation at the render, and Shesha passes sequentially while failing at the merge, so do-faithfulness and merge-faithfulness are independent axes, and both separations are kernel-checked. A final section states what the campaign does *not* give: per-replica soundness and convergence do not compose into "the merged state is some user's sequential result," and the precise corner where composition fails is the dead-anchor corner the sequence-CRDT trilemma names. Every claim below names its Lean identifier.

28 The specification gap

The framework's central guarantee is RA-linearizability at every reachable configuration: each version the store registers is fold(ρ) for some enumeration ρ of the version's event set that respects delivery order. This is the right guarantee for *replication*: the merge invented nothing, reordered nothing it was not entitled to reorder, and all replicas agree.

It says nothing about whether the fold is *right*. The reference sequence is the datatype's own `do`; if `do` is wrong, every replica is wrong in unison (Figure 13). Two incidents made this concrete in this repository. First, a mark-extent defect in an early Peritext render passed every convergence obligation and was caught only by eye. Second, and decisively: the rehoming tombstone-free RGA carries a fully general, kernel-clean RA-linearizability capstone (`rga_ra_linearizable3_eq`), and its `do` nonetheless reorders surviving text when an interior element is deleted, with no concurrency anywhere in the history (§33). Both defects are invisible to convergence because convergence is self-referential.

The remedy is an independent specification. For sequential behaviour there is a canonical one: *the sequential program a programmer would write for the same operations with no replication in mind*. A counter is addition. A queue is a list. A text buffer is insert-at-position and delete. A marked-text editor is a buffer of characters and boundary tokens with one left-to-right walk.

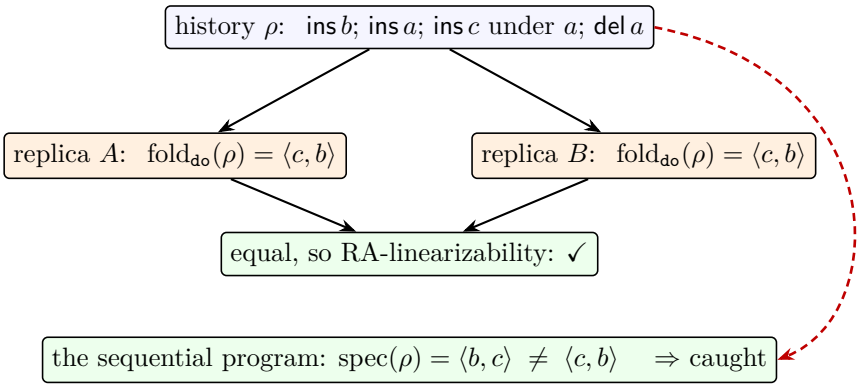


Figure 13: Why convergence cannot see a `do` bug. A defective fold is applied identically everywhere, so all replicas agree and every convergence theorem passes over the shared mistake. Only a reference computed by something *other than* `do` (dashed) can disagree. The history shown is the real refutation trace of §33.

29 The obligation, and the rules of engagement

The campaign's obligation, per datatype:

$$\textit{for every sequentially honest single-replica history } \rho\textit{:}$$
$$\text{read}(\text{fold}(\rho)) \;=\; \text{spec}(\rho)$$

proved by discovering an inductive invariant of the fold, never by weakening the spec to match the implementation. "Sequentially honest" is the datatype's own issuing discipline specialized to a linear history (fresh, monotone stamps; each operation applicable at the state it executes against), so the theorems attack no strawman: the histories quantified over are exactly the ones a single well-behaved replica produces, and in every case the side condition reuses a contract the convergence proofs already consume.

Three method rules were fixed in advance and held throughout. *Falsifiable statement first*: each tier states its equality before proof work begins, and a refutation is a first-class outcome (three landed, §33). *The spec is external*: the sequential program is written from the datatype's user-facing intent, never read off the implementation; where the two disagree, the disagreement is the result. *Tests carry both polarities*: every concrete test block pairs its expected values (hand-derived, never evaluated out of the code under test) with should-fail companions, so the harness demonstrably discriminates; the refutation files pair each negative with the positive prefix on which the two sides still agree, isolating the departing operation exactly.

30 The observation gap: positional interfaces, witnessed ops

Writing out the specifications exposes one structural pattern, and it is the campaign's most instructive artifact. A sequential structure addresses its state *positionally or implicitly*: `dequeue()` takes no argument because "the head" is unambiguous; `insert(i, e)` names an index; `write(v)` implicitly overwrites everything before it. A replicated datatype can afford none of this, because positions and implicit referents are not stable under concurrent edits: by the time an operation is applied elsewhere, "the head" may be a different element and index i a different character. So

the replicated operation carries a **witness**: the identity (tag, stamp, id, coordinate, snapshot) of what the issuing replica *observed* the positional referent to be, at its own state, at issue time (Figure 14).

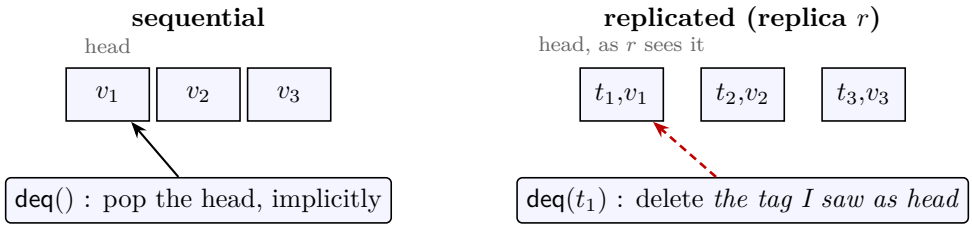


Figure 14: The observation gap, on the queue. The sequential `dequeue` is nullary; the replicated one names the tag its issuer read as the head of its own replica (dashed: the witness). Honesty is the faithfulness of the witness: `qOK` states that the fold at the issue point is literally $(t_1, v_1) :: \mathit{rest}$.

The sequential-honesty side conditions of §29 are then exactly the statement *the witness is what the sequential operation would have seen*, and each campaign theorem says: under that discipline, the witnessed-identity program implements the positional program. Where no witness is needed the alphabets coincide and the spec is a view homomorphism. The whole catalogue of §31 sorts by this principle, and the summary is one sentence: *every honesty condition in the campaign is the faithfulness of a witness that replaced a positional argument.*

31 The specifications, datatype by datatype

31.1 No gap: the alphabets already coincide

For nine of the flat datatypes the RDT operation is literally the sequential operation; the state carries invisible bookkeeping (tags, per-replica slots) and the spec is the evident program on the user-visible view.

- **Counter, increment-only counter.** Spec state $\mathbb{Z}$ (resp. $\mathbb{N}$); increment is $n \mapsto n + 1$. The fold *is* the history length (`counter_seq_sound`, `ioc_seq_sound`).
- **PN-counter.** Spec: $\#\mathsf{inc} - \#\mathsf{dec}$ (`pnSpec`); `pn_seq_sound`.
- **Grow-only set and map; conditioned G-set.** Spec: accumulation; x is a member iff some operation added x (`goset_seq_sound`, `gomap_seq_sound`, `gsetcond_seq_sound`).
- **OR-set, plain and efficient.** Spec state: a plain set; $\mathsf{add}\, e$ inserts, $\mathsf{rem}\, e$ deletes (`orSpecStep`). The tag machinery (fresh (t, e) stakes; remove filters every tag of e) is invisible to one replica: the element view of the fold is the naive program (`orset_seq_sound`, `orsete_seq_sound`), a fold homomorphism, no invariant needed.
- **Enable-wins flag.** Spec state: one boolean; $\mathsf{enable} \mapsto \mathsf{true}$, $\mathsf{disable} \mapsto \mathsf{false}$ (`ewSpecStep`); `ewflag_seq_sound`.
- **Add-wins priority queue.** A two-part spec pins both state components: membership behaves as the plain add/remove set with inc inert (`awpqSpecStep`, `awpq_mem_seq_sound`), and the increment component is a grow-only log of exactly the issued increments (`awpq_inc_log_sound`).

31.2 Gap: arbitration by stamp instead of program order

LWW and FWW registers. The sequential register is $\mathsf{write}\, v$ with *program order* deciding who wins; the spec programs are "the last write's value" (`lwwSpecFold`) and "the first write's value" (`fwwSpecFold`). The RDT operation carries its Lamport stamp *as the arbitration key in the payload* (a max- resp. min-semilattice on stamped writes), because concurrent replicas have no shared program order to appeal to. The gap is a transfer of ordering authority from control flow to data, and the honesty condition is precisely its repair: `stampsMono` (stamps strictly increase along the history, the sequential Lamport condition), under which the arbitration key agrees with program order and `lww_seq_sound` / `fww_seq_sound` recover last- and first-write semantics. Without `stampsMono` the statements are false, and should be: a replica that backdates a write is asking LWW to disagree with program order.

31.3 Gap: overwrite by snapshot instead of "everything"

Multi-valued register. The sequential $\mathsf{write}\, v$ implicitly overwrites everything before it. The RDT operation is $\mathsf{write}\,(v, O)$ where O is a **prepare-time snapshot**: the write-stamps the issuer currently sees, named explicitly so that a *concurrent* write (absent from O) survives, which is the point of an MVR. The sequential spec is still the plain last-write register (`mvrSpecFold`). Honesty (`mvrOK`): the stamp is fresh and O lists *exactly* the issuer's visible stamps, the formal form of "I overwrite precisely what I can see." Under it, `mvr_seq_sound`: the visible-value view of the fold is the last write. The proof needs the campaign's first genuinely discovered invariant, `mvr_over_tags`: every overwritten stamp was staked, which is what makes a fresh stamp provably not-yet-overwritten.

31.4 Gap: dequeue() becomes delete-what-I-saw

The mergeable queue, the cleanest specimen (Figure 14). The sequential queue is

$$\mathsf{enq}\, v : S \mapsto S +\!\!+ [v] \qquad\qquad \mathsf{deq}() : S \mapsto \mathrm{tail}(S).$$

The RDT operation is $\mathsf{deq}\, t$: remove *by tag*, t being the tag of the element the issuing replica read as the head of its own queue; the implementation filters t out of the whole list, wherever it sits. The spec keeps the sequential shape, `tail`, blind to t (`qSpecStep`). Honesty (`qOK`) is witness faithfulness: the fold at the issue point is literally $(t, v) :: \mathit{rest}$. The invariant `q_tags_nodup` (fresh stakes stay duplicate-free) makes filter-by-tag remove exactly the head, and `queue_seq_sound` closes the loop: delete-what-I-saw implements pop-the-head. Its axiom audit is the leanest in the campaign: {propext, Quot.sound}, no choice anywhere. The same witness discipline is what the convergence side already consumed (`qHonest_of_applicable`); nothing was invented for the occasion.

31.5 Gap: insert-at-index becomes insert-after-who-I-saw

The embedded-chain RGA. The sequential text buffer is $\mathsf{insert}(i, e)$ and $\mathsf{delete}(i)$, index-addressed. The RDT operation is $\mathsf{ins}(e, \pi, a)$: a is the *id* of the element the issuer saw at position $i-1$ (its anchor; the sentinel 0 for the head), and π is the anchor's stored coordinate as the issuer read it (the proof-carrying prefix); $\mathsf{del}(x)$ names the id seen at position i. The spec program is the buffer in id-clothing (`eSpecStep`): front-cons for $a = 0$, otherwise splice immediately after a's entry (`eInsAfter`); delete filters by id. On a sequential history the anchor id determines

the index uniquely, so this *is* the positional program with positions named by their occupants. Honesty (`eSeqOK`): stamps exceed all prior insert stamps, and each operation is applicable at the current fold, i.e. the anchor is live and carries exactly π. The theorem (`embed_seq_sound`, §32) is the strongest form in the campaign: the canonical sorted state equals the spec buffer *record for record*. The same spec, verbatim, is what the rehoming RGA is refuted against (§33); its gap is not the witness (the rehoming op carries one too, plus a path) but the implementation moving *other* elements' referents on delete.

31.6 Gap: one mark becomes a boundary pair

Fused Peritext. The sequential marked-text editor has an *atomic* $\mathsf{mark}(i, j, m)$. The fused RDT has no such operation: a mark is **two** ordinary inserts of boundary elements $\langle m \rangle$ and $\langle /m \rangle$ sharing a fresh mark id, anchored at the range ends; mark removal is two deletes. An atomic positional operation is traded for a *pair* of witnessed inserts, which is the atomicity horn of the mark-positioning trilemma, accepted knowingly. The spec therefore lives at the token level: the editor is the §31.5 buffer over char $\oplus$ boundary entries plus one left-to-right walk maintaining the open-mark set (a start boundary opens, the matching close removes, characters display with what is open). `peritextEmbed_seq_sound` says the datatype's render is that editor's screen, id for id in the `_ids` variant; the range-level reading is recovered by the positional intent theorems (`render_id_active_iff_between`: a character carries mark m iff it sits, in reading order, strictly between m's boundaries). The pair encoding is also why the character/boundary distinction is load-bearing in the delete theorem: deleting a *character* re-formats nothing (`renderIds_del`), while deleting a *boundary* is mark removal and must re-format its span (machine-checked necessary: `boundary_delete_breaks_filter_eq`).

32 The proofs: four tiers, three kinds of invariant

The campaign ran easiest to hardest, and the difficulty gradient turned out to be a gradient of invariant *kinds*.

Tier 1 (eleven flat datatypes), `SeqSpec_Flat.lean`. Mostly view homomorphisms, no invariant at all. The two exceptions need *identity hygiene*: facts about tag bookkeeping that hold on every reachable single-replica state and are exactly what the induction consumes. For the MVR, `mvr_over_tags` (overwritten stamps are staked); for the AWPQ, duplicate-freedom of stakes.

Tier 2 (the mergeable queue), `MergeableQueue_SeqSpec.lean`. One identity-hygiene invariant, `q_tags_nodup`, as described in §31.4.

Tier 3 (the embedded-chain RGA), `EmbedRGA_SeqSpec.lean`. The invariant becomes *geometric*. The spec splices an insert immediately after its anchor; the datatype sorts records by birth-chain coordinates; the bridge is the **adjacency lemma** (`chainBefore_snoc_iff`, Figure 15): a fresh insert's chain $c_a +\!\!+ [\delta]$ sits directly after its anchor's chain c_a in display order, because the fresh timestamp gap δ exceeds every delta any existing chain hangs off c_a, a fact that costs only sum-telescoping (a chain's deltas sum to its id, and every existing id is below the fresh stamp). The sorted canonical state then *is* the spec buffer, record for

record (`embed_seq_sound`). Two transports come for free. The published tombstoned RGA is read-equivalent to the embed on honest event sets (the compaction theorem, §27), so it inherits the buffer spec: `rga_seq_read_eq_buffer` and element-level `rga_seq_read_element` (`EmbedRGA_SeqSpec_RGA.lean`, a standalone build target because the two RGA models share top-level names). And the sided generalization simulates the one-sided instance on all-R histories unconditionally (`sFold_liftOp`), so the buffer spec transports to the sided datatype's all-R fragment at zero proof cost; the full two-sided theorem has since landed as `sided_seq_sound` (`SidedRGA_SeqSpec.lean`): a sentinel-rooted buffer where an R-insert splices immediately after its anchor and an L-insert immediately before it, via the two sided adjacency lemmas (`schainBefore_snocR_iff`, `snocL_iff`, axioms {propext, Quot.sound}).

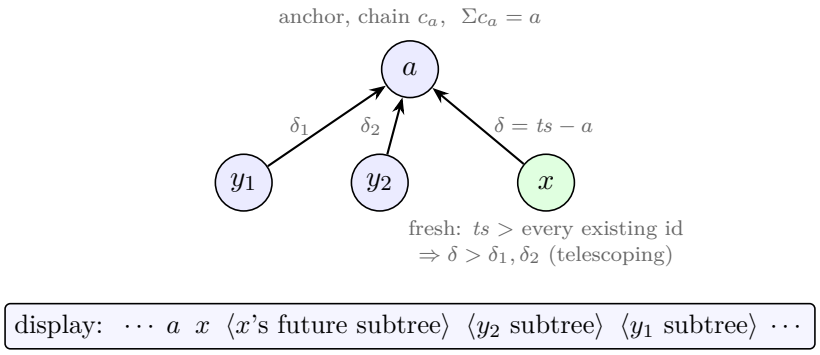


Figure 15: The adjacency lemma, the geometric heart of tier 3. Siblings display newest-first; a fresh insert's delta beats every existing child's because deltas telescope to ids and every existing id is below the fresh stamp. Hence the newcomer lands *immediately* after its anchor: exactly what the sequential buffer's splice does.

Tier 4 (fused Peritext), `PeritextEmbed_SeqSpec.lean`. The invariant kind is *composition*, and that is the finding: tier 3's buffer soundness, made payload-generic, already says the fold's (id, element) sequence is the naive buffer, and the datatype's render and the editor's screen are literally the same pure fold over that sequence (`peritextEmbed_seq_sound`). No new invariant was needed. This is also why the tier had to wait for the re-based Peritext: against the rehoming kernel the same statement is false (§33).

33 The refutations

The statement shape is falsifiable per datatype, and three refutations landed, all machine-checked, each with its honesty guards discharged on the witness so the failure is never an artifact of a malformed trace.

The rehoming RGA fails at `do` (`rehoming_seq_refuted`). Figure 16. Four operations on one replica, honest in the datatype's own terms (every op `accurate` and `fresh_ts` at its state): build $\langle b, a, c\rangle$ with c under a, then delete a. The naive buffer keeps survivor order, $\langle b, c\rangle$; the rehoming `do` re-homes c to the root, where the newest-first sibling tiebreak leapfrogs it over b: the read is $\langle c, b\rangle$. The should-pass half (`seq_agrees_before_delete`) shows both sides agree on the delete-free prefix, isolating the delete as the exact point of departure. The design's

convergence capstone is untouched: it converges, to its own reordering fold. This refutation is what demoted the design from the repository's canonical seat.

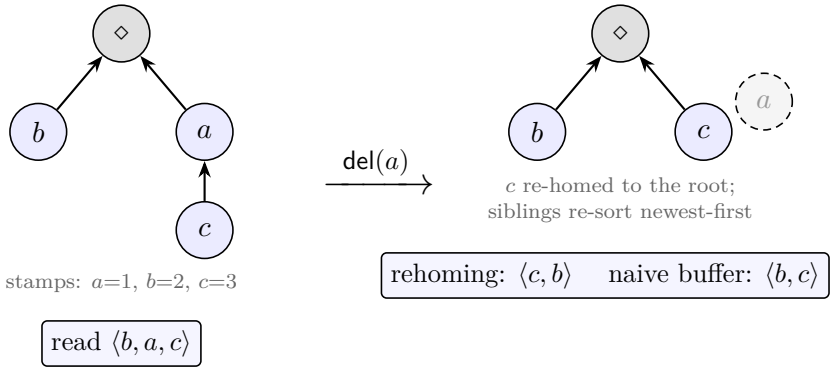


Figure 16: The four-op refutation. One replica, no merge. Deleting a swaps the surviving b and c in the rehoming RGA's read; the naive buffer keeps their order. The embedded-chain RGA reads $\langle b, c\rangle$ on the same trace: c's immutable coordinate still carries a's chain as pure geometry, so nothing moves.

Fused Peritext on the rehoming kernel fails at the render (`fused_delete_reformats_survivor`). The same reorder lifted to char $\oplus$ boundary: deleting a plain character re-homes a survivor past a mark boundary, and an untouched character changes formatting, on one replica. This upgraded the re-base of Peritext onto the embed kernel from housekeeping to a correctness fix. On the embed kernel the corresponding *general* theorem holds: `renderIds_del` (axioms $\{\text{propext}, \text{Quot.sound}\}$): deleting a character leaves every other character's rendered formatting bitwise untouched, because character entries are open-set-neutral and delete is a filter on an order-canonical state.

Shesha passes sequentially and fails at the merge. The mirror image. Shesha's single-replica reads are the naive linked list (`sequential_soundness`, kernel-clean), and it uniquely preserves survivor order under delete among the tombstone-free designs of its generation (`delete_preserves_survivor_order`). Its *merge* is machine-checked non-RA-linearizable (`Shesha_Rows_Refuted`): at an honest reachable configuration the three-way merge splits a deleted node's children around a concurrent sibling, producing a read that is the fold of no delivery-respecting linearization.

34 The two-axis separation

Together the refutations witness that do-faithfulness and merge-faithfulness are *independent* axes (Figure 17): the rehoming family occupies merge-yes/do-no, Shesha do-yes/merge-no, and the embedded-chain family shows the top corner is achievable, by one design idea in both sequence datatypes: immutable birth coordinates, so that deletion moves nothing and the merge decides nothing.

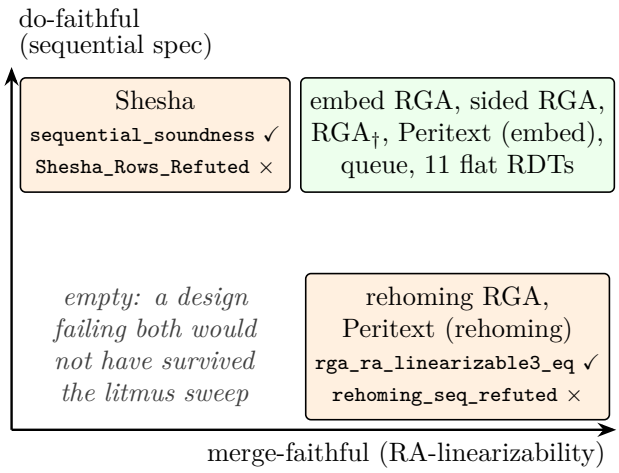


Figure 17: Every placement is a named kernel theorem. The two failure modes are orthogonal: converging to a reordering fold (bottom right) and folding correctly into a non-linearizable merge (top left).

35 What composition does not give

It is tempting to compose the guarantees: RA-linearizability says every version is fold(ρ) for some delivery-respecting ρ over the event union; the campaign says folds of sequentially honest histories equal the spec program. May one conclude that every merged document is the naive program's output on *some* ordering of everyone's operations? **No**, and the failure is precise.

The campaign's theorems quantify over *sequentially honest* histories; an RA-linearizability witness is delivery-respecting but need not be sequentially honest on either count. Stamp monotonicity fails harmlessly: when the state is a function of the event set (fold-canonicity, `e_fold_canon`), any enumeration can be re-sorted. Applicability is the real corner (Figure 18): in the union of two branches, an insert's anchor may be deleted by a *concurrent* operation carrying a smaller Lamport stamp, so in stamp-sorted order the delete precedes the insert, the anchor is dead at the insert's prefix, and the naive buffer no-ops the splice, while the datatype (whose coordinates travel in the op) places the element correctly. Some interleavings of the union match the spec program and some do not.

Whether a matching order always exists is genuinely open; it is recorded as Open Question 40.12 in the consolidated list.

Independently of the question, the composition that *is* sound and kernel-checked today: every version is the datatype's fold of a delivery-respecting enumeration (RA-linearizability); the fold is canonical per event set (`e_fold_canon`); each replica's own contribution obeyed the sequential spec while it was being made (this campaign); and merge-level intent is delivered by separate, named theorems where it holds (delete-order preservation as sublist stability, subtree convexity as non-interleaving, read-equivalence to the published RGA as the compaction theorem). The campaign does not manufacture a merge-level user story; it pins the single-replica story so that merge-level claims have something sound to stand on.

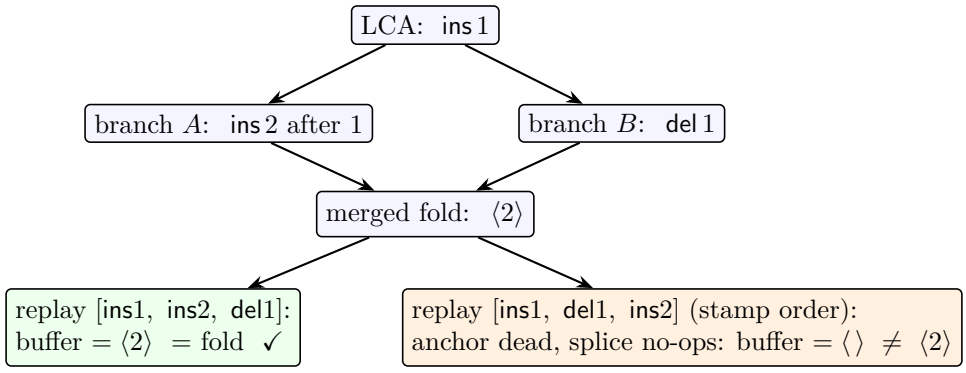


Figure 18: The dead-anchor corner. The merged fold keeps 2 (OR-set survival on immutable coordinates); whether a naive-buffer replay of the union agrees depends on which delivery-respecting order is chosen, and the stamp-sorted one can be wrong. This is the same corner the sequence-CRDT trilemma names for insert-after-already-deleted-anchor.

36 The verified matrix

The anomaly-comparison matrix (task #64, `whiteboard/anomaly-matrix/`) ran six implementations and four production libraries through 25,917 merges per row, every cell backed by executed code. Its in-repo rows have since been upgraded from machine-run to kernel-checked; the table below is the matrix's load-bearing core with each cell a Lean identifier. External libraries (Yjs, Automerge, Loro, list-positions) remain empirical by nature; their rows live in the matrix report unchanged.

design	sequential spec (do)	RA-lin (merge)
embedded-chain RGA	`embed_seq_sound`	`embed_ra_linearizable3`
sided RGA (two-sided)	`sided_seq_sound`	`sided_embed_ra_linearizable3`
published RGA$_\dagger$	`rga_seq_read_eq_buffer`	`rgaWithTombstones_ra_lin..._eq`
Peritext (embed)	`peritextEmbed_seq_sound`	`peritextEmbed_ra_linearizable3`
mergeable queue	`queue_seq_sound`	`queue_ra_linearizable3`
eleven flat RDTs	`*_seq_sound`	`*_ra_linearizable3_eq`
rehoming RGA	**refuted**: `rehoming_seq_refuted`	`rga_ra_linearizable3_eq`
Peritext (rehoming)	**refuted** (render): `fused_delete_reformats_surv...`	`peritext_ra_lin.._eq`
Shesha	`sequential_soundness`	**refuted**: `Shesha_Rows_Refuted`

Every positive theorem above is kernel-clean (axioms $\subseteq$ {propext, Classical.choice, Quot.sound}; the queue's and `renderIds_del`'s audits omit choice). Refutations by concrete witness follow the SPOT convention (`native_decide`, adding the compiler axioms); the refuted *statements* are quantified properties whose honesty guards are discharged on the witness.

Part IV
The verified peer-to-peer runtime

The preceding parts are a metatheory and its instances: theorems about what a merge computes and what a compaction preserves. This part is a systems pillar built on them. The same design runs as a general peer-to-peer MRDT runtime, datatype-parametric over the framework signature, of which collaborative sequence editing is one instance; the runtime's garbage collectors are

the executable form of Part I's GC-safety and stability theorems, its save format is the run table of Part II, and a benchmark measures it against production CRDT libraries on the standard editing-trace corpus. One boundary is drawn throughout: the Lean development is the verified artifact, and the JavaScript runtime is an *unverified transliteration* of the verified kernel that mirrors its semantics and whose GC callbacks correspond, named theorem by named theorem, to the mechanized results. The runtime does not re-prove convergence; it exercises the semantics the theorems certify and asserts the observable consequences (equal reads, reads unchanged by GC, git round-trip). Every performance number is a run of checked-in scripts on one machine, labeled measured or measured-in-model.

37 The runtime: one content-addressed replica

Datatype-parametric, proven over two datatypes. The core object is `DistributedReplica` (`runtime/src/replica.js`, tasks #94/#108): a single content-addressed commit store carrying three capabilities at once — delta gossip over the wire, the certified stability GC, and the keep-set commit GC — every one of them datatype-agnostic (`init`/`apply`/`merge3`/`read`) except state compaction, which a datatype opts into by additionally providing `compact`/`remapState`. The runtime is therefore not a text editor with a CRDT inside; sequence editing is one instance. The test suite runs convergence, the content-address round-trip and tamper gate, and the commit GC over *both* the embedded-chain RGA and an OR-set, and the certified state GC over the RGA (refuse-then-fire) with the OR-set correctly declining it (an OR-set has no coordinate re-coding to do). The pieces that Part I proved generic run generically.

Content addressing: one hash for wire and disk. Separate peers assign different local ids to the same commit, so every commit is named by the SHA-256 of its canonical content, a Merkle DAG folding in parent ids (`runtime/src/hash.js`, #108): an authored commit hashes $\{\text{parents}, \text{replica}, \text{seq}, \text{payload}\}$, a merge commit hashes its *sorted* parent ids (so $\mathsf{merge}(a, b)$ and $\mathsf{merge}(b, a)$ are literally the same commit and never diverge into a spurious criss-cross), the root hashes $\{\text{root}\}$. Ingest is guarded: a peer recomputes each incoming commit's state (`apply` for authored, `merge3` for merges) and the recomputed content id must match the wire id, so a transmitted state is never trusted. The pure-JS SHA is checked bit-for-bit against the platform crypto library. The *same* id names a commit on the wire and on disk, which is what makes git persistence content-addressed with no second hash.

Delta sync, head-sync preserved. Two peers exchange their DAG frontiers (head content-ids), each computes the ancestor-set difference the other lacks, and ships it as a *delta*: per commit only the op payload, parent refs, and author, never a whole state. A whole-state snapshot (the run-table serializer below) is used only for a genuine bulk catch-up, chosen when the delta would be larger (a brand-new or very-far-behind peer). Crucially a peer only ever merges its current head with a head another peer just advertised, never a stale interior commit: this is exactly the head-sync hypothesis the GC-safety theorem (§15) consumes, made structural rather than advisory. Merges route through the same criss-cross gate as the in-process runtime.

The document is a git repository. Persistence writes one JSON file per commit named by its SHA, a heads file, and a materialized `doc.txt`, then commits them *in a git repository*; loading

replays the commits into a fresh replica whose reads equal the original and which re-enters the live wire with identical SHAs. `git clone` of the repository therefore yields a self-contained, loadable document (clone is a join; a fresh replica id is a fork), and because `doc.txt` tracks the content, plain `git log -p` reads as sensible edit history. The content-address SHA is the git-object discipline reused: the same id on disk as on the wire.

Certified GC in the running system. The stability callback is the executable `settledAt_of_allHeard` (§16). `compactStable` replaces asserted settledness with a *checked* certificate built from the frontier: if any registered replica has not been heard from since the cut, compaction is refused — a no-op reporting the missing replica — and fires only once that replica is heard from. Absence of evidence is refusal, never assumption; this is the runtime witness of the theorem's createReplica breaker. The directed test drives the discriminating-remove countermodel of §16: a lagging replica makes `compactStable` refuse, then fire when heard from, with reads identical to a never-compacted control throughout, and a FAIL companion in which the *old* asserted compaction at the same point diverges (a frozen-delta order flip), so the certificate's refusal is load-bearing, not pessimism. Under a certified cut the in-flight crutch vanishes: every op concurrent with the cut is already delivered (SettledAt), so passing an empty in-flight set is *proved* sufficient rather than asserted. The keep-set commit GC runs off the same frontier read from above (§15), and the run-table serializer (§24.4) supplies the save format.

Honest limits. The demo (`p2p-demo/`, task #95) shows several replicas editing one document over a real WebSocket transport, converging, saving to and cloning from git, and firing certified GC while the session is live. What it is not: the transport is a *star relay*, not a network-layer mesh (the transport interface is shaped so a WebRTC data-channel mesh drops in unchanged); there is no authenticated identity (the SHA gate bounds tampering with a commit's content, not who may author); open membership is cooperative, over the closed replica set the stability certificate and the commit GC both quantify over; and compaction is *linearized* by a coordinated checkpoint barrier, because concurrent divergent compaction across replicas — two peers compacting incomparable cuts and then merging across epochs — is the deferred protocol half, the multi-replica layer above the single-replica lemma ($*$) of §24.2. A cross-epoch merge throws rather than guess. And the datatype is the unverified transliteration noted above.

38 Performance

Where does the shipped artifact sit against production CRDT libraries? The benchmark (`benchmarks/`, task #98) runs the runtime against Yjs 13.6, Automerge 3.3 (wasm), Loro 1.13 (wasm), and list-positions 2.0 on three workloads, and every cell is a run of the checked-in scripts on one machine (node v26.3.1, Apple M4 Pro, 24 GB); nothing is quoted from literature, and each system's convergence and final text are gated before any number is reported. The corpus is the standard collaborative-editing replay set (the josephg traces: `automerge-paper` is Kleppmann's LaTeX-editing trace, `seph-blog1` is Gentle's blog draft, and `friendsforever/clownschool` are concurrent sessions, here also replayed single-author as the `_flat` twins). One honesty note up front: we benchmark *on* this corpus but not *against* Eg-walker's own implementation (§25), which is an event- graph replay engine rather than a state library; that comparison is a genuine gap.

Per-metric methodology, stated natively. *Apply* is timed with a high-resolution clock around each single-character op, reported as median and p95 after a 1000-op warmup (JIT and wasm lazy init); timer overhead of 30 to 60 ns per op is *not* subtracted and biases the sub-microsecond medians upward. For ours each op includes the adapter's position-to-id bookkeeping (an id-array splice) on top of the datatype `apply`. *Save size* is the bytes of each library's native serialization, and what a save *contains* differs, so every row is labeled: Automerge's `save` is the full change history (compressed; no state-only mode exists), Loro's `snapshot` carries state plus history while `shallow-snapshot` drops history at the frontiers, Yjs's update drops deleted content but not tombstone id structure (v2 run-length codes the same content), list-positions saves live characters plus position metadata as JSON, and ours saves live state only (delete is pure removal). Ours is reported in four labeled representations, discussed below. *Load* is from the primary save into a fresh document, median of five, including materializing the text once; for ours it includes re-deriving the display order (the coordinate sort). *Merge/sync* times the bidirectional exchange per round in the concurrent workload. *Heap* numbers are flagged not comparable across the wasm boundary (Automerge and Loro hold state in wasm linear memory, invisible to the JS heap counter). Labels throughout distinguish *measured* from *measured-in-model*.

The honest headline: apply latency. Where we lose, as shipped, is per-character apply, by three to four orders of magnitude (median apply, by trace and final size):

apply, median (final chars)	`ff_flat` (21k)	`cs_flat` (21k)	`seph` (57k)	`am-paper` (105k)
ours (embed RGA, shipped)	364 μs	347 μs	1.66 ms	3.16 ms
Yjs	1.5 μs	1.7 μs	1.4 μs	1.4 μs
Automerge	24 μs	26 μs	27 μs	27 μs
Loro	0.6 μs	0.8 μs	0.6 μs	0.5 μs
list-positions	0.6 μs	0.7 μs	1.0 μs	0.7 μs

The diagnosis is in the trend: ours grows with document size while Automerge is flat and the rest are flat and sub-microsecond. The persistent `apply` returns a fresh Map, an O(live set) copy per op, so cost is linear in the document. This is *interface-inherited* — the persistent-state datatype interface the verified semantics is stated over — not order machinery, and a transient or batched apply path would move ours toward the microsecond class without touching the verified semantics. Load sits at 31 to 253 ms (JSON parse plus the coordinate sort) against Loro's sub-millisecond snapshot load, though within range of Automerge's full-history load (on `automerge-paper`, ours 253 ms versus Automerge's 319 ms). Retained heap after replay is small (2 to 12 MB: absolute coordinates share prefixes structurally as cons strings, so the bit cost materializes only at serialization), paid for as allocation churn.

Where we compete: merge and sync. The unique-LCA three-way merge under the head-sync runtime is Yjs-level and 20 to 25× faster than the wasm libraries in these sessions:

system	sync median freq	sync median bulk	wire payload/sync freq	wire payload/sync bulk
ours (embed RGA, shipped)	73 μs	731 μs	in-process[†]	
Yjs	90 μs	1.13 ms	1.6 KB	15.1 KB
Automerge	1.82 ms	18.3 ms	in-process	
Loro	6.29 ms	17.1 ms	0.4 KB	4.6 KB
list-positions	34 μs	626 μs	6.4 KB	123.6 KB

(`freq` = 60 rounds of 25 ops/replica, final 1804 chars; `bulk` = 6 rounds of 500 ops/replica, final 3604 chars; commit GC runs outside the timed window at millisecond totals.) The honest caveat: these are small documents, and our merge is O(live set), so the numbers do not extrapolate to 100k-character documents. [†]The shared-store benchmark merges in-process, so the payload column is not applicable; the separate-store wire protocol (§37) ships a delta that is a function of that round's ops, constant across rounds while a whole-state resync grows with the document, and the runtime's sync tests measure it at 0.28 to 0.60× the whole-state baseline in steady state.

Save size, four representations. Our shipped runtime stores absolute chain coordinates as bit-strings, so the naive save is far above production; the run table (§24.3) is the designed successor, and task #104 ships it as a real serializer. The four representations, in bytes per live character (final state of each sequential trace):

representation (ours)	`ff`	`cs`	`seph`	`am-p`
as-shipped JSON	1637	2503	2991	2320
+ settled-cut compaction	871	1486	933	1295
run-table serialized, SHIPPED	1.5	1.6	1.6	1.2
+ compaction, SHIPPED	1.1	1.1	1.3	1.1
run-table projection (model)	3.8	3.8	3.9	4.0

In absolute terms the shipped, compacted run-table save is **24 / 22 / 73 / 120 KB** for the four traces, down from the as-shipped JSON's **35 / 53 / 170 / 243 MB**: four orders of magnitude, lossless (`decode(encode(s))` reads identically, gated on every trace and by a 120-trial merge/delete property test). The serializer is anchored to the shipped state, not a different datatype: its `accountingBits` routine reproduces the projection's bit totals bit-for-bit, and the shipped bytes sit *below* the projection because the model deliberately charges the recoverable positional fields (per-record run id and offset, and the tail-attachment parent offset) that a real encoder drops. Against production, the shipped run-table save reaches rough parity with the smallest state-only saves:

system, variant	contains	`ff`	`cs`	`seph`	`am-p`
ours, run-table + compaction (SHIPPED)	state	1.1	1.1	1.3	1.1
ours, run-table projection	state (model)	3.8	3.8	3.9	4.0
Yjs update-v1	state + tombstone ids	3.8	4.4	6.0	3.0
Yjs update-v2	run-length coded	1.9	2.0	2.4	1.5
Automerge save	full history	1.3	1.2	3.6	1.2
Loro snapshot	state + history	2.9	3.1	5.9	2.4
Loro shallow-snapshot	state only	1.0	1.1	0.9	0.6
list-positions	state (JSON)	4.7	2.9	10.1	4.7

The shipped save is below Yjs update-v2 and Automerge's compressed history and on par with Loro's shallow-snapshot (0.6 to 1.1 B/char, still the smallest); history-carrying rows (Automerge always, Loro snapshot) are never compared against state-only rows without the label saying so.

The categorical win: storage on delete. Ours is the only save that shrinks when the document does. The churn workload runs five cycles of insert 2000 then delete 1800 characters, then a final 200-character insert, and asks per row whether the save *grows across a delete phase* (bytes, ours in the packed binary estimate):

system, variant	c1-ins	c1-del	c5-ins	c5-del	final	grows?
ours (binary est.)	22278	2191	72322	25891	31596	never
ours, compacted (est.)	11576	1090	18884	6524	7882	never
Yjs update-v1	32931	31671	163331	161995	165395	never
Yjs update-v2	9539	8812	45776	44841	45856	never
Automerge save	8004	11387	56294	59724	60742	**always**
Loro snapshot	13349	15337	76144	78290	79679	**always**
Loro shallow-snapshot	6170	878	8677	3296	3882	never
list-positions	121933	94172	454518	428465	436314	never

The final document is 1200 live characters in every row. Automerge's save grows by roughly 3.4 KB across every delete phase (history-carrying, monotone by design) and Loro's full snapshot likewise; Yjs never grows but cannot shed tombstone structure (165 KB v1 for those 1200 characters against our compacted 7.9 KB and Loro's shallow 3.9 KB). This is the storage-on-delete axis of the retention taxonomy (§26) measured live: tombstone-freedom is the design's semantics showing up as the only save that gets smaller when the document does.

Caveats, so the numbers cannot be over-read. Timer overhead is not subtracted and biases the sub-microsecond medians; Automerge is driven one change per character (the automerge-perf convention; batching would lower its per-op cost and history size); list-positions is a positions library, not a full CRDT, and its sync row is its documented op-log integration with an unoptimized JSON payload; heap numbers are not comparable across the wasm boundary; and cross-system merged *order* may legitimately differ on concurrent insertions, so only intra-system convergence is gated.

39 Push-button discharge: an SMT translation-validation layer

This is a tool, not a theorem, and the section labels it as such. The eight verification conditions the flat capstone consumes (§11, `flat_ra_linearizable3_eq`) are, for a flat instance, algebraic enough to hand to an SMT solver, and the `smt/` layer (tasks #99/#49) does exactly that: per instance it discharges the eight by z3, via unconditional laws that each *imply* the corresponding conditioned VC (the `DeltaVCs3` on-ramp for the feasible laws, `all_comm` for the causal-delta bound). This is *translation validation*: the Lean metatheory stays the source of truth, and the trust gap is the fidelity of the hand-mirrored z3 term to the Lean signature, closed by calibration rather than by a verified translator.

Calibration, exact against Lean. Four shapes span the space: Counter/PN (numeric group), GSet (set lattice), LWW (timestamp lex), and ORSet (instance-set with kill sets, non-commuting). The first four return UNSAT (VC valid) on all eight, matching their Lean discharges exactly. The OR-set is the informative one: six of eight are push-button, and the two that come back `sat` are precisely the two that are *configuration-conditioned in Lean* (its `feasible_local_redistribute`, which needs canonical-state tag-freshness, and `CDVC3`, which needs the maximal-remove trichotomy). Their unconditional over-approximation is genuinely false, so the solver's `sat` is not a mismatch with Lean but a precise identification of the two VCs that required hand proof. Five deliberately broken mutants (drop the $-l$ in a counter merge, `max`-merge a PN, project a GSet merge, arbitrate LWW on the timestamp alone, drop the $\neg l$ guard in the OR-set) are each caught with a readable countermodel.

The #49 loop, and the normal form it forced. The success metric was a remove-wins set whose remove-records are garbage-collected against the LCA. Its literal specification comes back `sat` on both feasible-redistribute laws, and the loop's finding is why: the LCA-conditioned supersession drop is *order-sensitive* (whether a remove is collected depends on adds introduced by the very delta being redistributed), so the merge is not a convergent semilattice and the delta laws fail unconditionally, with a countermodel exhibited. The repair the loop forces is a *normal form*: normalize each element's remove-records to the per-element maximum timestamp (a remove superseded by a higher one is intrinsically absent), and likewise its adds; the merge becomes a product of two `max`-semilattices, LCA-blind, and all eight VCs then discharge push-button, zero hand proof, with the semantics preserved exactly (an element is present iff its maximum add strictly exceeds its maximum remove). Two further kill-tests (eager GC without the LCA guard; `min`-ing the remove timestamp) are refuted with countermodels. The SMT loop therefore did not merely certify #49's spec; it refuted the literal one and pushed the design to the max-timestamp normal form under which the whole bundle is push-button.

Recommendation. The right integration is certificate replay: use the SMT layer as a pre-flight oracle that tells a new flat instance's author instantly which VCs hold and which need conditioning (the OR-set-style boundary) and produces the countermodel when a design is broken, with a thin Lean tactic re-proving the UNSAT cells natively (most already close by `omega` or `cases <;> rfl`). The residual hand proof then shrinks to exactly the configuration-conditioned cells the triage flags — for the whole commuting class, and for the repaired #49 design, empty.

Part V
Open questions and the mechanization map

40 Open questions

The consolidated list: the framework questions of the metatheory (Part I), the campaign's successor question (Part III), the retention question of the FugueMax arc (Part II), and the questions opened by the runtime and storage extension (virtual LCAs, GC, stability, the run table). Two resolved questions are kept at the end in compressed form, because the later parts

depend on their corrections; the Weidner–Kleppmann Theorem-9 gap, listed as open in the previous revision of this document, is closed (§22) and no longer appears.

Open Question 40.1 (Is there a rely-guarantee program logic for MRDTs?)**.** *The framework already carries the skeleton of a verification-condition logic: the signature-plus-conditioning is the "program", RA-linearizability up to $\approx$ is the judgment, the eight VCs are local proof obligations, the metatheorem is their soundness theorem, and the product $\otimes$ with* `JoinLemma3At` *is a frame/composition rule (the coupling hierarchy L0–L3 stratifying it by interaction strength, as disjoint-vs-shared reasoning does in separation logic). The conditioning layer is specifically rely-guarantee: honest delivery is the rely (an assumption on other replicas' and clients' steps), convergence/safety up to $\approx$ is the guarantee, merges are environment steps, and the "stable under concurrent honest extension" condition the safety metatheorem turns on (§13: the bounded counter succeeds and BudgetCart/FWW fail on exactly it) is the rely-guarantee stability side-condition, arrived at independently. What is missing to make it a program logic proper is a syntax: a language building* do/merge *from primitives (a per-replica counter cell, OR-set membership, an LWW/FWW* max/min *cell, a sequence node) and combinators (the product $\otimes$, the indexed map, the payload-parametric wrap of* `ORSetCore`*), each carrying its local VC contribution and its join-species, such that well-formedness entails RA-linearizability by construction — so that writing an MRDT and proving it converge become one act, as separation logic made heap-program verification syntax-directed. The semantic combinators already exist and are each separately proven sound; what is open is the grammar together with a single soundness theorem for the grammar (not one per combinator). This question subsumes the two framework-theory questions below:* CDVC3*-derivability (40.2) asks whether the rule set is minimal, and the structure theorem asks to classify the primitives.*

Open Question 40.2 (CDVC3-derivability)**.** *Is* CDVC3 *derivable from VCs 1–7 (plus the unconditional delta laws where they hold)? Specializing the merge to an LCA-blind lattice join — the CRDT case — the question closes exactly: there, the corresponding causal-delta bound is equivalent to the Join Lemma and no strictly weaker bridge VC exists, with both attack routes characterized — the positive induction is circular at two identified case shapes, and the countermodel space is narrowed to non-atomic lattice states (all of this mechanized, in* `Sal/CRDTs/Metatheory/`*). The ternary question, by contrast, is now* ***closed****:* `CDVC3` *is an independent rule of the ternary bundle (*`cdvc3_not_derivable_from_core_delta`*,* `Refutations/CD_Not_Derivable_Ternary.lean`*, kernel-clean) — there is a* `ConditionedMRDTSig` *(*`AWSetF3`*, the LCA-blind lift of the binary inflation-separator, with a deflationary* rem*) satisfying every* `CoreVCs3` *and* `DeltaVCs3` *clause yet falsifying* `CDVC3`*, so* CoreVCs3 + DeltaVCs3 $\not\Rightarrow$ CDVC3 *(and, via* `joinLemma3_iff_cdVC3`*, $\not\Rightarrow$ the Join Lemma). The reason ternary closes where binary (b'') stays open is instructive:* `CDVC3`*'s $\sqsubseteq$-half demands update inflation ($A \sqsubseteq \mathsf{update}\, A\, e$), which the binary* `LatticeVCsPlus` *ships as a separate axiom but the ternary* `DeltaVCs3` *honestly omits — so the inflation-free delta laws cannot back* `CDVC3`*, and a deflationary update refutes it. The eight-VC bundle is therefore* ***minimal at*** `CDVC3`*: the causal-delta rule is a genuine independent axiom, not a derived lemma (a load-bearing point for the rule-basis of Open Question 40.1)*

Open Question 40.3 (structure theorem)**.** *Empirically, the unconditional delta contract holds exactly on the group and lattice classes. Is every unconditional-*DeltaVCs3 *merge a torsor-like group$\otimes$ lattice combination? (The naïve two-sorted umbrella* $\mathsf{mergeL}(l, a, b) = a \boxplus (b \boxminus l)$ *does not derive the contract uniformly, so the question is genuinely open.)*

Open Question 40.4 (the automation layer)**.** *The published catalogue's seventeen induction-scheme VCs were an SMT-facing Skolemization of feasibility, aimed at the wrong target. Design the analogous per-data-type induction scheme whose induction implies* CDVC3 *and the feasible delta laws — restoring push-button discharge, now of VCs that actually imply RA-linearizability. The observed uniformity of the discharges (σ-characterization by sandwich invariant + absorber trichotomy) suggests the scheme exists.*

Open Question 40.5 (tightness of the MCA-closure keep set)**.** *Under virtual merges the keep set retains the upward event-set closure of* mcasClosure(heads)*, and safety is proved: every future virtual resolution reads only kept versions (`gc_safetyV`, §15). Tightness is the unexamined converse: is every closure member actually readable by some future virtual merge? The harness already shows non-members can be dead, but whether the closure itself over-retains is open; a negative answer (some closure members are unreadable in every future) would license a strictly smaller keep set, and the gateway lemma's structure suggests where to look: closure members reachable only through non-head prune-time versions.*

Open Question 40.6 (open membership and certificate transport for stability)**.** *The stability gate is no longer assumed. On an honestly reachable configuration* AllHeardSince *(a purely local frontier fact) implies* SettledAt *(`settledAt_of_allHeard`, §16), and the running system realizes the theorem as `compactStable`'s refuse-then-fire (§37). Two operational gaps remain. Open membership: the certificate quantifies over a known replica set, and a replica that joins after a compaction is the theorem's one breaker (*createReplica*), handled today by a policy gate; a model where the replica set is itself a grow-only MRDT could turn the gate into a theorem. Certificate transport: can a replica hand a peer a compact, checkable certificate of SettledAt (a frontier vector plus the evidence commits) so the callback fires at nodes that did not compute the frontier themselves? The commit-shaped-not-event-shaped erratum constrains the answer: the certificate must name commits, since an event-set summary can miss the pull that carries no new events yet opens the gate.*

Open Question 40.7 (the honesty-rebasing lemma $(*)$)**.** *The single-epoch honesty gap (T3: the re-coded history is honest only modulo the coordinate order-embedding, §24.1) and the multi-epoch* CompatChain *residue (§24.2) are the same obligation, and both of their supporting halves are already mechanized: the causal-stability half (`nested_cuts_settled`, from* settledAt_of_allHeard *and frontier antitonicity) and the order-iso freshness half (`rED_hocc_fresh`). What remains is one lemma $(*)$: exhibit an epoch-2 configuration honest modulo the order-embedding F_1, so that anchored-existence re-derives at the epoch boundary and the capstone re-invokes natively in the new epoch. This is the whole remaining single-replica compaction protocol, and Open Question 40.8 is the multi-replica layer above it. A refutation would be as informative: an honest re-coded configuration that no order-embedding rebasing can certify.*

Open Question 40.8 (concurrent compaction: the epoch DAG)**.** *The mechanized stability metatheorem covers compaction epochs that are linearized per replica (the v1 runtime, and the demo's checkpoint barrier, refuse compaction off the newest epoch), VC-S6 gives observational coherence for nested cuts on one lineage, and the single-replica multi-epoch composition is now closed up to the shared honesty-rebasing lemma (Open Question 40.7). Concurrent divergent compactions are the remaining, multi-replica, protocol half: two replicas compact at incomparable cuts S, S', and their states must still merge. The conjectured shape is an epoch DAG with common-refinement translation (translate both sides to the join epoch before merging), and the*

remap species makes the obligation concrete: two rank-renumberings of overlapping sibling groups must commute observationally after refinement. The harness finding that epoch commits perturb the subject's commit graph (a control fast-forward can be a subject merge) says the twin-execution proof technique will need the version-map generality the self-merge trick deliberately avoided.

Open Question 40.9 (rc-expressiveness: chains, LWW, and merge-recoverability)**.** *The catalogue's* rc *relations are all star-shaped: the conflicting pairs are cross-class (*rem/add*; add/checkout), so every edge runs from a class that is never second to a class that is never first. This is forced: the update layer's directionality law (`rc_non_comm_directional`) orients every non-commuting concurrent pair, while its companion `no_rc_chain` forbids length-2 paths, and any orientation of a self-conflicting class (concurrent writes; concurrent claims of one slot) contains one. So total-order policies — last-writer-wins first among them — are inexpressible in the* rc *mechanism as constrained by the VC set. Two questions, one of them already closed at its boundary. (i) Would a chain-tolerant generalization (the concurrent arm of* lo *as a transitive tiebreak) be sound? The semantics is coherent — with Lamport clocks the composite of* vis *and any timestamp- or priority-oriented tiebreak is acyclic, and canonical states remain well-defined — so the restriction appears to be an artifact of the swap-flavored proof machinery (`cond_comm_lift` cannot bubble past two consecutive* rc *edges), not of the theorem. The decisive experiment: re-prove the swap VC and the causal-delta peel over a transitive tiebreak; we expect the swap route to break and the canonical-state route (which peels a global maximum, and a total order has one everywhere) to survive. (ii) What would it buy? Not a metadata-free LWW register: whatever the order arbitrates, the converged outcome must be produced by* $\mathsf{mergeL}(l, a, b)$ *— a function of three states — and for a plain-value LWW two executions present identical state triples with opposite timestamp orders, so no such function exists (`lww_merge_needs_timestamps`, `Refutations/LWW_Merge_Needs_Timestamps.lean`, kernel-checked). The arbitration key is forced into the state by the merge; once there, writes commute and* rc *is moot — which is why every LWW implementation stores its timestamp. What a chain-tolerant arm could buy: metadata-free priority policies across three or more op classes (outcomes state-recoverable, orientations chained). The general principle the kill-test isolates:* rc *is the mechanism for conflicts whose outcome is recoverable from branch states; clique conflicts pay for their arbitration key in state, and chains would extend* rc *only within the recoverable class.*

Open Question 40.10 (Causal canonicity under general rc)**.** *The generic safety metatheorem consumes causal canonical witnesses — enumerations linearizing* $\xrightarrow{\mathsf{vis}}$ *and respecting* lo^E *jointly. With every concurrent pair unordered (Either), the two discharge species of §13 suffice. Under a general* rc *the existence of a joint linearization is open: a cycle would need at least two* rc*-edges separated by* $\xrightarrow{\mathsf{vis}}$*-paths — a single* rc*-edge closed by a* $\xrightarrow{\mathsf{vis}}$*-path contradicts concurrency via transitivity, and a non-commuting first* $\xrightarrow{\mathsf{vis}}$*-step is an absorber killing the edge — and `no_rc_chain` forbids adjacent* rc*-edges but not* $\xrightarrow{\mathsf{vis}}$*-separated ones. `CausalCanonical` is therefore a hypothesis with two discharge lemmas rather than a theorem. The budget cart (`BudgetCart`, `MRDT_Instances/BudgetCart/`) has now forced the question — and widened it: its convergence discharges along the OR-set route, but the ungated safety obligation `SafetyStepOn` is false for it, by a two-event refutation — `CausalFold` pins only* $\xrightarrow{\mathsf{vis}}$*-respect, and the fold of a causal past containing a concurrent same-item* add/rem *pair is enumeration-dependent (the orders disagree exactly where add-wins should arbitrate). So for* rc*-nontrivial datatypes the honesty fold needs* rc*-orientation too: the right notion is folds along jointly* $\xrightarrow{\mathsf{vis}}$*- and* lo^E*-respecting enumerations, for both the version witnesses (`CausalCanonical`) and the causal-past folds (`CausalFold`). The*

cart's safety theorem is delivered gated (`bcart_version_inv_gated`, with the missing transfer named explicitly as `BCartSpendMono`); closing the gate is this question.

Open Question 40.11 (Does conditioning compose? — convergence: yes, now a theorem)**.** *The catalogue is built one data type at a time; practice builds data types from data types — the canonical case is the key-value map whose values are themselves MRDTs (the certified-MRDT line's own composite). The factored framework suggests composition should happen at the `JoinLemma3At` boundary — the map combinator would consume each value's per-configuration join, whatever species produced it, so a map of queues composes as readily as a map of counters — with a small once-only kit: folds and configurations project per key, cross-key pairs commute so* lo^E *localizes, and per-key joins re-interleave. Conjecturally the coupling strength stratifies what survives composition: payload parametricity (free), the indexed product = the grow-only-keys map (a theorem to prove), read coupling — one layer's* app *reads another's state, as in a budgeted cart — (convergence composes; safety needs a monotone-transfer lemma), uniform coupling — key removal, reaching into the value layer the same way for every value type (one generic obligation: a reset element and a* remove $\|$ *v-op policy) — and ad-hoc effect coupling (rich-text marks anchored in a sequence), where nothing is expected to compose automatically. The convergence half is now mechanized (`Metatheory/Product.lean`): the binary heterogeneous product* $D_1 \otimes D_2$ *composes at exactly the `JoinLemma3At` boundary — cross-component pairs commute definitionally, so* lo^E *has no mixed edges and the glued join witness is a plain concatenation* $\iota_1\rho^1 +\!\!+ \iota_2\rho^2$ *(`joinLemma3At_prod`; composite metatheorem `prod_ra_linearizable3_of_honest_reach`, axioms `propext`, `Quot.sound`; consumability demo: a queue* $\otimes$ *counter capstone with zero bespoke proof, `MRDT_Instances/ProductDemo/`). Two boundary findings from the pen-and-paper pass (`Development/COMPOSITION_PENPAPER.md`): reachability does not project (a second-component apply is a version-duplicating stutter for the first's projection), so finished capstones never compose — only per-configuration certificates do, which retroactively forces the factored interface of §13; and two-sided causal-witness re-interleaving is refuted (a realizable four-event cross-*$\xrightarrow{\mathsf{vis}}$ *cycle), with the sound repair — pin one side, re-sort the other — covering products with at most one* rc*-nontrivial component. The safety and* $\approx$ *kits are also mechanized: honesty, `SafetyStep`, and one-sided causal witnesses compose (`Metatheory/Product_Safety.lean`: the pinned-extension lemma, `safetyStepOn_prod`, `causalCanonical_prod_of_one_sided`, composite `prod_version_inv_on_of_one_sided`), and the eq-quotient layer lifts at the pragmatic cut* $\approx_2 = =$ *(`Metatheory/ProductEq.lean`: every `GenericEqQuotient` obligation componentwise, `eqJoinLemma3C_H_prod` glued by concatenation with no congruence chasing, capstone `prod_ra_linearizable_up_to_eq_H` — one quotiented component, one flat, exactly the Peritext shape, with the quotiented side's reachability supplies as premises per the memo's cost-center analysis). Still open: the general* $\approx_1 \times \approx_2$ *lift (no new ideas, double the plumbing — deferred until a second quotiented component exists), the indexed (map-of-MRDTs) generalization, and the L2.5/L3 coupling boundaries, where composition breaking is expected to be as informative as where it holds.*

Open Question 40.12 (the sequential witness at merges)**.** *On every honestly reachable configuration of the embedded-chain RGA, does the event union admit a delivery-respecting enumeration that is also sequentially honest, so that the merged read is the naive buffer's output on it? Caution from adjacent territory: for the rehoming design the analogous "order dependency edges first" discipline is refuted (the `DepComp` refutation, via rehoming non-transitivity). The embed's immutable coordinates remove that refutation's mechanism, but no proof exists in either direction.*

Open Question 40.13 (the chain-price program: possibility and impossibility of retention)**.** *The fooling pair of §23 gives this document its first representation-level impossibility (previous negatives were all specification-level), and it suggests a program: for each intent tier (the RGA reads, Fugue, FugueMax; delete-order preservation), characterize the exact information a tombstone-free state must retain, fooling pairs as lower bounds, capstones as upper bounds. Three concrete instances. (i) Is the full dead chain necessary, or only its order type relative to live stamps? The renumber epoch already exploits rank-compressibility below settled cuts; is the settled gate exactly the boundary of compressibility, or can unsettled retention also be rank-coded against a fooling-pair obstruction? (ii) The FugueMax tag retention (Open Question 40.17) is this program's maximal-intent instance. (iii) Delete sensitivity: is every delete-sensitive read function unimplementable over retention-free states, the general form of "no policy reaches Shesha"?*

Open Question 40.14 (the sided run table, and the batched-apply engineering face)**.** *The one-sided run table (§24.3) is now mechanized (`RunTable.lean`): T-tail (`tail_attachment`), T-repr as a genuine bijection (*tableOf/stateOf *mutually inverse on canonical tables, over labels, sides, and liveness, deliberately not timestamps), T-cmp (`cmpTable_eq_keyLt`), T-walk (`walk_display`), T-mut (split, liveness split, materialize, and the forced coalesce), and the single stability-conditioned clause T-epoch (`runTable_epoch`). What remains theorem-side is the sided run table: its structural core is in place (`SidedRunTable.lean: stail_attachment`, an explicit side field, the band-order comparator seed), and the full sided repr iso and walk mirror the one-sided development. Two engineering faces of the same item, both now half-closed by the shipped artifact: task #104's serializer already turned the projection column from measured-in-model into measured bytes (§38), so what is left there is a delta/op wire format for sync (the concurrent payload column is still in-process) and a transient or batched apply path to remove the interface-inherited $O($live set$)$ per-op copy that dominates apply latency, neither of which touches the verified semantics.*

Open Question 40.15 (FugueMax tag re-mapping)**.** *The re-coding cluster (§24.1) rewrites coordinates at the record's top level. The FugueMax variant embeds keys inside R-entry tags, so an epoch re-map must rewrite coordinates occurring inside tag payloads, applying the stable-prefix factorization under a congruence over the tag structure. The TagsOK finding (§22) raises the stakes: tag wellformedness is convergence-load-bearing, so an ill-formed tag re-map would break convergence itself, not merely display intent. Neither the congruence nor its H2 argument exists yet.*

Open Question 40.16 (generation-discipline dependence of storage arguments)**.** `AnchorsFactorBeyond` *(§24.2) is a theorem under the generation discipline and refutable from bare honest reachability (the countermodel forges the carried split while keeping the minted coordinate honest). How much of the storage layer has this shape? The conjecture: every storage-layer hypothesis that quantifies over carried payloads (op prefixes, tags, in-flight declarations) is dischargeable from generation honesty and refutable without it, so that a uniform metatheorem ("payload facts come from the mint," the storage echo of Part I's* `GenHonest` *shape) would collapse the per-instance discharges. A refutation instance would be equally informative: a payload fact needed by some compaction that generation honesty does not pin.*

Open Question 40.17 (the retention lower bound for tombstone-free maximal non-interleaving)**.** *The FugueMax kernel variant realizes its sibling order by embedding the mint-time successor's key into contested R-entries, at $\Theta(|$origin key$|)$ coordinate cost where plain Fugue pays*

zero (§27). Is that retention a lower bound: must any tombstone-free design achieving maximal non-interleaving carry the right origin's ordering information in its keys? The question has a Myhill–Nerode shape (task #89): distinguish states that demand different verdicts, and count what any key scheme must remember. It is the entropy law of §18 meeting the ordering intent, and the fooling-pair technique of §23 is the designated tool: exhibit two histories whose tag-free states coincide while maximal non-interleaving owes them different orders.

Resolved: criss-cross merges and virtual LCAs. The previous revision asked whether the eight VCs imply that a recursively computed virtual base equals $\sigma(E_1 \cap E_2)$, guessing inclusion-exclusion arguments per datatype and a candidate new store invariant. The answer landed stronger and cheaper than forecast (§14): the covering proposition makes the virtual base's event set *exactly* the intersection from invariants already carried (`StoreInv.origin` plus monotonicity, no new invariant), one fold induction from the per-configuration Join Lemma gives canonicity of the virtual state, and the per-datatype VC surface does not move. The per-datatype question dissolved into the hook the framework already demanded. The residue that remains open is operational, not semantic: keep-set tightness (Open Question 40.5).

Resolved: the self-referential-spec limit. RA-linearizability certifies that a version's state is the fold of some delivery-respecting linearization of its events: convergence to the datatype's *own* `do`-fold, with that fold as the reference. It says nothing about whether the fold's single-replica semantics is the intended one; a defect in `do` appears identically in the concurrent state and its fold, and is certified as agreement. Independent *intent* specifications are therefore a separate obligation, not a corollary of convergence. The question was resolved systematically by the sequential-specification campaign (Part III), which supplies the independent specification for every production RDT; its refutations also correct the earlier reading of the sharpest instance. The survivor reorder exhibited by the campaign's four-op trace (§33) is not intrinsic to tombstone-freedom, it is intrinsic to rehoming: the embedded-chain RGA is tombstone-free, satisfies the sequential list specification in the strongest form (`embed_seq_sound`: the canonical state is the naive buffer, record for record), and preserves delete order (`eUpdate_del_sublist`). The successor question is Open Question 40.12.

41 Mechanization map

Everything above is mechanized in Lean 4 (Mathlib; axioms: `propext`, `Classical.choice`, `Quot.sound`; no `sorryAx` anywhere in the cited chains; refutations by concrete witness follow the SPOT convention, `native_decide` adding the compiler axioms) in the Sal repository. Files are relative to `Sal/ConditionedMRDTs/` unless they carry an explicit `Sal/` prefix.

Artifact	Lean	File
Part I: framework and metatheorems		
signature, ternary lo	`MRDTSig`, `ConditionedMRDTSig`	`Framework/MRDTSig.lean`
store, DAG, transitions	`Configuration`, `Step3`	`Framework/ExecutionModel.lean`, `Metatheory/LCA_Lemma.lean`
Lemma 4.1	`lca_events_of_storeInv`	`Metatheory/LCA_Lemma.lean`
σ/lo^E layer (merge-independent)	`UpdateVCs`, `IsCanonicalState`	`Framework/Sigma_LoOn3.lean`
the eight VCs	`CoreVCs3CD`, `FeasibleDeltaVCs3`, `CDVC3`	`Framework/VC_Set.lean`
Theorem 8.1 + bridges	`ra_linearizable_of_core_feasible_cd3`	`Metatheory/Adequacy.lean`
defeater vs. 2-OP (§4.2)	`no_inter_lca_2op_rem_peel_of_defeater`	`Refutations/InterLca2op_Defeater_Arbiter.lean`
closure-indexed contract (§8)	`JoinLemma3C`, `no_proper_back_block`	`Metatheory/JoinLemma3C.lean`
closure-indexed adequacy + instances	`ra_linearizable3_of_joinC`, `ConditionedContract`	`Metatheory/ConditionedContract.lean`
all 9 flat discharges via the contract	`*_adequate_viaContract` (9)	`Development/AllMRDTs_viaContract.lean`
per-configuration join hook	`JoinLemma3At`, `goodConfig3_merge_at`	`Framework/VC_Set.lean`, `Metatheory/Adequacy.lean`
honest-reachability metatheorem	`HonestReach`, `ra_linearizable3_of_honest_reach`	`Metatheory/HonestReach.lean`
generic honesty shape	`GenHonest`, `AppHonest`	`Metatheory/GenHonest.lean`
witness-class join lemmas	`JoinLemma3AtW`	`Metatheory/WitnessClass.lean`
generic safety (causal witness)	`SafetyStep`, `version_inv_of_causal_canonical`	`Metatheory/GenericSafety.lean`
escrow metatheorem	`Measured`, `escrow_version_inv`	`Metatheory/EscrowSafety.lean`
product composition (raw kit)	`joinLemma3At_prod`, `prod_ra_linearizable3_of_honest_reach`	`Metatheory/Product.lean`
product safety kit	`safetyStep0n_prod`, `prod_version_inv_on_of_one_sided`	`Metatheory/Product_Safety.lean`
product $\approx$-lift	`eqJoinLemma3C_H_prod`, `prod_ra_linearizable_up_to_eq_H`	`Metatheory/ProductEq.lean`
observational quotient $D \mapsto D_{\approx}$	`QSig`, `qapplicable`	`Metatheory/GenericEqQuotient*.lean`
canonical-witness layer, raw-$\approx$ target	`IsRALinearizable3Eq`, `RA_linearizable_up_to_eq_H`	`Metatheory/GoodConfig3H.lean`
flat collapse of the framework	`eqJoinH_of_joinC`, `flat_ra_linearizable3_eq`	`Metatheory/FlatGeneric_Bridge.lean`
LWW merge needs timestamps (OQ 40.9)	`lww_merge_needs_timestamps`	`Refutations/LWW_Merge_Needs_Timestamps.lean`
CDVC3 independent of core+delta (OQ 40.2)	`cdvc3_not_derivable_from_core_delta`	`Refutations/CD_Not_Derivable_Ternary.lean`
impossibility kill-tests	—	`Refutations/Impossibility.lean`
findings journal	—	`Development/MRDT_METATHEORY_DRAFT.md`
Part I: the runtime arc (§14–§16)		
virtual-merge step relation	`Step3V` (base embedding + `mergeVirtual`)	`Metatheory/LCA_Lemma.lean`
covering proposition (Prop. 14.1)	`mca_events_cover`	`Metatheory/LCA_Lemma.lean`
virtual fold induction	`virtualLCAState_canonical`, `goodConfig3_mergeVirtual_at`	`Metatheory/Adequacy.lean`
honest reachability over `Step3V`	`HonestReachV`, `ra_linearizable3_of_honest_reachV`	`Metatheory/HonestReach.lean`
single-MCA picks refuted (criss-cross pins)	`tlf_pick_u_resurrects_y`, `tlf_pick_v_resurrects_x`	`Metatheory/VirtualLCA_Spot.lean`
H-layer + flat catalogue over `Step3V`	`flat_ra_linearizable3_eq_V`	`Metatheory/GoodConfig3H_V.lean`, `Metatheory/FlatGeneric_Bridge_V.lean`
queue over virtual merges (§14)	`queue_ra_linearizable3_V`	`MRDT_Instances/MergeableQueue/`
rehoming Eq capstone over `Step3V` (stress test)	`rga_ra_linearizable3_eq_V`	`MRDT_Instances/RGA_Rehoming/RA_Lin_V.lean`
commit GC: keep set + safety (§15)	`keepSet`, `gc_safety`, `lca_not_dropped`	`Metatheory/GC_Safety.lean`
GC under virtual merges	`mcasClosure`, `keepSetV`, `gc_safetyV`	`Metatheory/GC_Safety.lean`
bounded-state GC: clock swap (§15)	`clockPrune`, `ClockCoherent`, `clockLCA_sound`, `gc_safety_bounded`	`Metatheory/GC_BoundedState.lean`
SettledAt discharged from the frontier (§16)	`AllHeardSince`, `settledAt_of_allHeard`	`Metatheory/EvidenceDischarge.lean`
SettledAt def + stability bundle + metatheorem	`SettledAt`, `StabilityVC`, `stability_simulation`, `stability_ra_inherited`	`Metatheory/Stability_VC.lean`
OR-set stability instance	`orStabilityVC`, `ORSet_stability_reads`	`MRDT_Instances/ORSet/ORSet_Stability.lean`
static drop set refuted	`static_drop_unsound`	`MRDT_Instances/ORSet/ORSet_Stability.lean`
Part I: instances (Table 1)		
catalogue instance theorems	all instance theorems	`MRDT_Instances/` (one directory per RDT)
the flat instances, generically	`*_ra_linearizable3_eq` (11)	`MRDT_Instances/` (per-RDT)
bounded-counter safety	`bc_version_inv`, `bc_value_nonneg`	`MRDT_Instances/BoundedCounter/`
mergeable queue (direct join)	`q_join_at`, `queue_ra_linearizable3`	`MRDT_Instances/MergeableQueue/`
FWW register (payload arbitration)	`fww_version_min`	`MRDT_Instances/FWWRegister/`
BudgetCart (gated safety)	`BCart_ra_linearizable3_eq`, `bcart_version_inv_gated`	`MRDT_Instances/BudgetCart/`
Part II: the embedded-chain family		
model-layer kernel	`rc_non_comm'`, `display_iff_chainBefore`, `subtree_convex`	`Sal/MRDTs/RGA_Embed/`
embedded-chain RGA (queue route)	`embed_ra_linearizable3`, `e_fold_canon`	`MRDT_Instances/EmbedRGA/`
Elias-δ inheritance corollary	(capstone at the δ-flip)	`MRDT_Instances/EmbedRGA/EmbedRGA_EliasDelta.lean`
compaction theorem	`rga_read_eq_embed_read`	`MRDT_Instances/EmbedRGA/EmbedRGA_ReadEquiv.lean`
compaction order core	`visible_lt` = chain-lex	`Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean`
sided chain-lex kernel	`sdisplay_iff_schainBefore`, `schainBefore_liftR`	`Sal/MRDTs/RGA_Embed/Sided_ChainLex.lean`
sided RGA + per-policy intent	`sided_embed_ra_linearizable3`, `sFold_liftOp`	`MRDT_Instances/SidedRGA/`
Fugue/FugueMax vs. W–K Def. 4	`fugue_not_maximally_noninterleaving`, `fuguemax_same_origin_low_first`	`MRDT_Instances/SidedRGA/`, `Sal/MRDTs/RGA_Embed/SidedMax_ChainLex.lean`
Peritext on the embed kernel	`peritextEmbed_ra_linearizable3`, `renderIds_del`	`MRDT_Instances/Peritext_Embed/`
Part II: Theorem 9 and the storage layer (§22–§24)		
traversal theory, interval form	`schain_ext_after_is_R`, `schain_ext_before_is_L`	`Sal/MRDTs/RGA_Embed/Sided_Traversal.lean`
forward NI, FugueMax	`fuguemax_forward_ni`	`MRDT_Instances/SidedRGA/SidedRGA_NonInterleaving.lean`
forward NI, plain Fugue (as stated)	`fugue_forward_ni`	`MRDT_Instances/SidedRGA/SidedRGA_Fugue_ForwardNI.lean`
Theorem 9 capstone (§22)	`fuguemax_maximally_noninterleaving`	`MRDT_Instances/SidedRGA/SidedRGA_Backward.lean`
FugueMax convergence (TagsOK)	`fuguemax_ra_linearizable3`, `f_fold_canon`	`MRDT_Instances/SidedRGA/SidedRGA_FugueMax_RA_Lin.lean`
sibling-splice fooling pair (§23)	`sibling_splice_no_merge_function`	`Refutations/SiblingSplice_Fooling.lean`
re-coding cluster, T1–T3 + collapse (§24.1)	`eRecode_applySeq`, `eRecode_reads_identical`, `eRecode_ext_global_collapse`	`MRDT_Instances/EmbedRGA/EmbedRGA_Recoding.lean`
SettledAt bridge, remap species	`eRecode_settled_bridge`	`MRDT_Instances/EmbedRGA/EmbedRGA_Stability_Bridge.lean`
anchors factor at the cut	`anchorsFactorBeyond_of_anchored`, `eRecode_settled_bridge_honest`	`MRDT_Instances/EmbedRGA/EmbedRGA_AnchorsFactor.lean`
verified concrete compaction (§24.2)	`compactEliasDelta_settled_reads` + in-flight SPOTs	`MRDT_Instances/EmbedRGA/EmbedRGA_CompactEliasDelta.lean`
spine fusion (§24.2)	`compactThenFuse_reads`	`MRDT_Instances/EmbedRGA/EmbedRGA_Fusion.lean`
merge-vs-remap (VC-S4, §24.2)	`merge_remap_congr`	`MRDT_Instances/EmbedRGA/EmbedRGA_MergeCongr.lean`
multi-epoch composition (§24.2)	`StablePrefixMap.comp`, `multiEpoch_settled_reads`, `naive_composition_collides`	`MRDT_Instances/EmbedRGA/EmbedRGA_MultiEpoch.lean`
nested settled cuts ((*)'s closed halves)	`nested_cuts_settled`, `rED_hocc_fresh`	`MRDT_Instances/EmbedRGA/EmbedRGA_CompatChain.lean`
run table, one-sided (§24.3)	`tail_attachment`, `canonical_table0f`, `cmpTable_eq_keyLt`, `walk_display`, T-mut	`Sal/MRDTs/RGA_Embed/RunTable.lean`, `MRDT_Instances/EmbedRGA/EmbedRGA_RunTable.lean`
run table, sided core (full iso owed, OQ 40.14)	`stail_attachment`, `sided_keyLt_iff_schainBefore`	`Sal/MRDTs/RGA_Embed/SidedRunTable.lean`
Part III: the sequential-specification campaign		
tier 1: eleven flat RDTs	`*_seq_sound` (per RDT)	`MRDT_Instances/SeqSpec_Flat.lean`
tier 2: mergeable queue	`queue_seq_sound`	`MRDT_Instances/MergeableQueue/MergeableQueue_SeqSpec.lean`
tier 3: embedded-chain RGA	`embed_seq_sound`	`MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec.lean`
tier 3, two-sided: sided RGA	`sided_seq_sound`	`MRDT_Instances/SidedRGA/SidedRGA_SeqSpec.lean`
tier 3 corollary: RGA$_\uparrow$	`rga_seq_read_eq_buffer`	`MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec_RGA.lean` (standalone)
tier 4: fused Peritext	`peritextEmbed_seq_sound`	`MRDT_Instances/Peritext_Embed/PeritextEmbed_SeqSpec.lean`
render fix theorem	`renderIds_del`	`MRDT_Instances/Peritext_Embed/PeritextEmbed.lean`
negative: rehoming RGA	`rehoming_seq_refuted`	`MRDT_Instances/RGA_Rehoming/RGA_SeqSpec_Refuted.lean`
negative: rehoming Peritext	`fused_delete_reformats_survivor`	`MRDT_Instances/Peritext_Rehoming/Peritext_Read.lean` §10
negative: Shesha merge	`Shesha_Rows_Refuted`	`MRDT_Instances/Shesha/Shesha_Rows_Refuted.lean`
The runtime pillar: engineering artifacts (unverified, mirror the verified kernel)		
p2p runtime replica (§37)	`DistributedReplica`, `compactStable`	`runtime/src/replica.js`
run-table serializer (§38)	`encode/decode`, `accountingBits`	`runtime/src/serialize.js`
git persistence + p2p demo (§37)	`persist/load`, `WsTransport`	`p2p-demo/src/`
cross-system benchmark (§38)	the matrix	`benchmarks/`
SMT discharge — a *tool*, not a theorem (§39)	z3 translation-validation of the 8 flat VCs	`smt/`

The update-side layer (lo^E, convergence, σ) is merge-independent — it is stated over the update signature $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{rc} \rangle$ alone — and is shared with the binary (CRDT) specialization of the theory; in the repository it lives in `Sal/CRDTs/Metatheory/`, the directory that also holds the binary exactness results quoted in Open Question 40.2. The production data types of Table 1 are mirrored faithfully from `Sal/MRDTs/*` (encoding deviations documented per instance); their original 24-VC discharges are untouched.

The foundations live under `Framework/`, the metatheorems under `Metatheory/`, the machine-checked negative results under `Refutations/`, and the per-RDT conditioned instances under `MRDT_Instances/`; findings journals and superseded routes remain under

`Development/`, which nothing imports. Each cited theorem is kernel-checked with axioms in {`propext`, `Classical.choice`, `Quot.sound`} (no `sorryAx`). They import one linearization file carrying two unrelated `sorry`s that the cited theorems do not transitively depend on; `Development/CONDITIONED_METATHEORY_PLAN.md` is their roadmap.